JAVA

Sudhir Lama

TABLE OF CONTENTS

01 Basics of Programming in Java

02 Object Oriented Programming in Java

03 Building Components using Swing

04 Distributed Networking Programming

05 Database Connectivity

06 Servlets and JSP

07 Advanced Topics in Java

Basics of Programming in Java 01

WHAT IS JAVA?

- Java is a high-level, object-oriented programming language known for its portability, security and simplicity.
- It was first developed by a company called Sun Microsystems in the mid-1990s, and it’s now owned by Oracle
- Java is designed to be platform-independent, meaning code written in Java can run on any device that has the Java Virtual Machine (JVM) installed, making it a "write once, run anywhere" language.
- Known for its robustness, Java is suitable for creating everything from small desktop applications to complex, server-based systems.
- It powers a wide range of applications, including mobile devices, gaming, navigation systems and e-commerce platforms.

Features of Java

Object Oriented Programming(OOP)

- Java is an object-oriented programming language, structured around the concepts of classes and objects. In this framework, objects hold data and include methods that manipulate this data, while classes serve as blueprints that define the structure and behavior of these objects.

Platform Independent

- Java is frequently described as a "write once, run anywhere" language, indicating that Java code needs to be written only once and can run on various operating systems, such as Windows, Mac, or Linux, without modification.
- Java programs are compiled into bytecode by the Java compiler, which is then executed by the Java Virtual Machine (JVM). The JVM translates bytecode into machine code, ensuring compatibility across different operating systems and enabling seamless cross-platform functionality.

High Level

- Java is a high-level programming language, offering greater readability and ease of writing compared to low-level languages like assembly. It also features a comprehensive standard library that includes numerous functions and data structures.

Features of Java

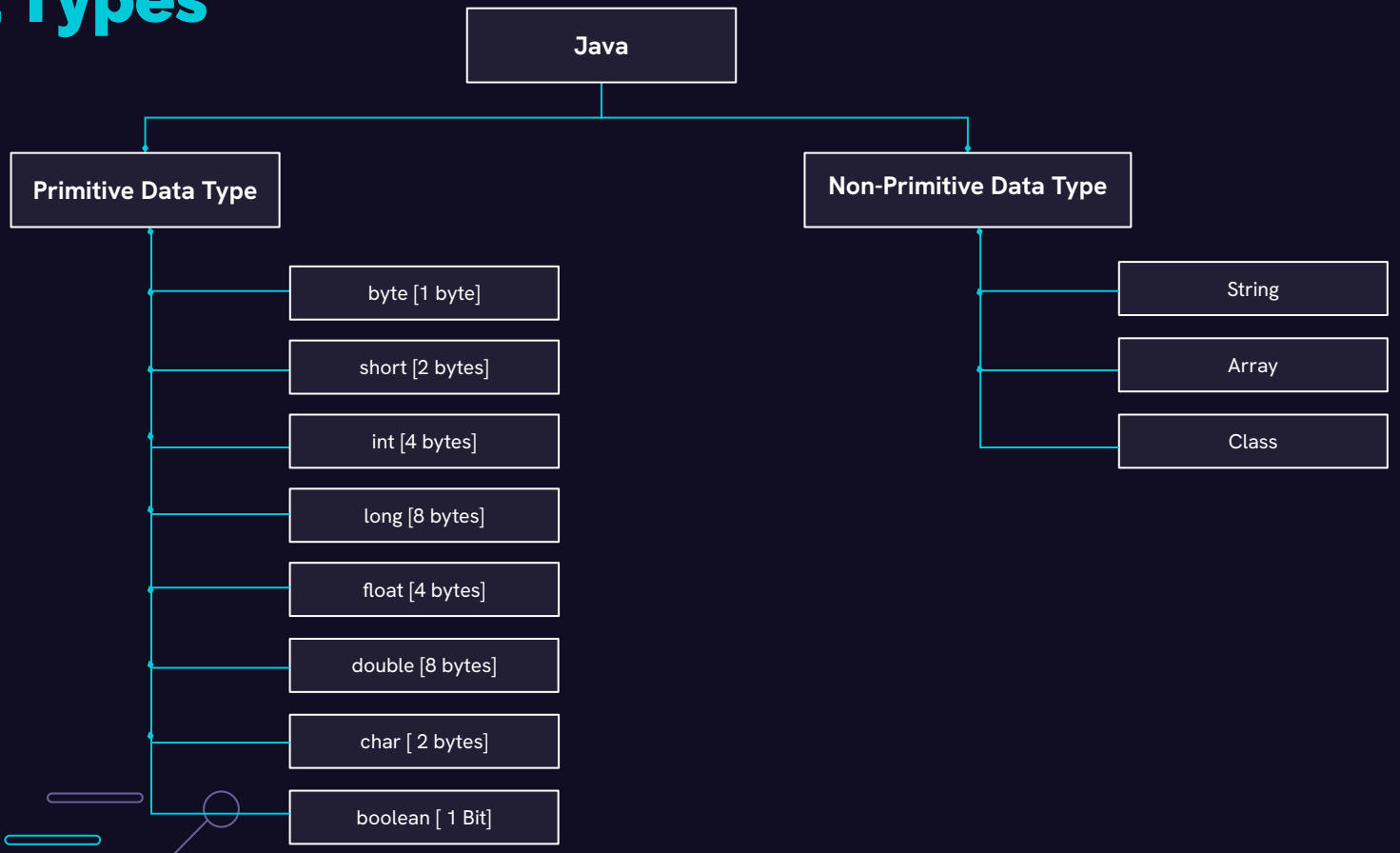
Robust and Secure

- Java is known for being a robust language, which means it can handle many types of errors that might happen during program execution.
- Java includes built-in security features to prevent common programming errors and safeguard data. For example, Java performs code checks for potential issues before execution, and its memory management system minimizes risks like memory leaks. For example, you can use the "final" keyword to make a variable read-only, which helps prevent accidental modification.
- Java emphasizes exception handling and type safety to enhance code reliability. Using try/catch blocks, exceptions can be managed to avoid program crashes. Additionally, Java's garbage collector automatically reclaims unused memory, helping to prevent memory leaks.

Multithreading Support

- Multithreading means that Java can execute multiple tasks (threads) at the same time within the same program to allow for concurrent processing. This is especially useful for applications that require multitasking, like games or software that requires simultaneous operations.
- For example, if you have a Java program that downloads a file and updates the user about the download status at the same time, it can do both without pausing either task.

Data Types


Java
Primitive Data Type
Non-Primitive Data Type
byte [1 byte]
short [2 bytes]
int [4 bytes]
long [8 bytes]
float [4 bytes]
double [8 bytes]
char [2 bytes]
boolean [1 Bit]
String
Array
Class

Data Types

- **Data types specify the different sizes and values that can be stored in the variable.**
- **Primitive Data Type**
  - Most basic data type which are the building blocks of data manipulation.
  - It store actual value and there are no methods or properties.
  - Limited to holding simple data type and default values depends on types.
- **Non-Primitive Data Type.**
  - It is reference data type and often hold the complex object rather than simple values.
  - Store references or memory addresses that point to the location of the object in memory.
  - Have methods and properties.
  - Can hold multiple or complex data.

▶ Wrapper class in Java

- A wrapper class in Java is a class that wraps a primitive data type into an object
- It provides the mechanism to convert primitive to object and object to primitive data type.
- Java provides a wrapper class for each of the eight primitive data types, allowing them to be treated as objects
- This object conversion is useful when objects are required in contexts such as collections or methods expecting object types.

Need of Wrapper class

- When object representation of primitive types is required, such as in object-oriented programming.
- When working with Java collections (since they only accept objects).
- When null values need to be represented or when utility methods from the wrapper class are needed for type conversion and manipulation.

Advantages of Wrapper class

- Collections Only Accept Objects.
- On object data we can call multiple methods compareTo(), equals(), toString()

▶ Wrapper class in Java

- **Advantages of Wrapper class**
  - Cloning Works Only with Objects
  - Null Values Allowed for Objects
  - Serialization Supports Only Objects
- **Java provides the following wrapper classes for its primitive types:**
  - byte → Byte
  - short → Short
  - int → Integer
  - long → Long
  - float → Float
  - double → Double
  - char → Character
  - boolean → Boolean

▶ Autoboxing

- Autoboxing in Java refers to the automatic conversion of a primitive data type into its corresponding wrapper class object
- The conversion happens without requiring explicit intervention by the programmer.
- It makes the java code clean and easy to use.

Eg :

```
int num = 10;

double decimal = 7.75;

char letter = 'A';

// Autoboxing: Converting primitives to wrapper objects

Integer intObj = num;     // Integer wrapper object

Double doubleObj = decimal; // Double wrapper object

Character charObj = letter;  // Character wrapper object
```

▶ Unboxing

- Unboxing is the process of converting a wrapper class object into its corresponding primitive data type
- The Java compiler automatically converts a wrapper object into its equivalent primitive type, making the code simpler and more intuitive.

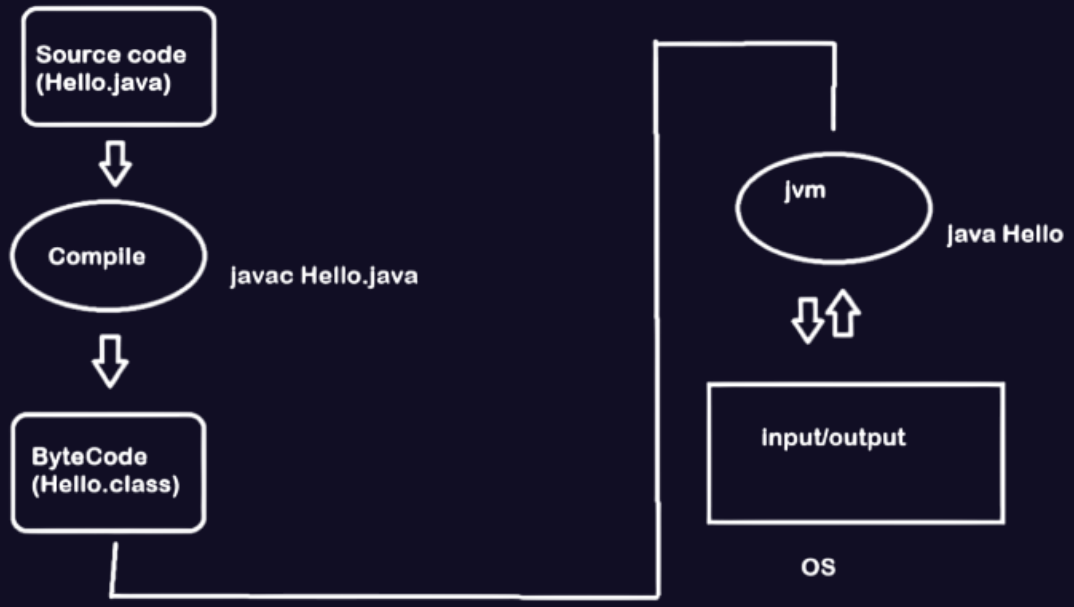
Eg :

```
Integer obj = 20;  // Wrapper object

int num = obj;     // Unboxing

System.out.println("Primitive int: " + num);
```

Java Architecture

- Java architecture is an integrated system designed to support the development and execution of Java applications across multiple platforms.
- This architecture is based on three primary components
  - Java Development Kit (JDK)
  - Java Runtime Environment (JRE)
  - Java Virtual Machine (JVM)
- It combines the process of compilation and interpretation and explains the various process involved for formulation the Java program.
- The Java code is first converted into bytecode by the Java Compiler.
- JVM then convert the bytecode into machine code.
- Finally the machine code is execute by the Machines.
- Java architecture provides security , portability and efficiency.

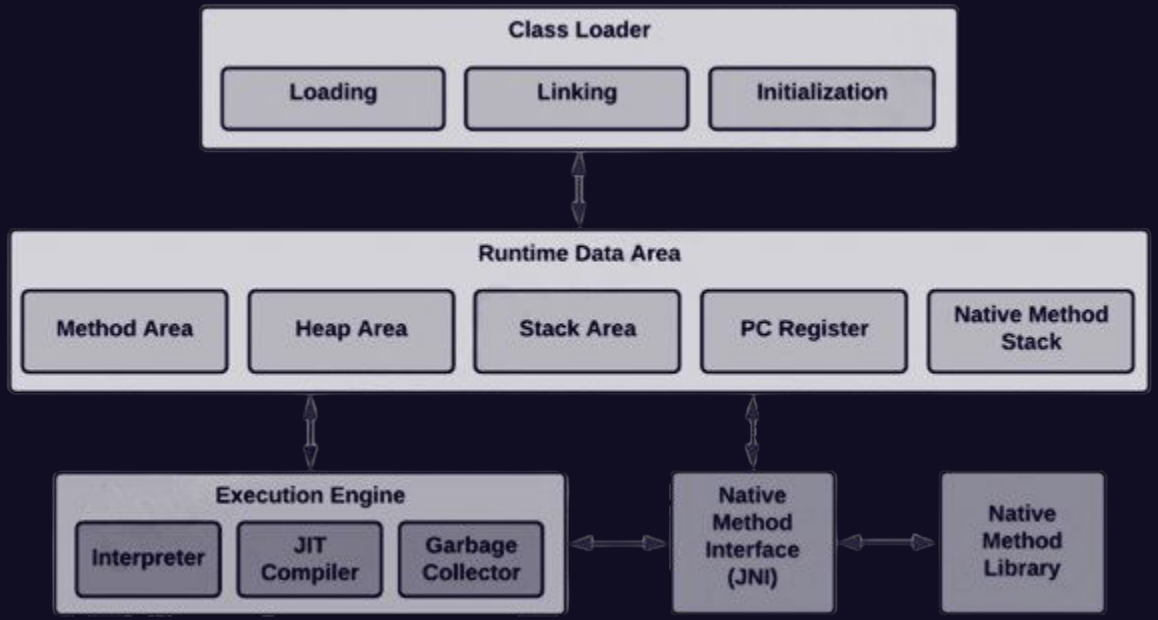
▶ Java Architecture and Components



▶ Components of Java Architecture

- Java architecture is a collection of components : JVM, JRE and JDK
- **Java Development Kit (JVM)**
  - JVM is a key component of the Java platform that provides an environment for running Java programs on various platforms.
  - The JVM interprets bytecode (platform-neutral code) into machine code.
  - It gives an environment to execute the java programs.
  - It loads , verifies and executes the code and provides runtime environment.

JVM Architecture



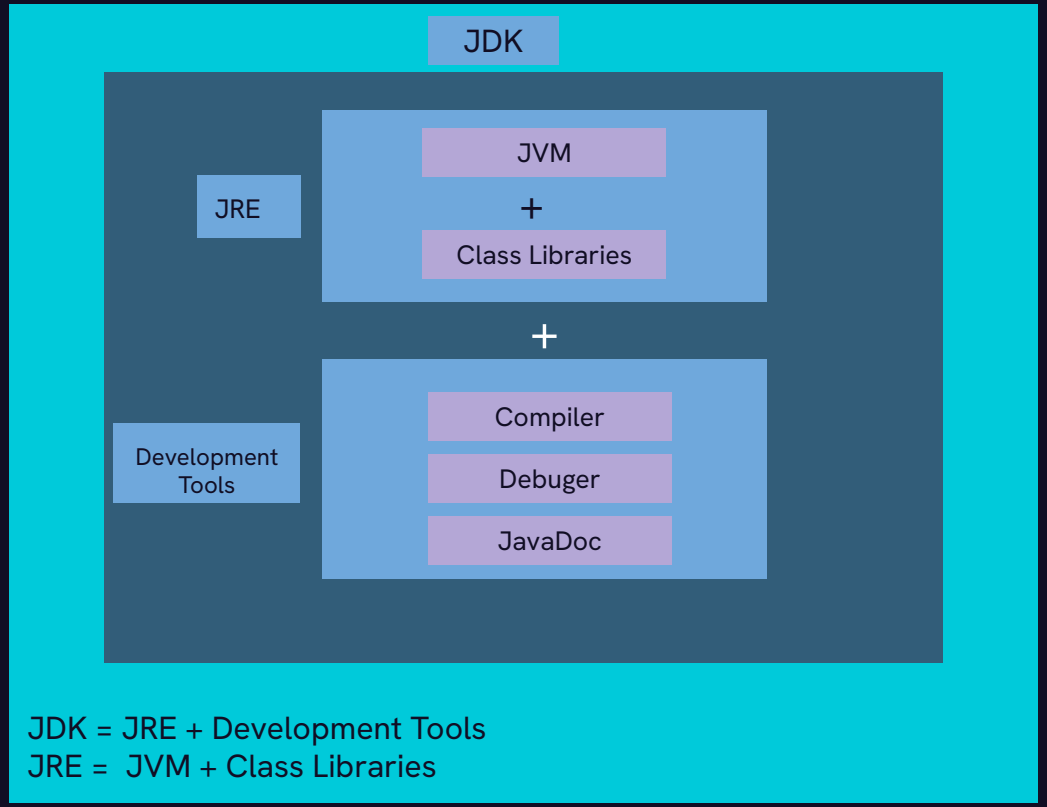
Components of Java Architecture

- **Java Runtime Environment (JRE)**
  - It is a software which builds a runtime environment in which Java programs can be executed
  - It contains libraries and software needed by Java programs to run.
  - It takes Java code, integrates it with the required libraries, and then starts the JVM to execute it.
  - It is the underlying technology that communicates between the Java program and the operating system which enables the Java program to run correctly.
  - It ensures Java applications have everything they need to perform their best.
  - JRE is for running programs not for developing it.
- Components of JRE
  - Java Virtual Machine(JVM)
  - Java Class Libraries.
  - Runtime Environment.

Components of Java Architecture

- **Java Development Kit (JDK)**
  - JDK is a software development environment which is used to develop Java applications.
  - It physically exists and contains JRE and development tools.
  - It provides the tools and libraries necessary for compiling, debugging and executing Java programs.
  - It is the cornerstone for Java developer as it allows Java programmers to write Java programs and convert them to format that can be execute by the JRE and JVM.
- Components of JDK
  - Compiler
  - Debugger
  - Java Runtime Environment.

Components of Java Architecture



▶ Java platform independent?

- Java is platform-independent as it is compiled to a bytecode that can be run on any device that has a Java Virtual Machine (JVM)
- It is designed to allow code to be written once and executed on any operating system or hardware without modification.
- During compilation, Java code is converted into a platform-neutral format called bytecode, which is not tied to any specific operating system or hardware.
- Bytecode is executed by the Java Virtual Machine (JVM), which translates it into machine code specific to the operating system. This ensures consistent execution across platforms.
- Java programs operate independently of the underlying operating system by relying on the JVM for execution, enabling platform neutrality.
- Programs developed in Java can run seamlessly on various platforms, such as Windows, macOS, Linux, or mobile devices, as long as a compatible JVM is available.

Java Classpath

In Java, the classpath is an environment variable or a parameter used by the Java Virtual Machine (JVM) and Java compiler to locate classes and packages when running or compiling a Java program. It tells the JVM where to look for user-defined classes, libraries, and other resources required to run a Java application.

Importance of Classpath

- Class Loading

  The JVM uses the classpath to locate the classes and resources required during the execution of the Java program

- Dependency Management

  By setting the classpath, dependencies such as external libraries and third-party JAR files are easily linked to the project.

- Flexibility

  Developers can control where the JVM looks for classes, making it easier to organize and manage large projects with external dependencies.

Java Control Structures

Java control structures are used to control the flow of execution in a program. These structures determine the order in which statements are executed based on certain conditions or loops.

Java provides several control structures, which are categorized as:

- **Decision-Making Statements**
  - if Statement

    The if statement is used to execute a block of code if the given condition is true.

```
if (condition) {
    // code to be executed if condition is true
}
```

Java Control Structures

- If-else Statement

Executes one block of code if the condition is true and another block if it is false.

```
if (condition) {
    // code if condition is true
} else {
    // code if condition is false
}
```

Java Control Structures

- if-else if Statement

  This is used when there are multiple conditions to be checked.

```
if (condition1) {
    // code for condition1
} else if (condition2) {
    // code for condition2
} else {
    // code if none of the conditions are true
}
```

Java Control Structures

switch Statement

The switch statement is used to execute one out of multiple possible blocks of code based on a variable's value.

```
switch (expression) {
    case value1:
        // code for value1
        break;
    case value2:
        // code for value2
        break;
    default:
        // code if no case matches
}
```

Java Control Structures

Looping Statements

Looping structures allow repetitive execution of a block of code until a specified condition is met.

for Loop

The for loop is used when the number of iterations is known in advance. It has three parts: initialization, condition and increment/decrement.

```
for (initialization; condition; increment/decrement) {
    // code to be executed
}
```

```
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}
```

Java Control Structures

- while Loop

The while loop runs as long as the condition is true. It checks the condition before executing the loop body.

```
while (condition) {
    // code to be executed
}
```

```
int i = 0;
while (i < 5) {
    System.out.println(i);
    i++;
}
```

Java Control Structures

do-while Loop

The do-while loop is similar to the while loop, but it checks the condition after the loop has executed, ensuring that the code inside the loop runs at least once.

```
do {
    // code to be executed
} while (condition);
```

```
int i = 0;
do {
    System.out.println(i);
    i++;
} while (i < 5);
```

Java Control Structures

Jump Statements

Jump statements are used to control the flow of execution by transferring control to another part of the program.

break statement

The break statement is used to exit from a loop or switch statement prematurely.

```
break;
```

```
for (int i = 0; i < 5; i++) {
    if (i == 3) {
        break; // exit the loop when i equals 3
    }
    System.out.println(i);
}
```

Java Control Structures

- continue statement

The continue statement is used to skip the current iteration of a loop and proceed with the next iteration.

```
continue;
```

```
for (int i = 0; i < 5; i++) {
    if (i == 3) {
        continue; // skip when i equals 3
    }
    System.out.println(i);
}
```

Java Control Structures

- return statement

The return statement is used to exit from a method and optionally return a value. If no value is returned, it simply exits the method.

```
return;

return value;
```

```
public int add(int a, int b) {

    return a + b; // returns the sum

}
```

Java Collections

- Java collections provide an architecture to store and manipulate the group of objects.
- It provides various classes and interfaces for representing and managing collections of data in Java efficiently.
- It is the powerful tool for managing and processing data.
- Key Interface in Java Collections are List, Set, Queue,Deque and Map
- Commonly used class are ArrayList, LinkedList, Hashset, Treeset, Hashmap and Treemap.
- The readymade classes simplify the coding.
- Adding elements , iterating, removing elements and sorting are the basic operations.

Array

- An array is a collection of similar types of data.
- Used to store multiple values in a single variable, instead of declaring separate variables for each value.
- It is fixed in size.
- Elements are accessed via an index(0-index);
- No built in method for for common operations(eg: add/remove)

```
// Declaration
int[] numbers;
// Initialization
numbers = new int[5];
// Combined
int[] numbers = {10, 20, 30, 40, 50};
```

Array

- A fixed-size data structure used to store elements of the same data type.
- Fixed at the time of declaration and cannot be changed.
- Stores primitives (e.g., int, double) or objects
- Faster for operations due to its simplicity.
- Less flexible; you must know the size beforehand.
- Memory allocation is fixed and cannot change during runtime.
- Does not provide built-in methods for manipulation.
- Does not require importing any packages.
- **Declaration:**

  int[] numbers = new int[5];

Arraylist

- A resizable collection class in java.util package that implements the List interface.
- grows or shrinks as elements are added or removed.
- Stores only objects (autoboxing converts primitives like int to Integer).
- Slightly slower due to dynamic resizing and additional features.
- More flexible; size adjusts automatically.
- Provides rich APIs like add(), remove(), contains(), etc.
- Requires importing java.util.ArrayList.
- **Declaration:**

  ArrayList<Integer> roll= new ArrayList<Integer>();

String

- A String is a sequence of characters, treated as an object in Java.
- It is immutable (cannot be changed after creation).
- Java provides a robust and flexible API for handling strings, allowing for various operations such as concatenation, comparison and manipulation.

```
//Declaration

String text = "Hello, World!";

//Concatenation

String fullName = “Ram” + “Prasad”;

//Comparison

String first = “Kathmandu”

String second = “Kathmandu”

Boolean result = first.equals(second)

//Length of String

String name = “Nepal”

Int length = name.length();
```

Java Package

- Java package is a namespace that organizes classes and interfaces.
- It prevents naming conflicts and helps in organizing code efficiently.
- Its helps to avoid the name clashes, provide control access and simplified for locating and using of classes.
- Built in package and User defined package are the two types of Java Package.
- Popular Built in package:
  - java.lang: Core classes (e.g., String, Math, Object).
  - java.util: Utility classes (e.g., ArrayList, HashMap).
  - java.io: Input/Output classes (e.g., File, BufferedReader).
  - java.net: Networking classes (e.g., URL, Socket).
- User defined packages are the custom package build by developers to manages the classes.

User-defined package

- A user-defined package is a custom package created by a programmer to organize and group related classes, interfaces, and sub-packages.
- It helps in maintaining code modularity, reusability, and reducing name conflicts in large projects.

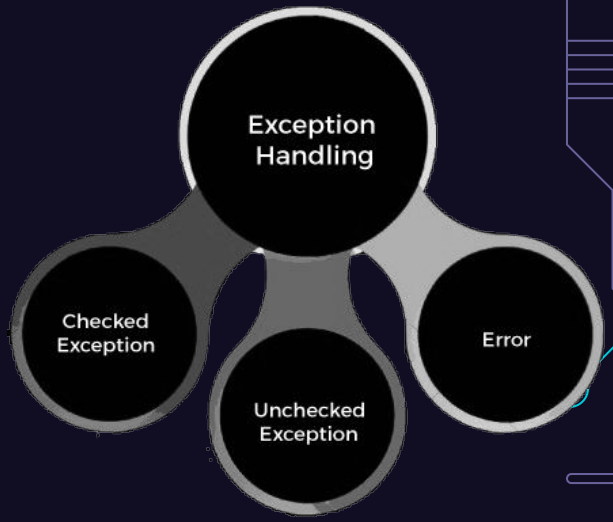
```java
//Create package
package custompackage;
public class MyClass {
    public void showMessage() {
        System.out.println("Welcome to User-Define Packages");
    }
}
```

User-defined package

```
//Access package
import mypackage.MyClass;
public class Hello{
    public static void main(String[] args) {
        MyClass obj = new MyClass();
        obj.showMessage();
    }
}
```

Exception handling

- Exception handling in Java refers to the process of responding to runtime errors (exceptions) to ensure that the program continues to function as intended.
- This is done using
  - try catch block
  - finally block
  - The try block contains the code that might cause an error, catch handles the error if it occurs, and finally ensures certain code runs no matter what (like cleanup actions).
- Checked Exception
  - Exceptions that are checked at compile-time.
  - Must be handled using try-catch or declared using throws.
  - Example: IOException, SQLException, FileNotFoundException



Exception handling

- Unchecked Exception
  - Exceptions that are checked at runtime.
  - Do not require explicit handling.
  - Example: NullPointerException, ArrayIndexOutOfBoundsException, ArithmeticException, ClasscastException.

- Errors
  - Errors represent exceptional conditions that are not expected to be caught under normal circumstances
  - They are typically caused by issues outside the control of the application, such as system failures or resource exhaustion
  - Errors are not meant to be caught or handled by application code
  - Example: OutOfMemoryError, StackOverflowError, .NoClassDefFoundError

Custom Exception

- A custom exception is a user-defined exception in Java that extends either the Exception class (for checked exceptions) or the RuntimeException class (for unchecked exceptions).
- It is used when the standard exceptions in Java do not adequately describe a specific error scenario for our application
- Custom exceptions provide a meaningful and context-specific way to handle application errors.
- Custom Checked Exceptions:

```
//Define the custom Excepton

public class InvalidAgeException extends Exception {

    public InvalidAgeException(String message) {

        super(message); // Pass the message to the parent Exception class

    }

}
```

Custom Exception

```java
//Use the custom Exception

public class Main {

    public static void checkAge(int age) throws InvalidAgeException {

        if (age < 18) {

            throw new InvalidAgeException("Age must be 18 or above.");

        } else {

            System.out.println("Age is valid!");

        }

    }

    public static void main(String[] args) {

        try {

            checkAge(16); // This will throw the exception

        } catch (InvalidAgeException e) {

            System.out.println("Caught Exception: " + e.getMessage());

        }

    }

}
```

Concurrency

- Concurrency refers to the ability of a system to handle multiple tasks at the same time or in overlapping time periods, without necessarily executing them simultaneously.
- It is a fundamental concept in modern programming, especially for handling multiple tasks in real-time.
- In Java, concurrency is achieved through the use of threads.
- A thread is a lightweight process that allows for concurrent execution within a program.
- Two main ways to create and run thread.
  - By extending the Thread class.
  - By implementing the Runnable interface.

Concurrency

```
//Extending Thread Class
class MyThread extends Thread {
    public void run() {
        System.out.println(Thread.currentThread().getId() + " is executing run method.");
    }
}

public class ThreadExample {
    public static void main(String[] args) {
        MyThread thread1 = new MyThread();
        thread1.start();  // Starts a new thread

        MyThread thread2 = new MyThread();
        thread2.start();  // Starts another thread
    }
}
```

Concurrency

```
//Implementing Runnable Interface
class MyRunnable implements Runnable {
    public void run() {
        System.out.println(Thread.currentThread().getId() + " is executing run method.");
    }
}
public class RunnableExample {
    public static void main(String[] args) {
        MyRunnable runnable = new MyRunnable();
        Thread thread1 = new Thread(runnable);
        thread1.start();  // Starts a new thread

        Thread thread2 = new Thread(runnable);
        thread2.start();  // Starts another thread
    }
}
```

Thread State

- **New**:
  - A thread is in this state when it is created but not yet started.

    Example: Thread thread = new Thread();
- **Runnable**:
  - A thread is in the runnable state when the start() method is called. A thread in the runnable state is eligible for running but may not be running at the moment due to the CPU scheduler's decision.
- Blocked:
  - A thread enters the blocked state when it wants to access a synchronized resource but another thread is holding a lock on it.
- Waiting:
  - A thread enters the waiting state when it is waiting for another thread to perform a particular action (like Thread.wait())
- Timed Waiting:
  - A thread is in this state if it is waiting for a specific amount of time (e.g., Thread.sleep()
- Terminated:
  - A thread enters the terminated state when its run() method has finished or when it has been terminated manually.

Thread Priority

- Java help control the order in which threads are scheduled for execution. Java provides a way to assign different priority levels to threads.
- Priority value influence the order of execution.
- Thread priorities
  - Thread.MIN_PRIORITY = 1
  - Thread.MAX_PRIORITY = 5
  - Thread.NORM_PRIORITY = 10
- Every thread has the default priority of Thread.NORM_PRIORITY
- The valid priority range is from 1 to 10. Values outside this range will throw an IllegalArgumentException.
- To set the priority of a thread in Java, use the setPriority(int priority) method on a Thread object

  Eg : *thread.setPriority(int priority);*

Object Oriented Programming in Java

2

OOP

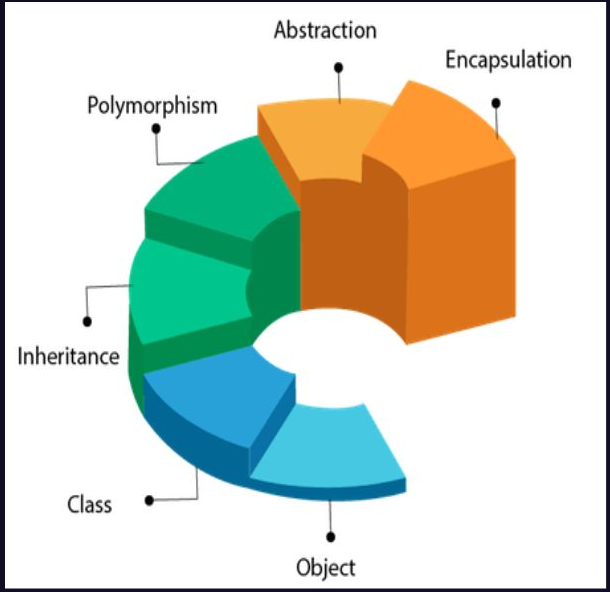
- Object-Oriented Programming (OOP) is a programming paradigm that relies on the concept of classes and objects
- It focuses on using objects to design and build applications.
- The object-oriented paradigm promotes the efficient design and development of software systems using reusable components that can be quickly and safely assembled into larger systems.
- Whenever we develop some programs using Class and Objects than it is called Object Oriented Programming System.

Why OOP ?

- OOP is faster and easier to execute.
- OOP provides a clear structure for the programs.
- Classes in OOP provide modular code that is easy to debug and update.
- OOP makes it possible to create full reusable applications with less code and shorter development time.
- OOP makes the code easier to understand, reusable and efficient and more maintainable and flexible

OOP Concept in Java

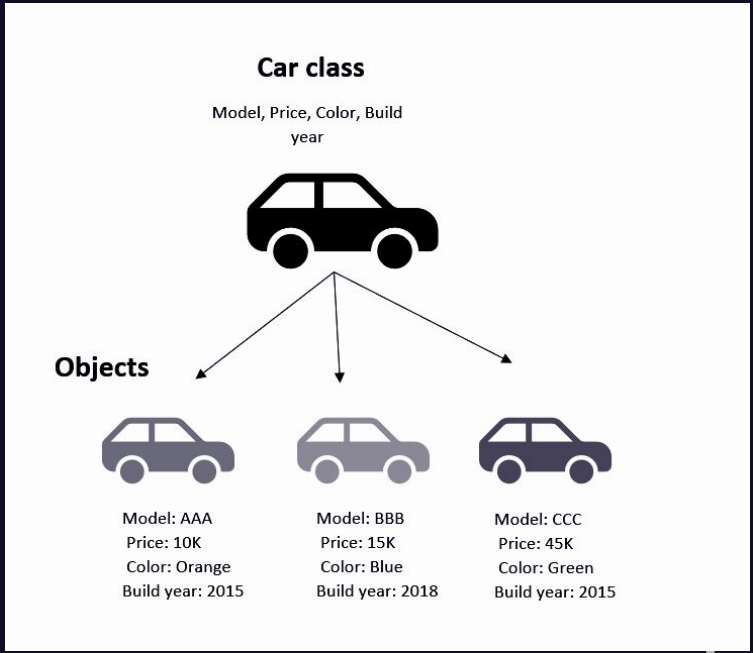
- Class
  - Templates or Blueprints
- Object
  - Real world entity
- Inheritance
  - Backward compatibility , parent child relation
- Polymorphism
  - Many forms
- Encapsulation
  - Hiding the functional details
- Abstraction
  - Visibility Controls



OOP Concept in Java

Class

- A class in Java is a blueprint or template used to create objects. It defines the properties (attributes) and behaviors (methods) that an object of the class will have.
- It acts as a container that groups related data and methods together, enabling code organization, reusability and abstraction.
- It is a basic concepts of OOPs which is a group of similar entities. It is only a logical component and not the physical entity.



OOP Concept in Java

Constructor

- In Java, a constructor is a special method that is used to initialize objects of a class. It is automatically invoked when an object is created using the new keyword.
- Its primary purpose is to assign initial values to the fields of the object and set it up for use.
- The name of the constructor must be exactly the same as the class name and it does not has any return types unlike other methods
- It helps to initialize objects and it is is automatically called when an object is created using the new keyword.
- Types
  - Default Constructor
  - Parameterized Constructor

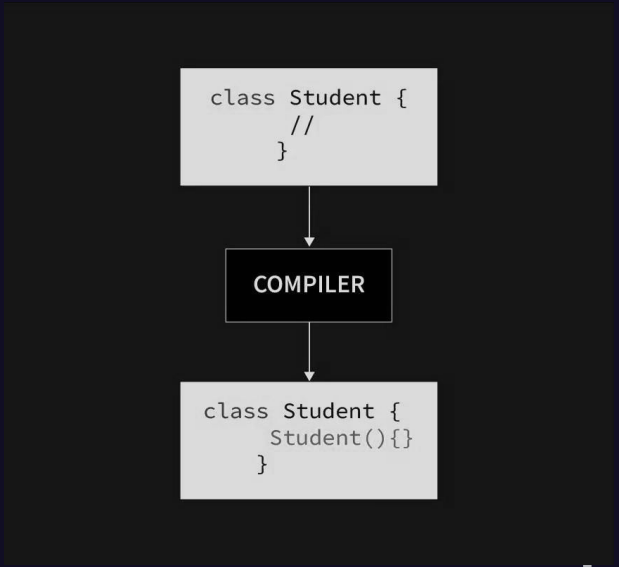
```
Class Student{

public Student(){

}

}
```

Constructor: Same name as class with no return type

OOP Concept in Java

Default Constructor

- A default constructor has no parameters. If no constructor is defined in the class, Java provides a default constructor implicitly.
- Its purpose is to initiainitializelizes the object with default values (e.g., 0 for integers, false for booleans and null for object references).



OOP Concept in Java

Default Constructor

```
class Student{
    String name;;
    int roll;

    // Default constructor (no parameters)
    public Student() {
        name= "Ram";
        roll= 10;
    }

    void displayDetails() {
        System.out.println("Student Name: " + name);
        System.out.println("Student Roll: " + roll);
    }
}

public class Main {
    public static void main(String[] args) {
        // Creating an object using the default constructor
        Student student= new Student();
        student.displayDetails();
    }
}
```

OOP Concept in Java

Parameterized Constructor

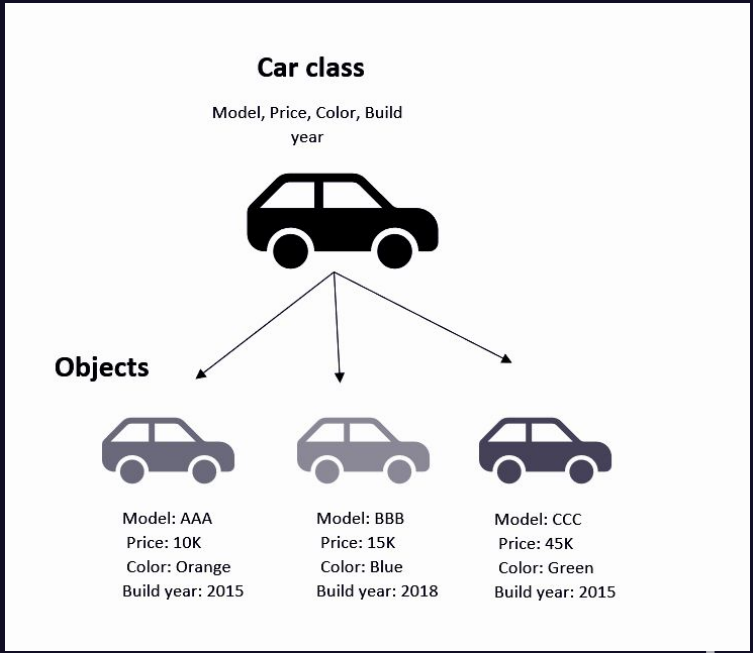
- A parameterized constructor allows us to pass values when creating an object which helps in initializing the object with specific values.
- Its purpose is to provide the values at the time of object creation.

```
class Student{
    String name;
    int roll;

    // Parameterized constructor
    public Student(String name, int roll) {
        this.name = name;
        this.roll = roll;
    }

    void displayDetails() {
        System.out.println("Student Name: " + name);
        System.out.println("Student Roll: " + roll);
    }
}

public class Main {
    public static void main(String[] args) {
        // Creating an object using the parameterized constructor
        Student student= new Student("Ram", 10);
        student.displayDetails();
    }
}
```

OOP Concept in Java

Object

- An object is the fundamental building block of Object-Oriented Programming (OOP). It is a real-world entity or instance of a class that represents a combination of data (attributes or fields) and behavior (methods or functions).
- It is created using a class as a blueprint.
- It is an Instance of a class
- Any entity that has state and behavior is known as an object.



OOP Concept in Java

"this" Keyword

- The this keyword in Java is a reference variable that refers to the current object. It is used to refer to the current instance of the class and it has several important uses in Java programming.
- "this" keyword makes the code cleaner, more readable and easier to maintain.
- It is particularly useful for
  - Referring to Instance Variables
  - Referring to Instance Methods
  - Calling Constructors
  - Avoiding Ambiguity Between Instance Variables and Parameters
  - Returning the Current Object from a Method

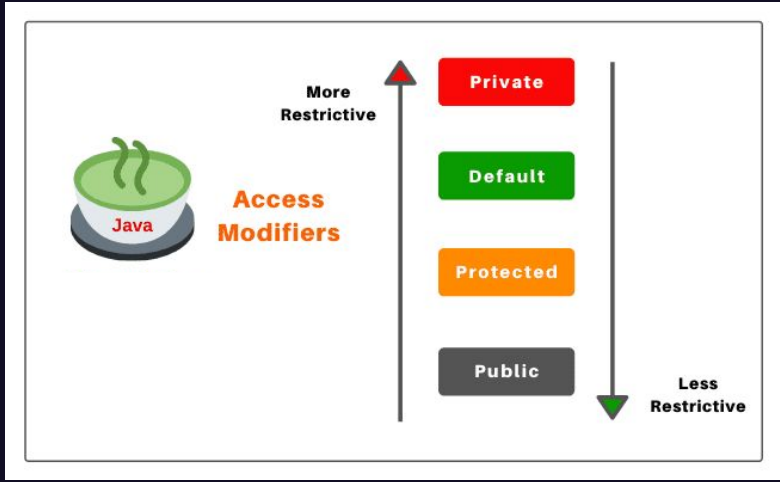
“this” Keyword

```
class Car {
    String model;
    int year;

    // Constructor with one parameter, calling the constructor with two parameters using 'this'
    public Car(String model) {
        this(model, 2020);  // Calling next Constructor
    }

    // Constructor with two parameters
    public Car(String model, int year) {
        this.model = model;  // Referring to the instance variable using 'this'
        this.year = year;    // Referring to the instance variable using 'this'
    }


    // Method to display car details
    public void display() {
        System.out.println("Car Model: " + this.model); // Referring to instance variable using 'this'
        System.out.println("Car Year: " + this.year);   // Referring to instance variable using 'this'
    }
}

public class Main {
    public static void main(String[] args) {
        // Using constructor chaining and constructor with one parameter
        Car car = new Car("Tesla");
        car.display();  // Displays the details using 'this' inside the Car class


    }
}
```

OOP Concept in Java

Access Modifiers

- Access modifiers in Java are keywords that define the visibility or accessibility of classes, methods and variables within different parts of a program.
- They play an important role in encapsulation and data hiding in object-oriented programming
- They allow to control access to the class members, protect the data and maintain flexibility of the programs



OOP Concept in Java

- **private**
  - The private modifier restricts access to only within the same class. The members (fields, methods) marked as private cannot be accessed from outside their class, not even by subclasses or other classes in the same package.
  - It hides sensitive information, ensuring that fields or methods are not accessible directly outside the class.
- **public**
  - The public modifier makes the member (class, method, or variable) accessible from anywhere in the program. It has the widest visibility.
  - It is used for classes, methods, variables that need to be accessed by any other class or package.
- **protected**
  - The protected modifier allows access within the same package and also in subclasses (even if the subclass is in a different package).
  - It is used for allowing access to fields and methods within the same package or in subclasses, while still restricting access from other classes.

OOP Concept in Java

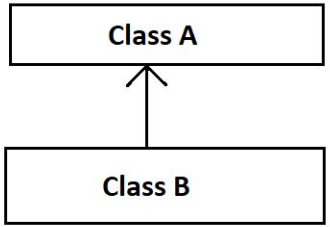
default

- Default Access Modifier limits access to only classes within the same package.
- If no access modifier is specified, Java uses the default access modifier. In this case, the member is accessible only within the same package.
- It is used for making classes, methods or variables accessible only within the same package.
- The default access is package-private.
- No access modifier means default access

OOP Concept in Java

Inheritance

- Inheritance is a key concept in Object-Oriented Programming (OOP) that allows a class (called the child class or subclass) to acquire properties and behaviors of another class (called the parent class or superclass).
- It enables code reuse, extensibility and promotes a logical relationship between classes to build modular and maintainable code.
- It is one of the basic concepts of OOPs in which one object acquires the properties and behaviors of the parent object.
- It's creating a parent-child relationship between two classes. It offers robust and natural mechanism for organizing and structure of any software.
- Java Class does not support multiple inheritance.



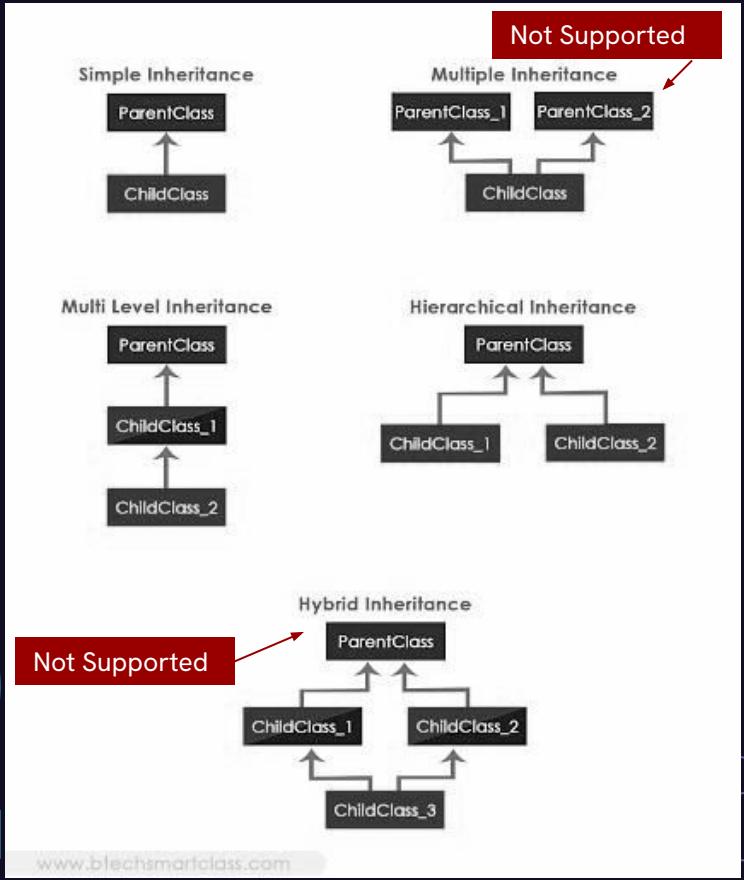

```
public class A {
  ..................
}

public class B extends class A {
  ..............
}
```

OOP Concept in Java

Types of Inheritance

- Single Inheritance
- Multilevel Inheritance
- Multiple Inheritance
- Hierarchical Inheritance
- Hybrid Inheritance

Hybrid Inheritance is mix of two or more than two types of inheritance. Here Multiple inheritance is included so it is not supported with classes in java. But hybrid inheritance combination of Multilevel and Hierarchical is supported in java.



OOP Concept in Java

Super Keyword

- The super keyword in Java is a reference variable used to refer to the immediate parent class (superclass) of the current object.
- The super keyword is essential in the context of inheritance to allow subclasses to interact with their parent classes effectively.
- Use of super keyword
  - Accessing parent class constructor [super()]
  - Accessing parent class method [super.method()]
  - Accessing parent class field [super.field]

OOP Concept in Java [Super Keyword]

```
class Animal {
    String name;              // Parent class field
    Animal(String name) {       // Parent class constructor
        this.name = name;
        System.out.println("Animal Constructor: Name is " + name);
    }
      void sound() {            // Parent class method
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
        String name;                  // Subclass (Dog) field, shadowing the parent class field
        Dog(String name) {          // Subclass constructor calling the parent class constructor
         super(name);                // Calling the parent class constructor using super()
         this.name = name;           // Setting the subclass field
         System.out.println("Dog Constructor: Name is " + name);
    }
     @Override
    void sound() {                  // Method overriding the parent class method
        System.out.println("Dog barks");
    }
        void showDetails() {                // Method to demonstrate the use of super for method and field access
        // Accessing the parent class method using super
        super.sound(); // Calls the Animal class sound method
         System.out.println("Accessing Parent Class Field: Animal's name is " + super.name);   // Accessing the parent class field using super
        // Accessing the subclass field directly
        System.out.println("Accessing Subclass Field: Dog's name is " + this.name);
    }
}
public class Test {
    public static void main(String[] args) {
        Dog dog = new Dog("Buddy");
        dog.sound(); // Calls the Dog's overridden method
         dog.showDetails();         // Calling the method that shows details from both parent and subclass

    }
}
```

OOP Concept in Java

Multiple Inheritance Issue in Class

Multiple inheritance refers to the ability of a class to inherit features (fields and methods) from more than one parent class.

- However, Java does not support multiple inheritance with classes to avoid ambiguities and complexity but it can be achieved by the interface in Java.

Reasons for not supporting multiple inheritance:

Consider the following scenario:

- Class A has a method read().
- Classes B and C inherit from A and override read().
- Class D inherits from both B and C.

When we call D.read(), it's unclear whether the method from B or C should be executed and this situation is known as the ambiguities in inheritance.

OOP Concept in Java

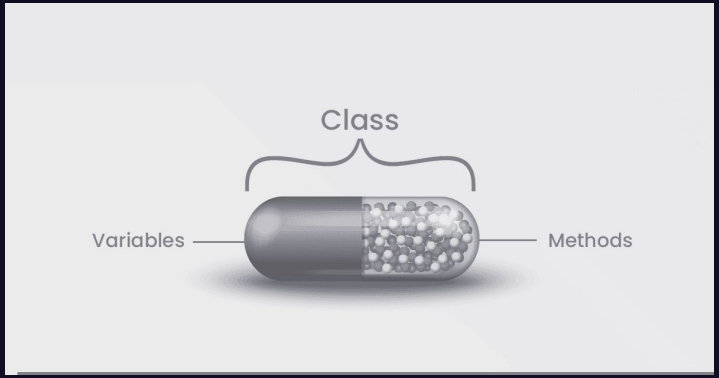
Multiple Inheritance issue in Class

```java
// Example of a potential ambiguity problem in multiple inheritance scenerio
class A {
    void read() {
        System.out.println("Class A");
    }
}

class B extends A {
    @Override
    void read() {
        System.out.println("Class B");
    }
}

class C extends A {
    @Override
    void read() {
        System.out.println("Class C");
    }
}

// Java doesn't allow this scenario
class D extends B, C {
    // Which display() should be called? B's or C's?
}
```

OOP Concept in Java

Encapsulation

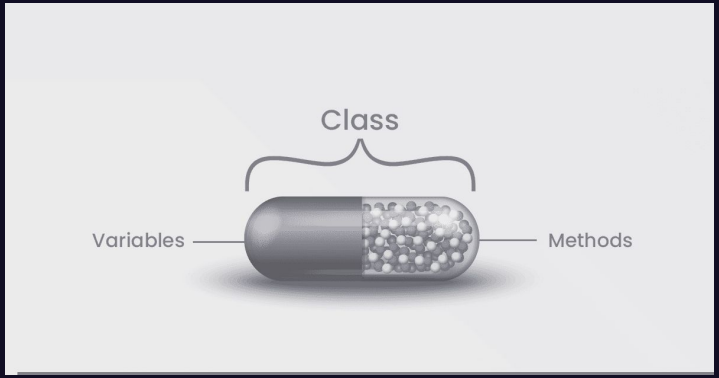
- Encapsulation is one of the best fundamental principles of Object-Oriented Programming (OOP). It is the process of bundling data (fields/attributes) and methods (functions) that operate on the data into a single unit, usually a class, and restricting access to the internal state of the object to ensure data security and integrity.
- In simpler terms, encapsulation hides the internal details of how an object works and only exposes what is necessary for other parts of the program to interact with it.
- In OOP concept, the variables of a class are always hidden from other classes. It can only be accessed using the methods of their current class.



OOP Concept in Java

Encapsulation

- Encapsulation helps to improve security as sensitive data are protected from the unauthorized access
- Enhance modularity, maintainability and scalability of the code.
- Access modifiers like private, protected and public help us to maintain the encapsulation.



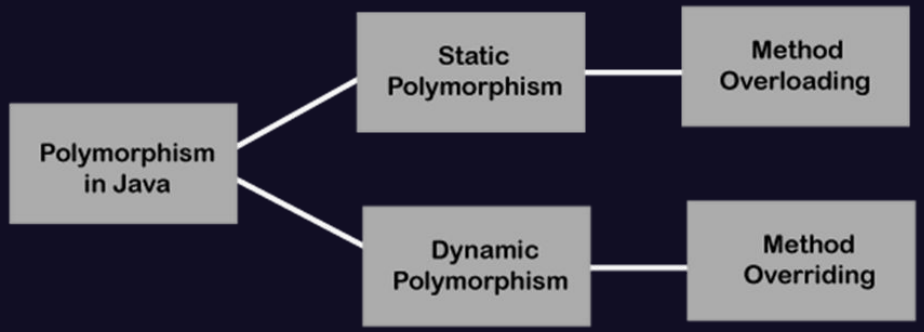
OOP Concept in Java

Polymorphism

- Polymorphism is a core concept of OOP that enables one interface or method to perform different tasks depending on the context.
- By leveraging method overloading (compile-time polymorphism) and method overriding (run-time polymorphism), polymorphism enhances flexibility, reusability and maintainability in object-oriented systems.
- Polymorphism allows a single action to behave differently depending on the object that is performing the action.
- In general, If one task is performed by different ways, it is known as polymorphism.

OOP Concept in Java

- Polymorphism



OOP Concept in Java

Static/ Compile Time Polymorphism

- Compile-time polymorphism occurs when the method or function call is resolved during compile time which is typically achieved through method overloading.
- It is needed when multiple methods have the same functionality but operate on different data types or parameter counts.
- It makes the code more readable and efficient.

Method Overloading

- Method overloading happens when a class has multiple methods with the same name but different parameters (either different number of parameters or different types of parameters).
- The compiler determines which method to invoke at compile time based on the method signature.

OOP Concept in Java

- Static/ Compile Time Polymorphism

  Method Overloading

```
class Calculator {
   // Overloaded methods
  public int add(int a, int b) {
      return a + b;
   }

   public double add(double a, double b) {
      return a + b;
   }

   public int add(int a, int b, int c) {
      return a + b + c;
   }
}

public class Main {
   public static void main(String[] args) {
      Calculator calc = new Calculator();
      System.out.println("Sum of two integers: " + calc.add(5, 10));       // Calls add(int, int)
      System.out.println("Sum of two doubles: " + calc.add(5.5, 10.2));    // Calls add(double, double)
      System.out.println("Sum of three integers: " + calc.add(5, 10, 15)); // Calls add(int, int, int)
   }
}
```

OOP Concept in Java

Dynamic/ Run Time Polymorphism

- Run-time polymorphism occurs when the method to be invoked is determined at run time, depending on the object that is calling the method.
- Run-Time Polymorphism (also known as Dynamic Polymorphism) in Java is a feature that allows a subclass to provide a specific implementation of a method that is already defined in its parent class.
- It is resolved at run-time, meaning that the Java Virtual Machine (JVM) decides which method to call based on the actual object type, not the reference type.

Method Overriding

- Method overriding allows a subclass to provide a specific implementation for a method that is already defined in its superclass.
- At runtime, the Java Virtual Machine (JVM) decides which method to call based on the actual object type, not the reference type.
- Method Overriding occurs when a subclass defines a method with the same name, return type and parameters as a method in its superclass.
- Overridden methods in subclasses can have different behaviors from the methods in the superclass, providing specific functionality for the subclass.

OOP Concept in Java

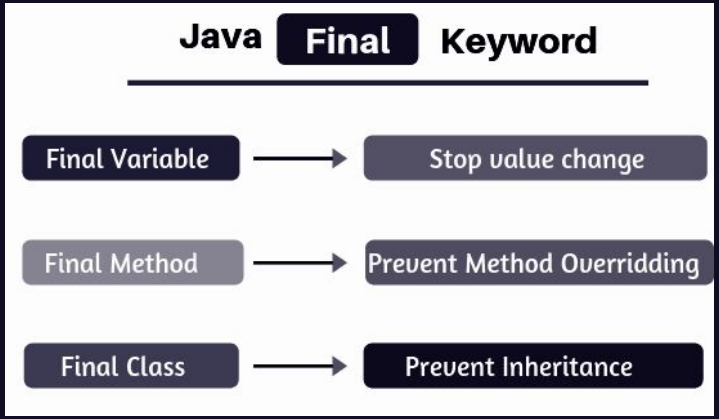
❖ Dynamic/ Runtime Polymorphism [Method Overriding]

```java
class Animal {
    // Overridden method
    void sound() {
        System.out.println("Animals make sounds");
    }
}
class Dog extends Animal {
    // Overriding the sound method
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}
class Cat extends Animal {
    // Overriding the sound method
    @Override
    void sound() {
        System.out.println("Cat meows");
    }
}
public class Main {
    public static void main(String[] args) {
        // Creating objects of Dog and Cat
        Animal myAnimal = new Animal();   // Animal object
        Animal myDog = new Dog();         // Dog object (Polymorphic reference)
        Animal myCat = new Cat();         // Cat object (Polymorphic reference)
        // Method calls
        myAnimal.sound(); // Calls Animal's sound() method
        myDog.sound();    // Calls Dog's sound() method
        myCat.sound();    // Calls Cat's sound() method
    }
}
```

Final and Static Variable

Final modifiers

- The final keyword in Java is used to declare constants, prevent method overriding and prevent inheritance.
- It has different behaviors depending on where it's applied: to variables, methods and classes
- **Final with Variable [Constant Variable]**
  - When a variable is declared as final, its value cannot be changed after it is initialized. This is typically used to create constant values.
- **Final with Method [Prevent method overriding]**
  - When a method is declared final, it cannot be overridden by any subclass.
  - This is useful to ensure that the behavior of a method remains unchanged in subclasses.



Final and Static Variable

Final modifiers

- **Final with Class [Prevent Inheritance]**
  - When a class is declared final, it cannot be subclassed. No other class can inherit from a final class.

```
Declaration of final
// Final variable
final String model = "BYD";

// Final Method
public final void start() {
System.out.println("Car is starting");
}

//Final Class
final class Car {
 public void start() {
System.out.println("Car is starting");
 }
}
```

Final and Static Variable

Static modifiers

- The static keyword in Java is mainly used for memory management.
- It is used to share the same variable or method of a given class. It can be applied with variables, methods, blocks and nested classes.
- The static keyword belongs to the class rather than an instance of the class.
- This keyword is used for a constant variable or a method that is the same for every instance of a class.
- Variable and methods declared static can be accessed using the class name, without creating an object.

Final and Static Variable

Static modifiers characteristics

- **Shared memory allocation:**
  - Static variables and methods are allocated memory space only once during the execution of the program. This memory space is shared among all instances of the class, which makes static members useful for maintaining global state or shared functionality.
- **Accessible without object instantiation:**
  - Static members can be accessed without the need to create an instance of the class. This makes useful for providing utility functions and constants that can be used across the entire program.
- **Associated with class, not objects**:
  - Static members are associated with the class, not with individual objects. This means that changes to a static member are reflected in all instances of the class, and that you can access static members using the class name rather than an object reference.
- **Can be overloaded but not overridden:**
  - Static methods can be overloaded, which means that you can define multiple methods with the same name but different parameters. However, they cannot be overridden, as they are associated with the class rather than with a particular instance of the class.

Final and Static Variable

Why Use static with final?

- **Class-Level Association:**
  - A final variable without static is instance-level. This means each object of the class will have its own copy of the final variable.
  - By adding static, the variable is associated with the class itself rather than any specific object. This ensures only one copy of the variable exists for the entire class, shared by all instances.
- **Global Constants:**
  - static final variables are often used for global constants in a program, such as mathematical values or configuration parameters, which should be universally accessible.
  - These variables are stored in the method area of the JVM (class memory) rather than instance memory, making them memory-efficient for constants.
- **Access Without Object Creation:**
  - A static final variable can be accessed directly using the class name, without the need to create an instance of the class. This is useful for constants like Math.PI or Integer.MAX_VALUE.
- **Design Clarity:**
  - Declaring a constant as static final clarifies its intent as a class-level immutable value, which can enhance code readability and maintainability.

Final and Static Variable

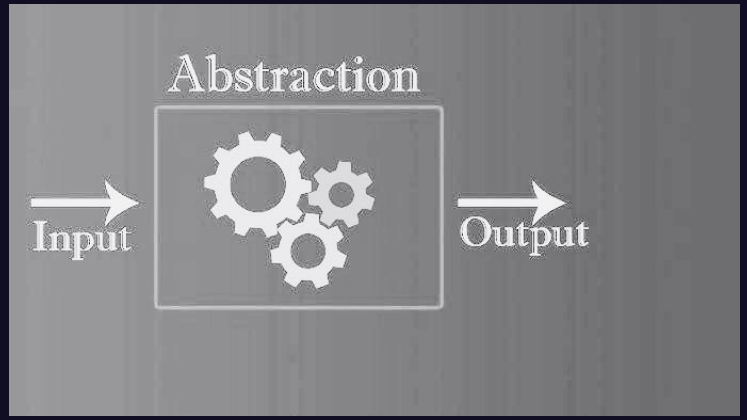
```
class Example {
    final int instanceVariable = 10;  // Final (instance-level)
    static final int classVariable = 20;  // Static and final (class-level)

    public void show() {
        System.out.println("Instance variable: " + instanceVariable);
        System.out.println("Class variable: " + classVariable);
    }
}

public class Main {
    public static void main(String[] args) {
        Example obj1 = new Example();
        Example obj2 = new Example();

        // Access final instance variable through different objects
        System.out.println("Object 1 - Instance Variable: " + obj1.instanceVariable);
        System.out.println("Object 2 - Instance Variable: " + obj2.instanceVariable);

        // Access static final variable directly using the class name
        System.out.println("Static Final Variable (via class): " + Example.classVariable);

        // Access static final variable through an object (not recommended)
        System.out.println("Static Final Variable (via object): " + obj1.classVariable);
    }

}
```

Output:
Object 1 - Instance Variable: 10
Object 2 - Instance Variable: 10
Static Final Variable (via class): 20
Static Final Variable (via object): 20

OOP Concept in Java

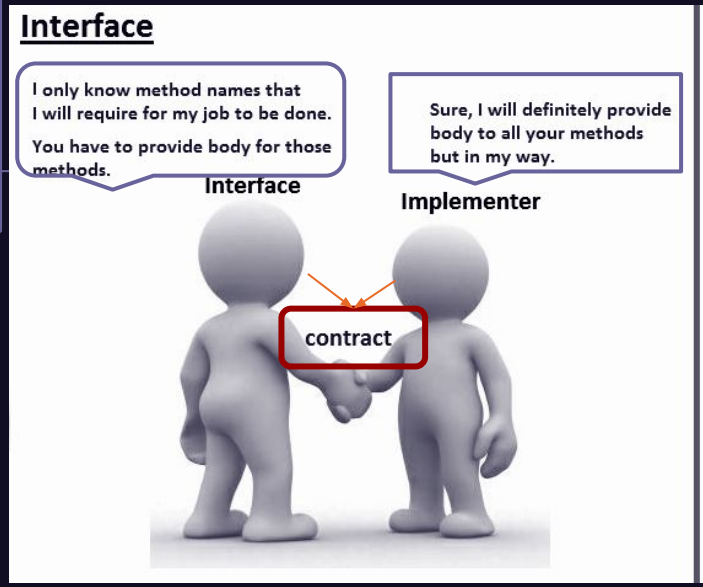
Abstraction

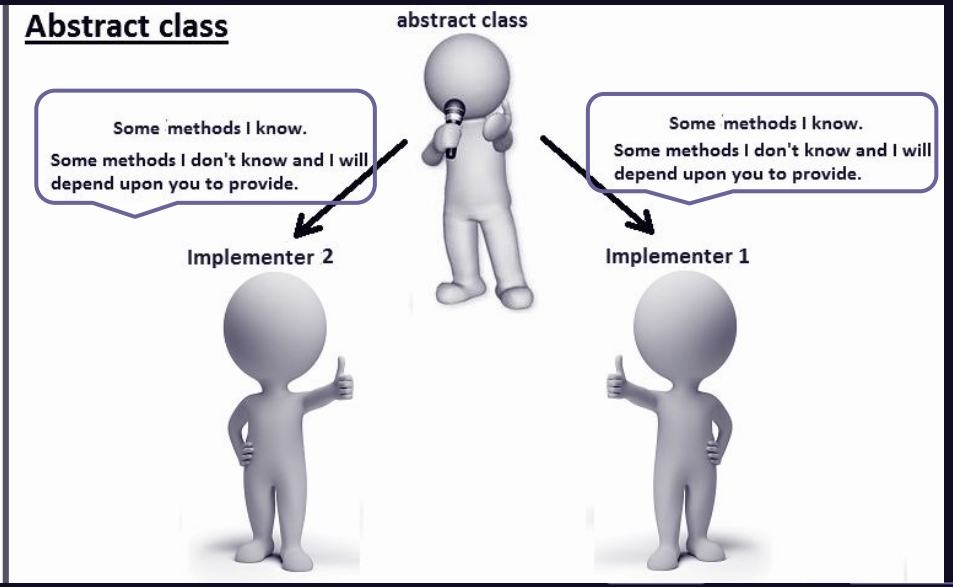
- Abstraction is one of the most powerful concepts in Object-Oriented Programming (OOP) and is essential for building flexible and maintainable software systems.
- It allows us to focus on high-level functionality while hiding complex implementation details.
- It refers to the process of hiding the implementation details of an object and exposing only the essential features or functionalities to the user.
- Basically, Abstraction deals with "what" an object does, rather than "how" it does .



OOP Concept in Java

- Implementation of Abstraction
  - Abstract Class
  - Interface





OOP Concept in Java

Abstract Class

- An abstract class serves as a blueprint for other classes as it cannot be instantiated directly. Instead, other classes must inherit from it and provide implementations for its abstract methods.
- It is a template or base class that enforces a structure but leaves some parts to be defined by its subclasses.
- It is contains both abstract methods and concrete methods. Abstract method refers to the methods that are declared but not implemented (only their definition is provided) whereas Concrete methods refers to the methods with complete implementation.
- It allows to create a common base for related classes while enforcing specific methods that must be implemented.
- On regard to its inheritance, only one abstract class can be extended at a time due to Java's single inheritance rule.
- It used when there is need to create a common blueprint for a group of related classes.

▶ OOP Concept in Java[Abstract Class]

```java
abstract class Animal {
    // Abstract method
    abstract void sound();

    // Concrete method
    void breathe() {
        System.out.println("All animals breathe");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }

}
class Cat extends Animal {
    @Override
    void sound() {
        System.out.println("Cat meows");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal dog = new Dog();
        dog.breathe();  // Inherited from Animal
        dog.sound();    // Implemented in Dog

        Animal cat = new Cat();
        cat.breathe();  // Inherited from Animal
        cat.sound();    // Implemented in Cat
    }
}
```

OOP Concept in Java

Interface

- An interface in Java is a special type of class that only declares methods and does not provide their implementations.
- It acts as a contract or blueprint that defines the behavior a class must implement. In general it is a set of rules that a class promises to follow.
- A class can implement multiple interfaces, allowing for more flexibility compared to abstract classes.
- Interfaces defines what a class should do, but not how it does it.
- Class implements the Interface by keyword *"implements"*
- It cannot have constructors, as they cannot be instantiated.

In summary, Abstract classes provide a way to share common functionality, while interfaces define strict rules for behavior. By combining both effectively, we can create systems that are reusable, modular, and easy to extend.

▶ OOP Concept in Java[Interface]

```
interface Animal {
    void sound();  // Abstract method
    void eat();    // Abstract method
}
class Dog implements Animal {
    @Override
    public void sound() {
        System.out.println("Dog barks");
    }

    @Override
    public void eat() {
        System.out.println("Dog eats");
    }
}
class Cat implements Animal {
    @Override
    public void sound() {
        System.out.println("Cat meows");
    }

    @Override
    public void eat() {
        System.out.println("Cat eats");
    }
}
public class Main {
    public static void main(String[] args) {
        Animal dog = new Dog();
        dog.sound();  // Calls Dog's sound method
        dog.eat();    // Calls Dog's eat method

        Animal cat = new Cat();
        cat.sound();  // Calls Cat's sound method
        cat.eat();    // Calls Cat's eat method
    }
}
```

Combination

```
abstract class Animal {
    abstract void breathe();
}

interface Pet {
    void play();
}

class Dog extends Animal implements Pet {
    @Override
    void breathe() {
        System.out.println("Dog breathes");
    }

    @Override
    public void play() {
        System.out.println("Dog plays fetch");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.breathe();
        dog.play();
    }
}
```

How multiple Inheritance in Java?

Java avoids multiple inheritance with classes because it can lead to issues of ambiguity and complexity, where the compiler cannot determine which method or property to inherit if two parent classes have conflicting implementations.

- While multiple inheritance with classes is not allowed, Java provides a solution using interfaces. A class can implement multiple interfaces, enabling a class to inherit behaviors from multiple sources while maintaining clarity and avoiding ambiguity.
- Multiple inheritance with interface avoids the ambiguity with confusion in order to use the proper implemented methods, also provides the separation of concerns where interfaces focus on behaviour while class focus on the implementation

Multiple Inheritance with Interface

```
// Interface 1
interface Electric {
    void chargeBattery();
}

// Interface 2
interface SelfDriving {
    void navigate();
}

// Class Implementing  interfaces
class Tesla  implements Electric, SelfDriving {

    @Override
    public void chargeBattery() {
        System.out.println("Tesla is charging its battery");
    }

    @Override
    public void navigate() {
        System.out.println("Tesla is navigating autonomously");
    }
}

public class Main {
    public static void main(String[] args) {
        Tesla myTesla = new Tesla();
        myTesla.chargeBattery();
        myTesla.navigate();
    }
}
```

Building components using Swing and JavaFx

3

AWT [Abstract Window Toolkit]

- AWT , is a Java library offering graphical user interface (GUI) components and classes for designing user interfaces or windows based applications in Java applications.
- As one of Java's first GUI frameworks, it is part of the Java Foundation Classes (JFC). AWT allows developers to create elements like windows, dialogs, buttons and text fields, facilitating the creation of interactive applications.
- Java AWT components are platform-dependent i.e. components are displayed according to the view of operating system.
- AWT was the original GUI library in Java, it has been largely surpassed by more advanced libraries like Swing and JavaFX
  - Components:
    - AWT offers a variety of components, including buttons, labels, text fields, checkboxes, radio buttons, and others. These components can be placed inside containers like frames and panels to build a user-friendly interface.
  - Event Handling:
    - AWT components trigger events like mouse clicks and keyboard inputs. It offers tools for managing these events through listeners and adapters.

AWT [Abstract Window Toolkit]

- Components:
  - AWT offers a variety of components, including buttons, labels, text fields, checkboxes, radio buttons, and others. These components can be placed inside containers like frames and panels to build a user-friendly interface.
- Event Handling:
  - AWT components trigger events like mouse clicks and keyboard inputs. It offers tools for managing these events through listeners and adapters.
- Layout Managers:
  - AWT offers layout managers like BorderLayout, FlowLayout, GridLayout, and CardLayout, which help in arranging and positioning components within containers
- Heavy weight components:
  - AWT components are often referred to as heavyweight components because they are directly mapped to native GUI elements provided by the operating system. This can sometimes lead to platform-specific behavior and performance issue.
- Cross Platform
  - AWT components are designed to be cross-platform, meaning that the same code can run on different operating systems with a consistent look and feel. However, this consistency might not be as refined as in later GUI libraries like Swing and JavaFX.

AWT [Abstract Window Toolkit]

- Integration:
  - AWT components can be integrated with other Java libraries and frameworks to create complex applications. They can be used in applets, standalone applications, and other types of Java programs

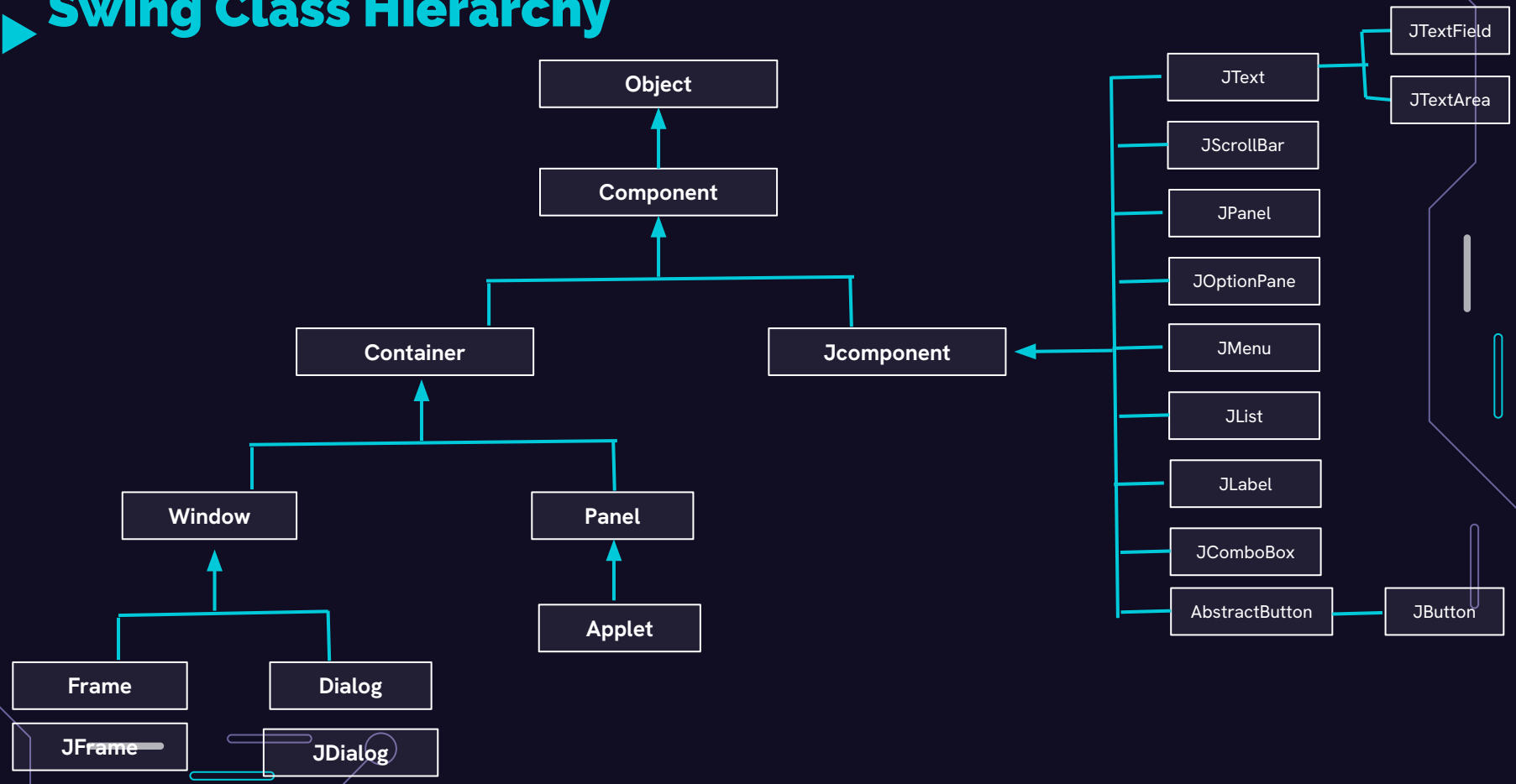
- Limitations
  - AWT has certain limitations, including restricted customizability of its appearance, less advanced rendering features, and a smaller selection of built-in components compared to newer GUI libraries.
  - AWT components rely on the native operating system, leading to inconsistencies across platforms with heavy weight components
  - AWT provides fewer components and functionalities compared to advanced frameworks like Swing or JavaFX.
  - As it is outdated at preset, Modern Java developers prefer Swing or JavaFX due to their enhanced features and flexibility.

Swing

- Swing is a powerful GUI (Graphical User Interface) toolkit for Java applications, designed to provide a rich set of components and features for creating interactive and visually appealing graphical user interfaces
- Swing was introduced as part of the Java Foundation Classes (JFC) to address the limitations of the original AWT (Abstract Window Toolkit) and provide developers with a more modern and versatile way to build GUI applications
- Swing is built on top of the AWT and It provides a richer and more flexible set of GUI components than AWT and allows for creating modern and visually appealing desktop applications.
- It overcomes many limitations of AWT by offering more components, greater flexibility, and a consistent look and feel across platforms

Java Swing

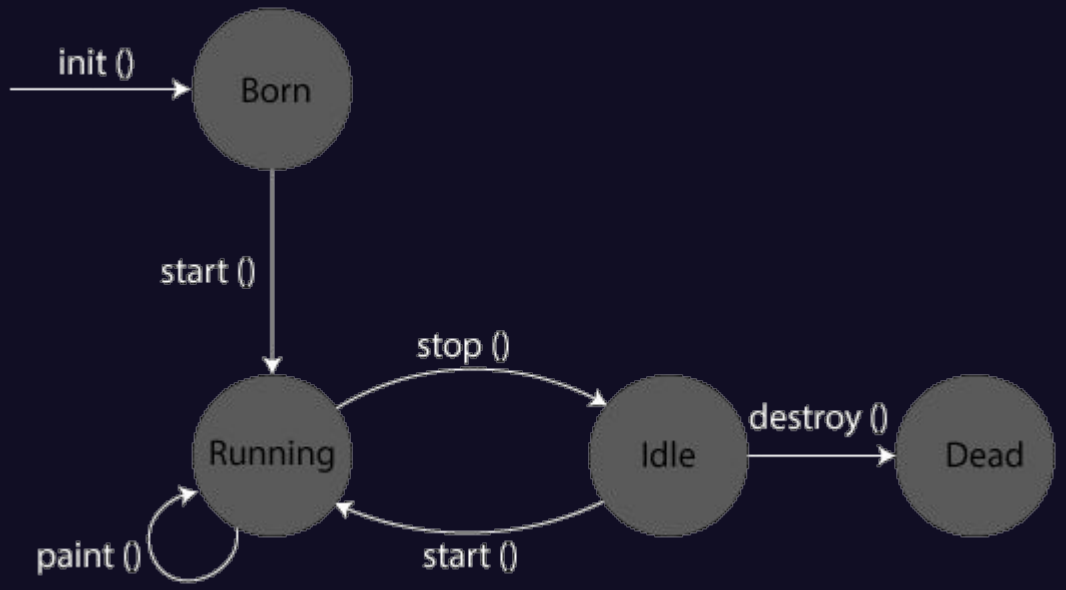
Swing Class Hierarchy



Applet

- An applet is a lightweight Java program designed to run within a web browser, enabling interactive and dynamic content on web pages.
- Applets were popular for incorporating multimedia, animations, and interactive features into websites.
- The use of Applet has significantly declined due to security concerns and the emergence of modern web technologies such as HTML5, CSS, and JavaScript.
- Applets are now largely outdated, however studying their types and characteristics offers valuable insights into the evolution of web development
- Features of Applet
  - Browser Integration
  - Platform Independence
  - Sandbox Security

Applet Life Cycle



Applet Types

- StandAlone Applets
  - Standalone applets are Java programs designed to run independently of a web browser.
  - Unlike web applets, they function as standalone applications and do not require integration with a web page or a browser for execution.
  - These applets are typically launched using an applet viewer or directly from a development environment.
  - Standalone applets are primarily utilized for purposes such as testing, debugging, or executing Java code in environments where a browser is not needed.

▶ StandAlone Applet

```
import java.applet.*;
import java.awt.*;
public class StandaloneApplet extends Applet {
    public void paint(Graphics g) {
        g.drawString("This is a Standalone Applet", 70, 70);
    }
    public static void main(String[] args) {
        StandaloneApplet applet = new StandaloneApplet();
        Frame frame = new Frame("Standalone Applet");
        frame.add(applet);
        frame.setSize(400, 300);
        frame.setVisible(true);
    }
}
```

Applet Types

- Web Applets
  - Web applets are designed to run within a web browser.
  - They are embedded in HTML pages using the <applet> or <object> tag and are executed by the Java Plug-in or Java Web Start.
  - Web applets provide interactivity and multimedia content directly within the web page
  - These rely on the browser's support for the Java plugin, which facilitates their execution in a web environment.
  - Web applets were widely used in the early days of the internet to enhance user experiences by incorporating multimedia, animations, and interactivity.

▶ Web Applets

```
<html>
<head></head>
 <body>
<applet code="WebApplet.class" width="400" height="300">Browser does not support the
<code>applet</code> tag.</applet>
</body>
</html>


import java.applet.*;
import java.awt.*;
public class WebApplet extends Applet {
    public void paint(Graphics g) {
        g.drawString("This is a web applet", 50, 50);
    }
}
```

Layout Management

- Layout management refers to the method of arranging graphical user interface (GUI) components within a container, such as a JFrame or JPanel.
- Layout managers simplify the process of designing user interfaces that are flexible and adaptable to various screen sizes and resolutions, ensuring a consistent and professional appearance across platforms.
- Java's Swing framework includes several layout managers, each offering unique strategies for positioning and sizing components.
- Importance of Layout Management
  - Eliminate the need for manual positioning and sizing of components.
  - Ensure components adjust dynamically to different screen dimensions and resolutions.
  - Simplify cross-platform GUI development by standardizing layouts.

FlowLayout

- FlowLayout arranges components in a left-to-right, top-to-bottom flow.
- Components are placed next to each other until the container is full, and then a new row is started.
- It's straightforward and suitable for small forms or dialogs.

```
JPanel panel = new JPanel(new FlowLayout());
```

BoxLayout

- BoxLayout is used to arrange components either horizontally or vertically in a container.
- It is part of the javax.swing package and offers more control over alignment and spacing compared to FlowLayout or GridLayout.

```
Container container = new JPanel();
container.setLayout(new BoxLayout(container, BoxLayout.Y_AXIS)); // For vertical alignment
```

BorderLayout

- It divides the container into five regions: North, South, East, West, and Center.
- Components are placed in these regions, and the Center region takes up any remaining space.
- It's useful for creating simple, main-content layouts.

```
JFrame frame = new JFrame();
frame.setLayout(new BorderLayout());
frame.add(new JButton("North"), BorderLayout.NORTH);
frame.add(new JButton("Center"), BorderLayout.CENTER);
```

GridLayout

- GridLayout organizes components in a grid, with a specified number of rows and columns.
- All components are given equal space in the grid.
- It's useful for creating forms with a consistent grid structure.

```
JPanel panel = new JPanel(new GridLayout(rows, columns));
```

GridBagLayout

- GridBagLayout is a flexible and powerful layout manager that allows precise control over the placement and sizing of components.
- It uses constraints to define how components should be arranged.
- It's suitable for complex and customized layouts.

```
GridBagConstraints gbc = new GridBagConstraints();
JPanel panel = new JPanel(new GridBagLayout());
gbc.gridx = 0; gbc.gridy = 0;
panel.add(new JButton("Button 1"), gbc);
```

GUI Controls

- In Java, GUI (Graphical User Interface) controls are elements or components that enable user interaction with the graphical interface of a Java application.
- Commonly referred to as widgets or components, these controls are part of the Java Swing framework and are essential for building visually interactive applications.
- They include elements like windows, dialogs, buttons, text fields and other interface components that provide a seamless way for users to engage with the application.
- Java GUI controls form the backbone of interactive applications, providing the means to create user-friendly interfaces

GUI Controls

- **JFrame**
  - JFrame is a top-level container that represents the main window of a Java Swing application.
  - It can contain other components such as buttons, panels, and text fields.

```
JFrame frame = new JFrame("Java Application");
```

GUI Controls

JPanel

- JPanel is a container that can hold and organize other components.
- It is often used to group related components and manage their layout.

```
JPanel panel = new JPanel();
```

GUI Controls

JButton

- JButton represents a button that can trigger an action when clicked.
- It is commonly used to initiate processes or perform specific actions.

```
JButton button = new JButton("Register");
```

GUI Controls

JTextField

- JTextField is a text input field that allows users to enter and edit text.
- It is used for accepting user input, such as names or other textual data.

```
JTextField textField = new JTextField("Text");
```

GUI Controls

- **JTextArea**
  - JTextArea is a multi-line text area that can display and allow editing of multiple lines of text.
  - It is suitable for handling larger amounts of text.

```
JTextArea textArea = new JTextArea("MultiLine Text");
```

GUI Controls

- **JLabel**
  - JLabel is a non-editable text component used for displaying text or images.
  - It is often used to provide information or labels for other components.

```
JLabel label = new JLabel("This is a Label");
```

GUI Controls

JCheckBox

- JCheckBox represents a check box that can be selected or deselected.
- It is used for binary choices, such as enabling or disabling a feature.

```
JCheckBox checkBox = new JCheckBox("Check Box");
```

GUI Controls

JRadioButton

- JRadioButton represents a radio button, a mutually exclusive option in a group of radio buttons.
- It is used when users need to select one option from a set of choices.

```
JRadioButton radioButton = new JRadioButton("Radio Button");
```

GUI Controls

JComboBox

- JComboBox is a drop-down list or combo box that allows users to select an item from a list of options.
- It is useful when there are multiple choices, and users can only select one.

```
String[] options = {"Option 1", "Option 2", "Option 3"};
JComboBox<String> comboBox = new JComboBox<>(options);
```

Menu Elements and Tooltips

Menu elements are components that allow users to navigate and interact with various functionalities in an application.

- **JMenuBar**
  - It is a container for holding menus.
  - Generally it is placed at the top of JFrame.

```
JMenuBar menuBar = new JMenuBar();

frame.setJMenuBar(menuBar);
```

Menu Elements and Tooltips

- **JMenu**
  - It represents a single menu in the menu bar.
  - It contains JMenuItem objects and submenus.

```
JMenu fileMenu = new JMenu("File");
menuBar.add(fileMenu);
```

Menu Elements and Tooltips

- **JMenuItem**
  - It represents an item in a menu, such as a command or action.

```
JMenu fileMenu = new JMenu("File");
JMenuItem openItem = new JMenuItem("Open");
fileMenu.add(openItem);
```

Menu Elements and Tooltips

- **JCheckBoxMenuItem**
  - A menu item with a checkbox, allowing toggling of options.

```
JMenu viewMenu = new JMenu("View");
JCheckBoxMenuItem checkItem = new JCheckBoxMenuItem("Show
Toolbar");
viewMenu.add(checkItem);
```

Menu Elements and Tooltips

- **JRadioButtonMenuItem**
  - A menu item with a radio button, used for mutually exclusive selections.

```
JMenu viewMenu = new JMenu("View");
JRadioButtonMenuItem rbItem = new JRadioButtonMenuItem("Light Mode");
viewMenu.add(rbItem);
```

Menu Elements and Tooltips

- **Submenus**
  - Menus nested within other menus.

```
JMenu fileMenu = new JMenu("File");
JMenu exportMenu = new JMenu("Export");
fileMenu.add(exportMenu);
exportMenu.add(new JMenuItem("PDF"));
exportMenu.add(new JMenuItem("Word"));
```

Menu Elements and Tooltips

- **Tooltips**
  - Tooltips are short messages displayed when a user hovers over a component.
  - They provide helpful hints or descriptions about the component's functionality.

```
JButton button = new JButton("Click Me");
button.setToolTipText("Click this button to view the detail");
```

Example for Menu Elements and Tooltips

```
import javax.swing.*;

public class MenuExample {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Menu and Tooltips Example");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(400, 300);

        // Creating menu bar
        JMenuBar menuBar = new JMenuBar();

        // Creating menus
        JMenu fileMenu = new JMenu("File");
        JMenu editMenu = new JMenu("Edit");
        menuBar.add(fileMenu);
        menuBar.add(editMenu);

        // Adding menu items
        JMenuItem openItem = new JMenuItem("Open");
        openItem.setToolTipText("Open a file");
```

Example for Menu Elements and Tooltips

```
        JMenuItem saveItem = new JMenuItem("Save");
        saveItem.setToolTipText("Save the current file");
        fileMenu.add(saveItem);

        JMenuItem exitItem = new JMenuItem("Exit");
        exitItem.setToolTipText("Exit the application");
        fileMenu.add(exitItem);

        // Adding menu bar to frame
        frame.setJMenuBar(menuBar);

        // Adding a button with a tooltip
        JButton button = new JButton("Click Me");
        button.setToolTipText("This is a clickable button");
        frame.add(button);

        frame.setVisible(true);
    }
}
```

JFrame(Frame)

- JFrame is a top-level container used to create the main window of an application.
- It represents the main graphical window where all the user interface components are placed.
- It is usually the main window of the application, displaying all the main components such as buttons, text fields, labels etc.
- It is the backbone of a GUI application, used to create the main application window.
- By default, JFrame can be resized and closed and Menu Bar can be added to a JFrame for adding menus like File, Edit, etc.

```
import javax.swing.*;

public class FrameExample {
    public static void main(String[] args) {
        JFrame frame = new JFrame("My JFrame Example");
        frame.setSize(400, 300); // Set size of the frame
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // Close the app on window close
        frame.setVisible(true); // Make the frame visible
    }
}
```

JDialog(Dialog)

- JDialog is a secondary window that is typically used to display pop-up messages, collect input, or display additional information.
- Unlike a JFrame, a JDialog is often used for temporary interactions or dialogues with the user.
- It is often used for prompting the user for input or showing important information without taking focus away from the main window.
- It can be used when we need to show temporary interactions, like displaying alerts, error messages, or collecting user input (e.g., login forms or confirmation dialogs).
- It's ideal for situations where the user must make a decision before proceeding.
- JDialog can be modal (where the user must interact with it before returning to the main window) or non-modal (where the user can interact with both the dialog and the main window).

JDialog(Dialog) Application

```
import javax.swing.*;

public class DialogExample {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Main Window");
        frame.setSize(400, 300);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        // Creating a JDialog (non-modal)
        JDialog dialog = new JDialog(frame, "Information", true); // 'true' makes it modal
        dialog.setSize(200, 150);
        dialog.add(new JLabel("This is a dialog message"));
        dialog.setVisible(true); // Show the dialog

        frame.setVisible(true); // Show the main frame
    }
}
```

Event Handling and Listener Interfaces

- In Java, event handling is a mechanism that allows an application to respond to user actions or events, such as clicking a button, moving the mouse, or typing in a text field.
- Java provides an event-driven programming model where different components (buttons, text fields, etc.) can trigger events, and these events are handled by event listeners.
- Event handling in Java Swing is based on an event-driven model, where the user actions (events) trigger the listener methods to perform the desired tasks.
- In event handling mechanism:
  - An Event source generates events.
  - An Event listener responds to those events.
  - The Event object carries details about the event, such as its type, source and additional data.

Event Handling and Listener Interfaces

- **Event Source**
  - The GUI component that triggers an event, e.g., a button, text field, or menu.
  - Events are generated when the user interacts with these components.
  - Example: A JButton generates an action event when clicked.
- **Event Listener**
  - An interface that defines methods for handling specific types of events.
  - Listens for events and executes code when the event occurs.
  - Example: ActionListener is used for handling button clicks.
- **Event Object**
  - **Encapsulates information about the event.**
  - **Subclasses of java.util.EventObject represent specific events, such as:**

    ActionEvent, MouseEvent, KeyEvent and WindowEvent

Event Handling and Listener Interfaces

- **Listener interfaces** in Java are part of the event delegation model, which allows GUI components to handle specific types of user interactions.
- These interfaces define methods that are triggered when specific events occur.

Most Common Listener Interfaces

- **ActionListener**
  - Handles action events, typically generated by buttons, menu items, or similar components.

```
JButton button = new JButton("Click Me");
button.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        System.out.println("Button clicked!");
    }
});
```

Event Handling and Listener Interfaces

- **MouseListener**
  - Handles mouse events such as mouse clicks, mouse enters and mouse exits.

```
JButton button = new JButton("Mouse Listener Example");
button.addMouseListener(new MouseListener() {
    public void mouseClicked(MouseEvent e) {
        System.out.println("Mouse clicked on button");
    }
    public void mouseEntered(MouseEvent e) {
        System.out.println("Mouse entered button");
    }
    public void mouseExited(MouseEvent e) {
        System.out.println("Mouse exited button");
    }
    public void mousePressed(MouseEvent e) {}
    public void mouseReleased(MouseEvent e) {}
});
```

Event Handling and Listener Interfaces

- **KeyListener**
  - Handles keyboard events like key presses and key releases.

```
JTextField textField = new JTextField();
textField.addKeyListener(new KeyListener() {
    public void keyPressed(KeyEvent e) {
        System.out.println("Key pressed: " + e.getKeyChar());
    }
    public void keyReleased(KeyEvent e) {
        System.out.println("Key released: " + e.getKeyChar());
    }
    public void keyTyped(KeyEvent e) {}
});
```

Event Handling and Listener Interfaces

- **WindowListener**
  - Handles window events, such as window opening, closing, minimizing, or maximizing.

```
JFrame frame = new JFrame("Window Listener Example");
frame.addWindowListener(new WindowListener() {
    public void windowClosing(WindowEvent e) {
        System.out.println("Window is closing");
    }
    public void windowOpened(WindowEvent e) {}
    public void windowClosed(WindowEvent e) {}
    public void windowIconified(WindowEvent e) {}
    public void windowDeiconified(WindowEvent e) {}
    public void windowActivated(WindowEvent e) {}
    public void windowDeactivated(WindowEvent e) {}
});
frame.setSize(300, 200);
frame.setVisible(true);
```

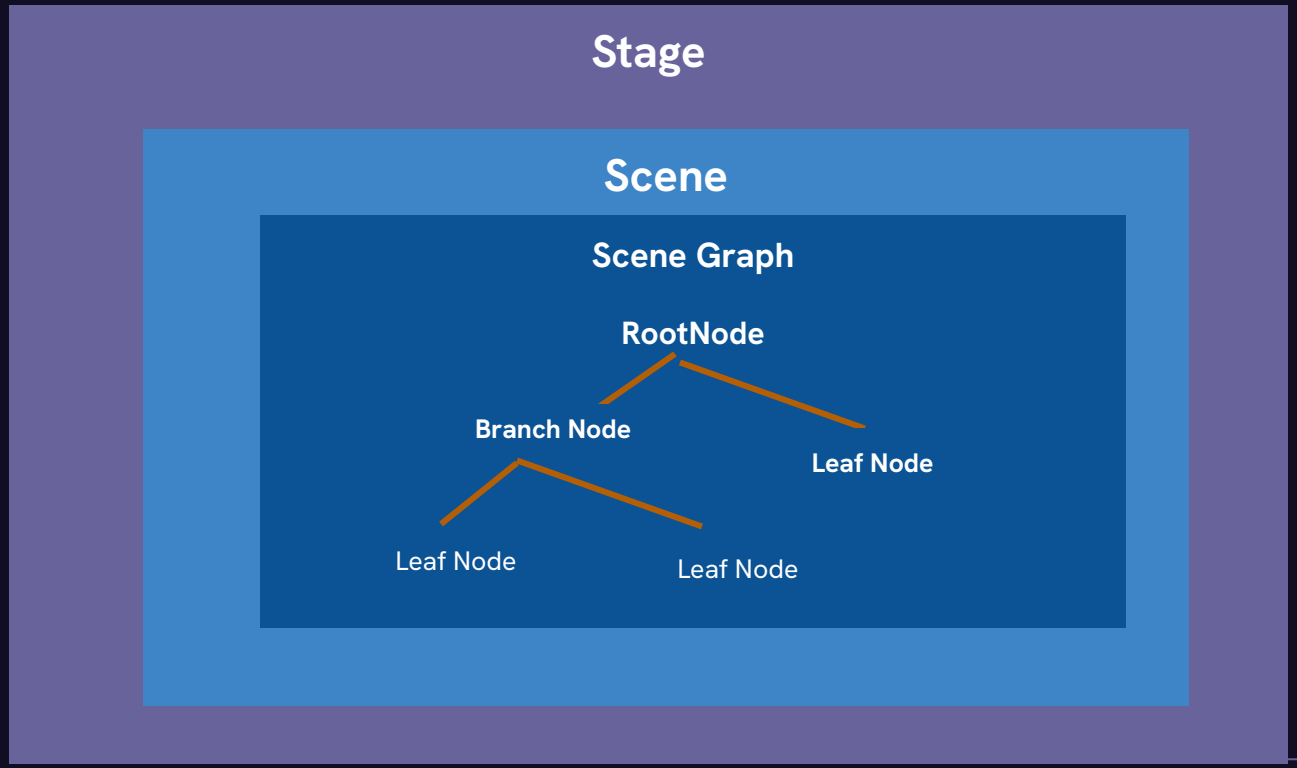
Event Handling

```
import javax.swing.*;
import java.awt.event.*;

public class EventHandlingExample {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Event Handling Example");
        JButton button = new JButton("Click Me");

        // Registering ActionListener for the button
        button.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                System.out.println("Button clicked!");
            }
        });

        frame.add(button);
        frame.setSize(300, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

JavaFX

- JavaFX is a modern Java framework used to create rich desktop and web applications with advanced GUI elements.
- It is a successor to Swing and AWT, offering improved features like CSS-based styling, FXML for UI design, and hardware-accelerated rendering for building Rich Internet Applications.
- It provides a set of graphics and media packages that allow developers to create modern, feature-rich graphical user interfaces (GUIs) with a rich set of controls, 2D and 3D graphics, animation, and multimedia support.
- JavaFX is part of the Java Standard Edition (SE) platform which seamlessly integrates with web services and APIs

JavaFX Structure



JavaFX Application

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class JavaFXDemo extends Application {
    @Override
    public void start(Stage primaryStage) {
        // Create components
        Button btn = new Button("Click Me");
        btn.setOnAction(e -> System.out.println("Hello, JavaFX!"));
        // Layout
        VBox layout = new VBox(10);
        layout.getChildren().add(btn);
        // Scene
        Scene scene = new Scene(layout, 300, 200);
        // Stage
        primaryStage.setTitle("JavaFX Example");
        primaryStage.setScene(scene);
        primaryStage.show();
    }
    public static void main(String[] args) {
        launch(args);
    }
}
```

Core Components of JavaFX

Stage

- The primary container for JavaFX applications which represents the window.
- Similar to Frame in a Swing Application, the JavaFX has the stage. Stage mainly acts as a container for all the JavaFX objects. The platform creates the primary stage internally, while the other stages can be further created by the application.

```
Stage stage = new Stage();
stage.setTitle("My JavaFX Application");
stage.show();
```

Core Components of JavaFX

Scene

- Scene is a container that holds all visual elements in a JavaFX application.
- To be able to visualize the contents on the stage, it is necessary to create a scene.
- Each scene object can only be added to one stage.
- The Scene can be easily created using the Scene class object by passing the layout object into the Scene class constructor.

```
Scene scene = new Scene(new Button("Click Me"), 400, 300);
stage.setScene(scene);
```

Core Components of JavaFX

Node

- The building blocks of the UI in JavaFX. Every UI component is a node.
- A node can be anything like a 2D or 3D geometric object (circle, sphere), container/layout objects (Border Pane, Grid Pane), or media element objects (audio, video,).
- Each node in the scene graph has a parent, if a node does not have a parent, then that is the root node. Similarly, each node can have one or multiple children. A node without any children is known as a leaf node, while the nodes having children are called branch nodes
- Categories of Node
  - Control Node: Interactive Components like Button, Textfield.
  - Layout Node: Container like Hbox, Vbox, GridPane
  - Shape Node: Graphics like Circle , Rectangle

```
Button button = new Button("Click");
```

Core Components of JavaFX

Layout

- Layout organizes nodes in a structured way within the scene.
- Types
  - HBOX : Horizontal Arrangement.
  - VBOX: Vertical Arrangement.
  - GridPane: Tabular Layout
  - Border Pane: divides the layout into top, bottom, left, right and center regions.

```
VBox layout = new VBox(10);  // Spacing 10px
layout.getChildren().addAll(new Label("Name:"), new TextField());
```

Core Components of JavaFX

Controls

- Predefined interactive components to capture user input or actions.
- JavaFX includes a rich set of built-in UI controls, such as buttons, text fields, tables, and more.
- It also provides charting components for creating various types of charts (line charts, bar charts, pie charts, etc.).

```
Button btn = new Button("Submit");
btn.setOnAction(e -> System.out.println("Button Clicked"));
```

Core Components of JavaFX

Media Management

- JavaFX provides built-in support for media playback, including audio and video.
- It supports common media formats and provides classes like MediaPlayer for handling media content.
- As a component of Media Management, Media represents the Media resource, Media Player controls the playback and Media view displays the media content.

```
Media media = new Media("file:/path/to/video.mp4");
MediaPlayer player = new MediaPlayer(media);
MediaView view = new MediaView(player);
player.play();
```

Core Components of JavaFX

FXML

- JavaFX supports FXML, an XML-based markup language that allows developers to define the user interface of an application in a declarative way
- FXML provides a separation of UI design and application logic
- It makes comfortable while designing

```
<Button text="Click Me" fx:id="myButton" onAction="#handleClick" />
```

Core Components of JavaFX

CSS Styling

- JavaFX applications can be styled using Cascading Style Sheets (CSS), providing a consistent and flexible way to customize the appearance of UI components.
- It makes the styling consistent across all applications.

```
.button {
    -fx-background-color: blue;
    -fx-text-fill: white;
}
```

Core Components of JavaFX

Webview

- JavaFX includes a WebView component that allows developers to embed web content within JavaFX applications.
- This is useful for integrating web-based functionality into desktop applications.

Swing vs JavaFX

Aspect	Swing	JavaFX
Introduction	Introduced in the 1990s as a part of Java SE.	Introduced in 2008 as a successor to Swing.
GUI Design Approach	Procedural code-based design.	Allows both code-based and FXML (declarative) design.
Styling	Limited styling options, primarily programmatic.	Extensive styling using CSS for modern and consistent UI designs.
Multimedia Support	Minimal multimedia support; requires external libraries.	Built-in support for audio, video, and images.
Performance	Heavier and slower due to AWT dependencies.	Lightweight with hardware-accelerated rendering for better performance.
Animation Support	Basic and limited animation features.	Advanced animation and transition effects out-of-the-box.

Swing vs JavaFX

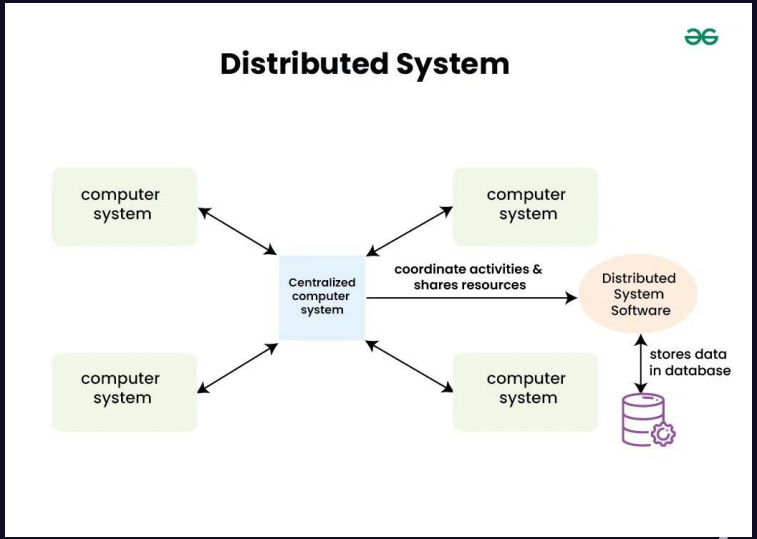
Aspect	Swing	JavaFX
Cross-Platform	Fully cross-platform but lacks modern features for responsive design.	Fully cross-platform with better adaptability for modern applications.
Hardware Utilization	Uses CPU for rendering.	Utilizes GPU for faster and smoother graphics rendering.
Modernity	Outdated for modern application needs.	Designed for building modern, visually rich applications.
Component Variety	Provides standard controls (e.g., JButton, JLabel).	Richer set of controls and layouts with features like WebView.
Ease of Learning	Straightforward but verbose for large applications.	Easier for modern applications, especially with FXML.
Dependency	Part of the Java Development Kit (JDK).	Requires separate installation in earlier JDK versions, integrated in JDK 11+.
Use Cases	Legacy enterprise applications, internal tools.	Modern desktop applications, multimedia applications, and web-integrated UIs.

Distributed Network Programming

04

Distributed System?

- A distributed system is a collection of independent computers or nodes that work together as a unified system to provide a coherent computing environment.
- These systems are designed to share resources, data and processing power, often appearing to users as a single, integrated system.
- Distributed systems are prevalent in various domains, including cloud computing, data storage and large-scale applications.

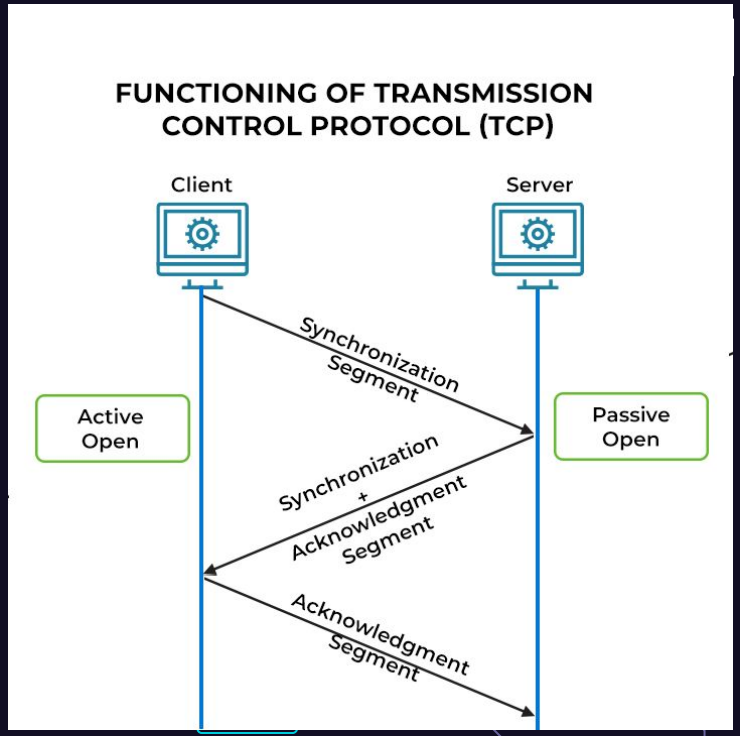


Distributed Network Programming?

- Distributed network programming involves developing software that operates across multiple computers or devices connected via a network, working together to perform tasks.
- It focuses on creating applications where different parts of the system are distributed across various nodes, which communicate and coordinate to achieve common goals
- These systems are distributed across different machines, often connected via a network and they function as a unified system to perform complex tasks.
- It is a fundamental concept in modern computing, powering technologies like cloud services, web applications, and distributed databases.

TCP

- TCP (Transmission Control Protocol) is a reliable, connection-oriented protocol that facilitates the transmission of byte streams between two devices over a network.
- It is designed to ensure the successful delivery of data packets across the internet. As one of the core protocols defined by the Internet Engineering Task Force (IETF).
- TCP plays a crucial role in internet communications. It is widely used in digital networks to guarantee end-to-end data delivery.
- TCP manages the transmission of data between a server and a client by organizing it into smaller packets, ensuring the data's integrity throughout the process.
- Before data transmission begins, TCP establishes a connection between the source and destination, maintaining this connection until the communication is complete. This process ensures that all data is delivered accurately and in the correct sequence.

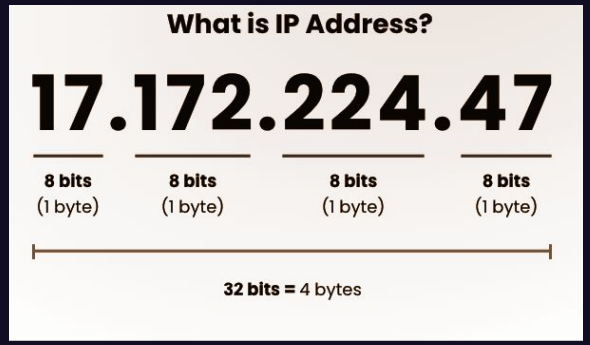


Common Characteristics of TCP

- Connection-oriented:
  - Establishes a connection before data transfer and ensures reliable delivery of data.
- Ordered and error-checked:
  - Guarantees that data is received in the order it was sent and performs error-checking to ensure data integrity.
- Acknowledgment:
  - Requires acknowledgment from the receiving end for each packet sent.

IP Address

- An IP (Internet Protocol) address is a unique numerical identifier assigned to each device connected to a computer network that uses the Internet Protocol for communication.
- IP governs how devices and applications exchange data packets, serving as the main protocol for defining the rules and formats for data exchange between computers on one or multiple networks connected via the internet.
- It operates through the Internet Protocol Suite (TCP/IP), which consists of four abstraction layers. IP is a key component of the internet layer within this suite, ensuring the delivery of data packets from the source application or device to the destination.
- It does this by embedding address information within the data packets, enabling accurate data transmission.

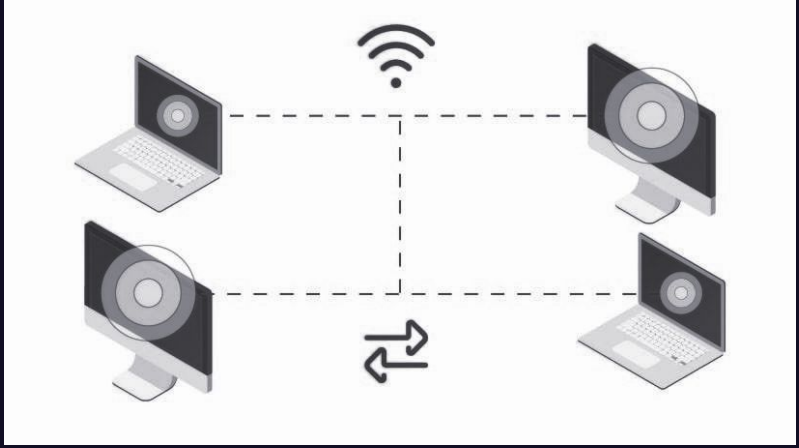


IP Address: Characteristics

- Unique identifier:
  - Every device connected to a network has a unique IP address.
- IPv4 and IPv6:
  - IPv4 addresses are 32-bit numerical labels written in decimal format, while IPv6 addresses are 128-bit written in hexadecimal format.
- IPv4 example:
  - 192.168.1.1
- IPv6 example:
  - 2001:0db8:85a3:0000:0000:8a2e:0370:7334

TCP/IP

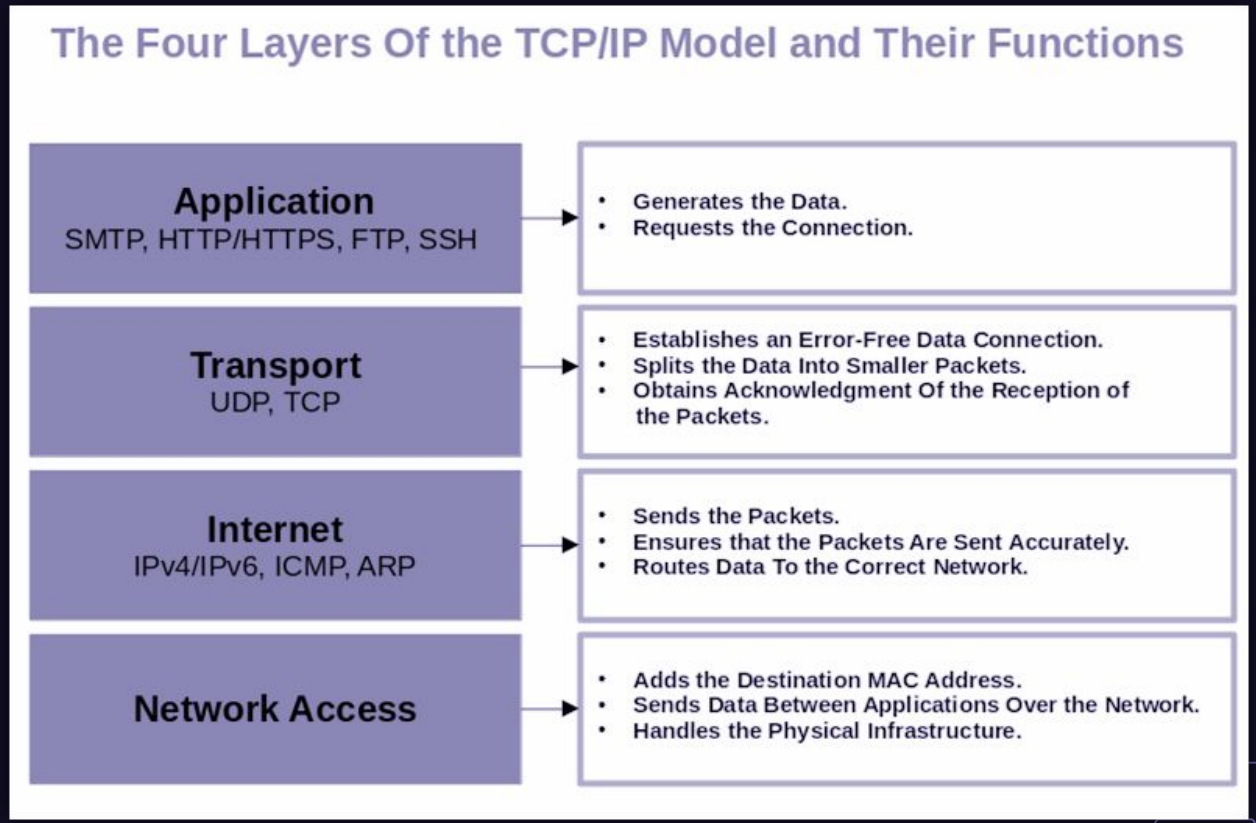
- The TCP/IP works by breaking messages into smaller packets, which are then sent independently which prevents the need to resend the entire message if an issue arises during transmission. Once these packets reach their destination, they are reassembled into the original message. Each packet can take a different route to avoid congested or unavailable paths.
- TCP/IP organizes communication into four layers, which standardize the process and remove the burden of managing communication from hardware and software providers. Data packets pass through these layers in sequence when sent and in reverse order when received.



TCP/IP

- It uses a connection-based approach where TCP establishes and maintains a connection between devices until data exchange is complete. TCP handles breaking messages into packets, numbering them, reassembling them, and managing the flow of data, ensuring all packets are delivered accurately. It also manages retransmissions for any lost packets.
- Like, when an email is sent using SMTP, the TCP layer of the sending server breaks the message into packets and sends them through the IP layer to the recipient's email server. Once the packets arrive, TCP reassembled them into the original message, which the server then delivers to the recipient's inbox.
- TCP/IP uses a three-way handshake to establish a connection between a device and a server. This handshake ensures both devices synchronize and acknowledge each other's packets before starting communication, allowing data to flow smoothly in both directions.

TCP/IP: layers



Application Layer

- Application Layer is where user-level programs operate and interact with the network. It provides the interface for software applications to communicate with other devices using the TCP/IP protocols.
- This layer includes various protocols that facilitate communication for services like email, web browsing and file transfers.
- The Application Layer in the TCP/IP model consolidates the functions of the Session, Presentation and Application Layers of the OSI (Open Systems Interconnection) model.
- This integration simplifies the model, focusing on the end-user processes such as data encoding, encryption and session management, which in the OSI model are handled separately.
- Protocols Supported by Application Layer
  - HTTP/HTTPS
  - SMTP
  - FTP

Transport Layer

- The Transport Layer is essential for creating a reliable connection between the source application or device and its intended destination.
- In this layer, data is broken into smaller packets, each given a sequence number to ensure they arrive in the correct order and manages how much data is sent, where it is sent and how quickly it is transmitted.
- The Transport Layer makes sure that all packets are delivered without errors, in the proper order and confirms that the destination has received them through acknowledgments which makes sure for smooth and accurate data communication between devices.
- Protocols Supported by TransportLayer
  - TCP
  - UDP

Internet Layer

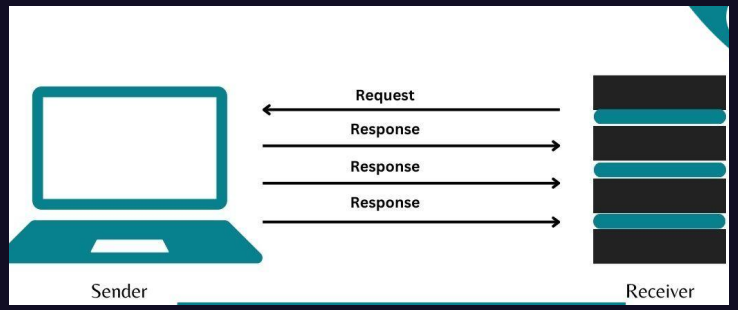
- The Internet Layer is responsible for transmitting data packets across a network and ensuring they reach their destination correctly.
- It handles the movement of data between devices and applications on different networks.
- It defines the protocols and procedures necessary for routing and delivering data sequences from the source to the destination.
- Protocols Supported by Internet Layer
  - IPv4
  - IPv6
  - ICMP

DataLink / Network Access Layer

- The Data Link Layer is responsible for defining how data is transmitted over a network and handling the physical transmission of data between devices or applications.
- It controls how data is signaled by hardware devices like a computer's network interface card (NIC), Ethernet cables or wireless networks and ensures that data is correctly sent and received across the network.
- It is sometimes called the Link Layer, Network Access Layer, Network Interface Layer or Physical Layer. In the OSI model, it combines both the physical and data link layers, which standardize communication functions for computing and telecommunications systems.
- Protocols Supported by Internet Layer
  - Ethernet
  - Wi-Fi

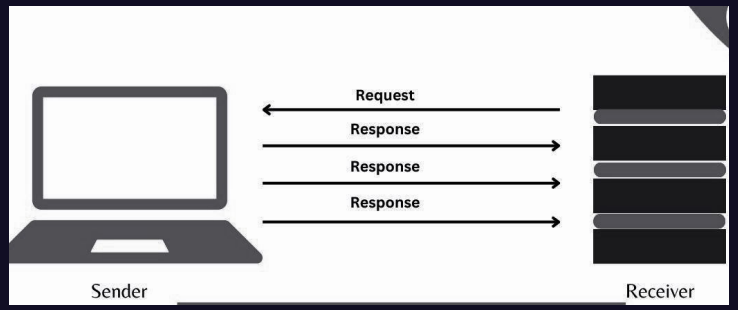
User Datagram Protocol(UDP)

- UDP (User Datagram Protocol) is a connectionless and lightweight protocol used to send short messages between applications on different computers.
- Unlike TCP, UDP is designed for low-latency communication, making it faster for time-sensitive applications.
- It does not have the mechanisms of TCP that ensure data delivery, such as error-checking, packet sequencing or acknowledgments.
- While UDP is less reliable, it is more efficient and suitable for situations where speed is more important than reliability, such as DNS lookups, VoIP (Voice over IP) and streaming media.
- UDP sacrifices reliability and error correction for efficiency, making it perfect for time-sensitive applications where some data loss is acceptable.



User Datagram Protocol(UDP)

- UDP (User Datagram Protocol) is a connectionless and lightweight protocol used to send short messages between applications on different computers.
- Unlike TCP, UDP is designed for low-latency communication, making it faster for time-sensitive applications.
- It does not have the mechanisms of TCP that ensure data delivery, such as error-checking, packet sequencing or acknowledgments.
- While UDP is less reliable, it is more efficient and suitable for situations where speed is more important than reliability, such as DNS lookups, VoIP (Voice over IP) and streaming media.
- UDP sacrifices reliability and error correction for efficiency, making it perfect for time-sensitive applications where some data loss is acceptable.

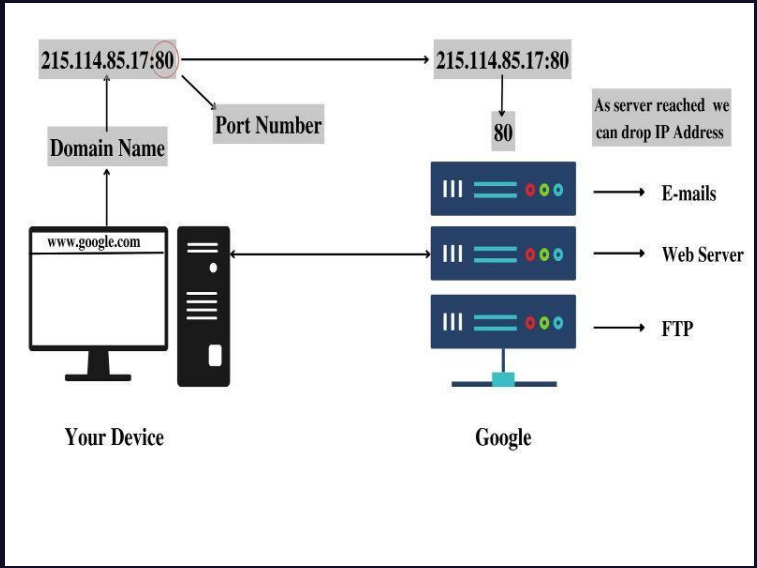


Ports

- A port is a virtual point where network connections are established and terminated which is managed by the computer's operating system, each port is linked to a specific process or service.
- Ports enable computers to distinguish between different types of network traffic, such as separating email traffic from web traffic, even though both utilize the same internet connection.
- They act as logical communication endpoints, allowing multiple services or processes to share the same IP address without conflict.
- A port is a 16-bit number ranging from 0 to 65535. It helps in directing network traffic to the appropriate application or service on a device.
- Well-Known Ports (0-1023):
  - HTTP (80), HTTPS (443), SMTP(25) and FTP(21)
- Registered Ports (1024-49151): Used by specific applications or services.

Ports

- Dynamic (Ephemeral) Ports (49152-65535): Temporarily assigned by the operating system for short-lived communications, such as client-side connections.
- Ports work with TCP and UDP at the transport layer to manage the flow of data between devices.
- IP addresses identify devices on a network and ports ensure the data reaches the correct application or service. Together, they form a complete address for network communication, commonly referred to as a socket (e.g., 192.168.1.1:80).
- Ports are fundamental for organizing and managing network traffic, ensuring different services can operate concurrently on a single device.
- Familiarity with port usage is crucial for networking and cybersecurity.



Socket Programming using TCP

ServerSide [Server Class]

```
import java.io.*;
import java.net.*;
public class Server {
   public static void main(String[] args) {
       int port = 9999;
       try (ServerSocket serverSocket =new ServerSocket(port)) {
           System.out.println("Server on port " + port);
           while (true) {
               try (Socket socket = serverSocket.accept()) {
                   System.out.println("Client connected");
                   InputStream input = socket.getInputStream();
                   BufferedReader reader =new BufferedReader(new InputStreamReader(input));
                   String message = reader.readLine();
                   System.out.println("Message from Client: " + message);

                  // Sending response to client
                  OutputStream output = socket.getOutputStream();
                   PrintWriter writer =new PrintWriter(output, true);
                   writer.println("Message sent from Server" + " " + message);
               } catch (IOException ex) {
                   System.out.println("Server exception: " + ex.getMessage());
               }
           }
       } catch (IOException ex) {
           System.out.println("Server exception: " + ex.getMessage());
       }
   }
}
```

Socket Programming using TCP

Client Side[Client Class]

```
import java.io.*;
import java.net.*;
public class ClientLocal {
   public static void main(String[] args) {
       String hostname = "localhost";
               int port = 9999;
               try (Socket socket = new Socket(hostname, port)) {
                   // Sending message to the server
                   OutputStream output = socket.getOutputStream();
                    PrintWriter writer =new PrintWriter(output, true);
                    writer.println("Hello, Iam Client!");

                   // Reading response from the server
                   InputStream input = socket.getInputStream();
                    BufferedReader reader =new BufferedReader(new InputStreamReader(input));
                    String response = reader.readLine();
                    System.out.println("Server response: " + response);

               } catch (UnknownHostException ex) {
                    System.out.println("Server not found: " + ex.getMessage());
               } catch (IOException ex) {
                    System.out.println("I/O error: " + ex.getMessage());
               }
           }
       }
```

Socket Programming using UDP

Server Side [Server Class]

```
import java.net.DatagramPacket;
import java.net.DatagramSocket;
import java.net.InetAddress;
public class MyServer {
   public static void main(String[] args) {
       DatagramSocket socket =null;
       try {
           socket = new DatagramSocket(9876);
           byte[] receiveBuffer = new byte[1024];
           System.out.println("Server is running");
           while (true) {
               //Received from Client
               DatagramPacket receivePacket =new DatagramPacket(receiveBuffer, receiveBufferlength);
               socket.receive(receivePacket);
               String receivedData =new String(receivePacket.getData(),0, receivePacket.getLength());
               System.out.println("Received: " + receivedData);
               //Send back the message to Client
               InetAddress clientAddress = receivePacket.getAddress();
               int clientPort = receivePacket.getPort();
               String responseData ="Hello from server";
               byte[] sendBuffer = responseData.getBytes();
```

Socket Programming using UDP

Server Side [Server Class] Continue...

```
            DatagramPacket sendPacket =new DatagramPacket(sendBuffer, sendBufferlength, clientAddress,clientPort);
                socket.send(sendPacket);
            }
        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            if (socket != null && !socket.isClosed()) {
                socket.close();
            }
        }
    }
}
```

Socket Programming using UDP

Client Side [Client Class]

```
import java.net.DatagramPacket;
import java.net.DatagramSocket;
import java.net.InetAddress;
public class MyClient {
   public static void main(String[] args) {
       DatagramSocket socket =null;
       try {
           //Sent to Server
           socket = new DatagramSocket();
           InetAddress serverAddress = InetAddressgetByName("localhost");
           int serverPort = 9876;
           String message ="Hello from client";
           byte[] sendBuffer = message.getBytes();
           DatagramPacket sendPacket =new DatagramPacket(sendBuffer, sendBufferlength, serverAddress,serverPort);
           socket.send(sendPacket);
           //Received from Server
           byte[] receiveBuffer =new byte[1024];
           DatagramPacket receivePacket =new DatagramPacket(receiveBuffer, receiveBufferlength);
           socket.receive(receivePacket);
           String receivedData =new String(receivePacket.getData(),0, receivePacket.getLength());
           System.out.println("Received from server: "+ receivedData);
```

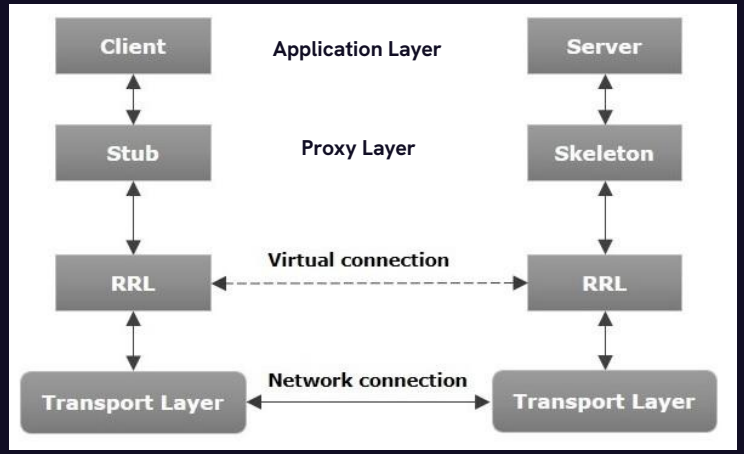
Socket Programming using UDP

Client Side [Client Class] Continue...

```
        } catch (Exception e) {
            System.out.println(e.getMessage());
        } finally {
            if (socket != null && !socket.isClosed()) {
                socket.close();
            }
        }
    }
}
```

RMI Architecture

- RMI is a Java API that facilitates the development of distributed applications by allowing an object running in one Java Virtual Machine (JVM) to invoke methods on an object in another JVM.
- RMI provides a high-level abstraction for executing code across different machines in a network, making it easier to create distributed, networked applications in Java.
- The RMI architecture is structured to facilitate the communication between clients and remote objects, enabling the invocation of methods on objects located in different Java Virtual Machines (JVMs).
- Clients invoke methods on remote objects without needing to know the location or details of the remote object.
- Unlike other distributed computing models, RMI is designed specifically for Java, taking full advantage of its platform independence and object serialization capabilities.



Key Components of RMI

- **Stub**
  - The stub is a client-side proxy that represents the remote object. When a client makes a method call on the stub, it forwards the call to the remote object through the network.
  - It serves as the intermediary between the client and the remote object, enabling the client to invoke methods on the remote object as though it were a local method call.
  - It simplifies the remote communication process by abstracting the underlying complexity of network interactions, serialization,and deserialization
  - The stub is an object, acts as a gateway for the client-side. All the outgoing requests are routed through it.
  - The client interacts with the stub as if it were the actual remote object, which means the client can invoke methods on the remote object without needing to know the details of where it resides (possibly on a different machine).
  - In modern Java (since JDK 5), the stub is generated automatically through dynamic proxies, eliminating the need for manually creating stubs.

Key Components of RMI

- **Skeleton**
  - The skeleton is a server-side component that received remote method calls from the client-side stub and invoked the appropriate method on the remote object and then returned the result to the stub.
  - The skeleton is an object, acts as a gateway for the server-side object.
  - It is responsible for unmarshaling the incoming data and invoking the actual method on the remote object.
  - Instead of using a skeleton to receive and dispatch remote method calls, from Java 5, it now uses the RMI runtime and dynamic proxies to handle the method invocations automatically.

Key Components of RMI

- **RMI Registry**
  - The RMI Registry is a core component of the Java Remote Method Invocation (RMI) architecture, acting as a directory service that allows remote objects to be located and accessed by client applications.
  - It plays an essential role in the communication between RMI clients and servers by binding remote objects to names, enabling clients to look up and invoke methods on these remote objects.
  - It is a simple server-side naming service provided by Java that binds names (strings) to remote objects (or their stubs) and serves as a directory where remote objects are registered and enables clients to look up and access those objects using the names they were registered with.
  - It is a crucial part of the RMI architecture, acting as a directory service that enables clients to find and access remote objects by name. It simplifies the process of remote communication by allowing clients to look up and invoke methods on remote objects as though they were local

Key Components of RMI

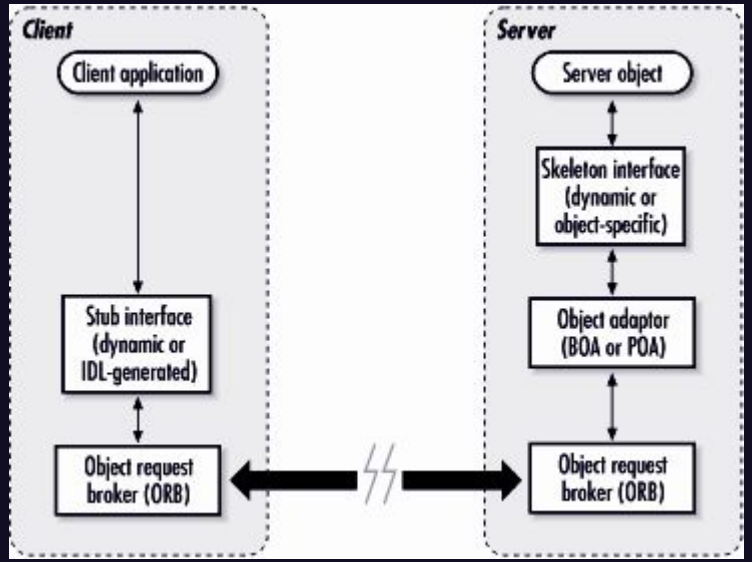
- **Transport Layer**
  - The Transport Layer in RMI is a critical component responsible for the actual communication and data transfer between the client and server.
  - It provides a communication channel over which method invocations and data are sent between the client and the server.
  - It operates below the application layer (which includes the RMI runtime and other higher-level components) and works by utilizing TCP/IP or UDP protocols to ensure reliable and efficient communication across the network.
  - It ensures that the communication is reliable and handles the low level detail of network communication
  - RMI relies on network communication so latency and bandwidth limitations can affect performance.

Key Components of RMI

- **Remote Reference Layer(RRL)**
  - It is a critical part of the RMI architecture, enabling the abstraction of remote objects and making it easier for clients to invoke methods on objects located in different Java Virtual Machines.
  - It is responsible for managing the references to remote objects and It handles the process of invoking remote methods on objects located on a different JVM (Java Virtual Machine) across a network.
  - When a remote method is invoked on an object, the Remote Reference Layer manages the transmission of the method invocation request from the client to the server, ensuring that the call reaches the correct method of the remote object.
  - It ensures that remote references are handled correctly, keeping track of them throughout the lifecycle of the remote method invocation and guarantees that the correct method is called on the remote object by using remote references in the correct context.

CORBA

- CORBA is a standard defined by the Object Management Group (OMG) to enable communication between distributed objects in a network.
- It allows objects written in different programming languages and running on different platforms to communicate with each other seamlessly
- CORBA enables distributed computing by providing a standard way for objects to communicate across platforms and languages, abstracting the complexity of network communication and offering features like object location, security, and transaction management.



CORBA Components

- **Object Request Broker(ORB)**
  - The Object Request Broker (ORB) is the central component of the CORBA architecture which acts as a middleware that facilitates communication between distributed objects.
  - It manages object references, marshalling and unmarshalling of data, routing requests to the appropriate servers, and returning responses to clients.
  - It abstracts the complexity of network communication, allowing objects to communicate seamlessly even if they reside on different machines or are written in different languages.
  - It handles the mapping of object references to actual objects on the server.
- **Stubs**
  - A stub is a client-side proxy object that acts as a representative of a remote CORBA object.
  - When a client makes a method call on a stub, the stub forwards the call to the ORB, which in turn routes it to the appropriate server object.
  - The stub marshals the request, serializing the parameters, and sends it over the network to the server.
  - When receiving data, the stub unmarshals the response (i.e., converts the response data back into usable data) and returns it to the client.

CORBA Components

- **Skeletons**
  - Skeletons are server-side components that receive requests from the ORB and invoke the appropriate method on the server object.
  - The skeleton receives the request from the ORB, unmarshals the data and invokes the correct method on the server-side object.
  - After the method execution, the skeleton marshals the return values and sends them back through the ORB to the client.
  - It also handles exceptions and error conditions, ensuring proper communication and error reporting.
- **Object Mapper**
  - The object adapter is a server-side component that helps bind the CORBA object with the ORB. It provides an interface between the CORBA object and the ORB, ensuring that requests are properly routed to the correct object.
  - The object adapter registers the server object with the ORB so that the ORB can find and communicate with it.
  - It helps in dispatching incoming client requests to the appropriate methods in the server objects..

CORBA Components

- **Interface Definition Language (IDL)**
  - IDL is a language-neutral specification used to define the interfaces of CORBA objects. It allows developers to define the methods and attributes of an object without worrying about the programming language used to implement it.
  - IDL defines the structure of the interfaces of objects, specifying the methods, their parameters, and return types.
  - IDL is not dependent to a particular language so it allows objects to be implemented in various programming languages (such as C++, Java, Python, etc.) and still communicate with each other.
  - The IDL compiler generates stubs (client-side) and skeletons (server-side) in the target programming languages.

RMI Vs CORBA

Aspect	RMI (Remote Method Invocation)	CORBA (Common Object Request Broker Architecture)
Definition	RMI is a Java-specific API for remote communication, enabling method invocation on objects in different JVMs.	CORBA is a language-independent framework for enabling communication between objects in different systems, regardless of the programming language.
Platform Dependency	RMI is platform-dependent; it is primarily for Java-based applications.	CORBA is platform-independent and can be used across different operating systems and programming languages.
Communication Protocol	RMI typically uses Java's native protocol (JRMP) or IIOP (Internet Inter-ORB Protocol) for communication.	CORBA uses IIOP (Internet Inter-ORB Protocol) as the standard communication protocol for distributed object communication.
Supported Languages	RMI supports only Java applications.	CORBA supports multiple languages including Java, C++, Python, COBOL, and more.
Interface Definition	RMI interfaces are Java interfaces, and remote objects implement these interfaces.	CORBA uses **IDL (Interface Definition Language)** for defining interfaces, which can be mapped to multiple programming languages.

Aspect	RMI (Remote Method Invocation)	CORBA (Common Object Request Broker Architecture)
Complexity	RMI is simpler to use for Java-based applications, as it is fully integrated with Java's language features.	CORBA is more complex, as it deals with multiple languages, platforms, and an additional layer of abstraction (ORB).
Middleware	RMI uses a simpler communication layer and is tightly integrated with the Java environment.	CORBA requires an Object Request Broker (ORB), a middleware component that handles communication between objects.
Interoperability	RMI can only communicate with other Java-based systems.	CORBA supports interoperability between applications written in different programming languages across different platforms.
Usage	RMI is typically used for Java-to-Java communication in distributed systems.	CORBA is used for building large-scale, heterogeneous distributed systems with applications written in different programming languages.
Security Model	RMI has its own security mechanisms, often relying on Java's security manager and other Java-specific features.	CORBA has a built-in security framework for authentication, confidentiality, and authorization, often managed by the CORBA Security Service.
Deployment Complexity	Easier to deploy in Java-based systems as it requires only JVMs.	CORBA deployment is more complex, especially in heterogeneous environments, due to the need for ORBs and IDL compilers.

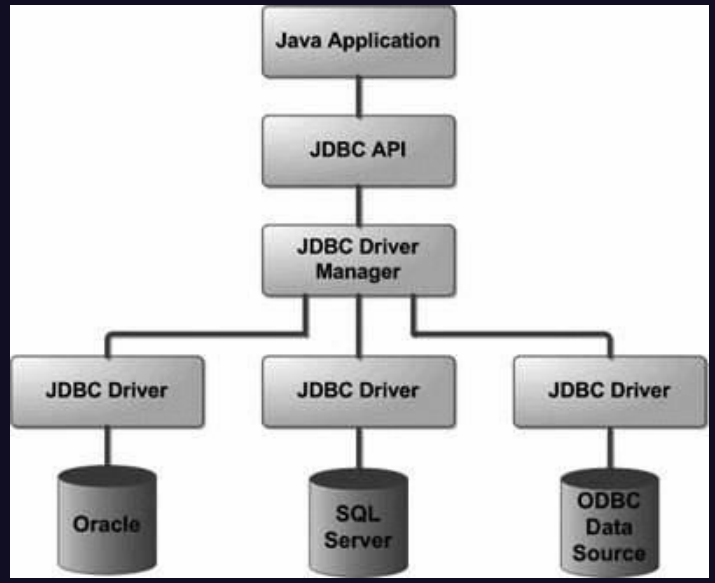
Database Connectivity

5

WHAT IS JDBC?

- JDBC (Java Database Connectivity) is a Java-based API (Application Programming Interface) that serves as a standardized framework for enabling Java applications to interact with relational databases.
- It facilitates the establishment of database connections, execution of SQL queries, retrieval of data, and modification of database records.
- It acts as a bridge between a Java application and a database, enabling seamless communication and data exchange. It is widely used in Java applications for integrating database functionality.
- It is an important tool for creating Java applications that need to work with databases as it makes easier to connect to and interact with databases, providing a common way to work with different database systems.
- Whether you're building web apps, desktop programs, or server-side applications, JDBC helps you easily add database features to your Java projects
- It is a API that is used to interact with databases, issue queries and commands and process database result sets. JDBC and database drivers operate together to access spreadsheets and databases. The API classes and interfaces of JDBC allow an application to send a request to a particular database. It helps Java applications interact with ORACLE, MYSQL, and MSSQL databases.

JDBC Architecture



JDBC Architecture

- Java Application
  - In the JDBC Architecture, the Java application serves as the foundation and primary layer that interfaces with the JDBC API. It is where developers implement the business logic to establish database connections, execute queries, and handle the resulting data.
- JDBC Api
  - The JDBC API (Java Database Connectivity Application Programming Interface) is a part of Java's standard library that provides a structured way for Java applications to interact with relational databases. It includes a set of classes, interfaces, and methods that enable developers to establish database connections, execute SQL queries, retrieve results and update database records.
- JDBC Manager Driver
  - The JDBC DriverManager is a core component of Java Database Connectivity (JDBC) that helps manage a set of database drivers. It acts as an interface between Java applications and the various database management systems (DBMS). The DriverManager class is responsible for managing database drivers and establishing a connection to the database.
  - It acts as a connection factory for creating database connections and provides the mechanism to establish a link between a Java application and a specific database using an appropriate JDBC driver.

JDBC Architecture

- JDBC Driver
  - These drivers are used to establish connections with various data sources. Different databases like MSSQL, Oracle, and MySQL each have their own specific drivers. To connect with these databases, we need to load their respective drivers. The Java class used for loading these drivers is Class, and the Class.forName() method is employed to load the drivers in the JDBC architecture.
- Data Source
  - These are the databases that can be accessed using the JDBC API. They serve as the storage sources for data, which can be utilized by Java applications. The JDBC API facilitates connections to various databases, including MySQL, MSSQL, PostgreSQL, Oracle, and others.

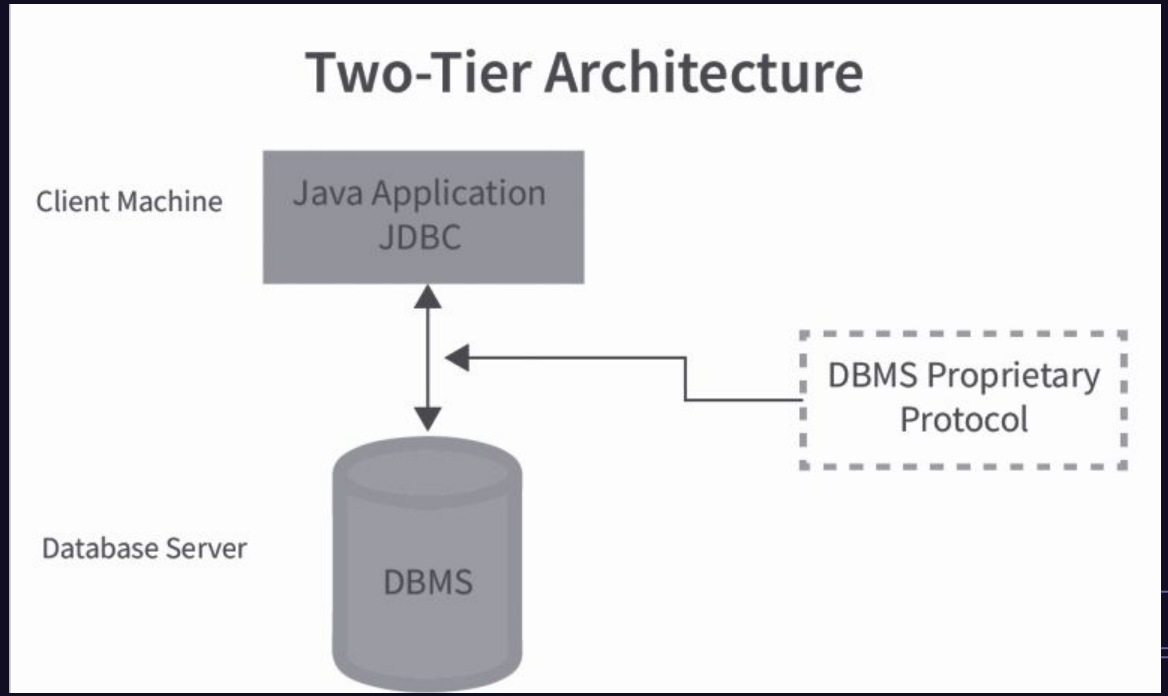
JDBC Architecture types

Two-Tier Architecture

- It is a basic model, where a Java application and Java applet (Client) communicate directly with the data source.
- JDBC acts as the bridge between the client application and the database.
- This architecture is typically used in small applications where the database and application are on the same or a closely connected system.
- The user's machine acts as a client, and the machine has the data source running acts as the server.
- Direct interaction between the client and database which makes it simple.
- Faster communication due to fewer layers.
- Limited scalability as all clients must maintain a connection to the database.

JDBC Architecture types

- Two-Tier Architecture



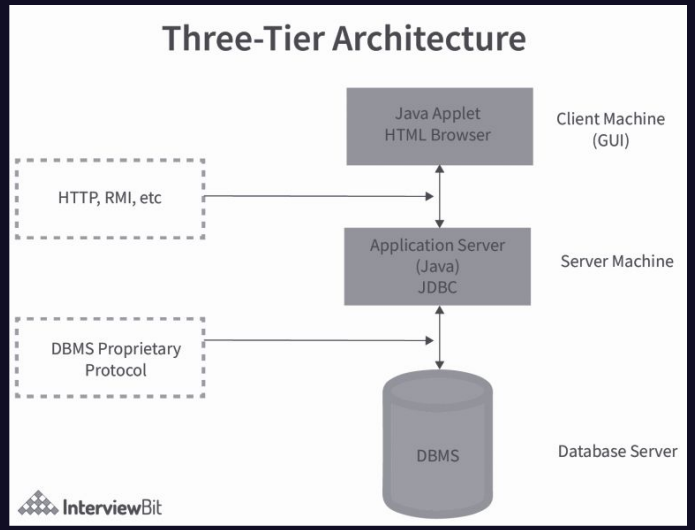
JDBC Architecture types

Three-Tier Architecture

- This Model is a more secure and complex model of JDBC architecture in Java The user queries are sent to the middleware tier and then executed on the data source
- The results received on the database are again sent to the middle tier and then to the application.
- There is an intermediate layer (often called the middleware) between the client and the database which can include an application server, web server, or any other logic-processing entity.
- Is more secure as client does not directly interact with the database.
- Is more scalable as middleware can manage connections to the database, allowing more clients to be supported.
- Business logic is separated into the middleware, which makes the system more modular and flexible.

JDBC Architecture types

- Three-Tier Architecture



JDBC Drivers

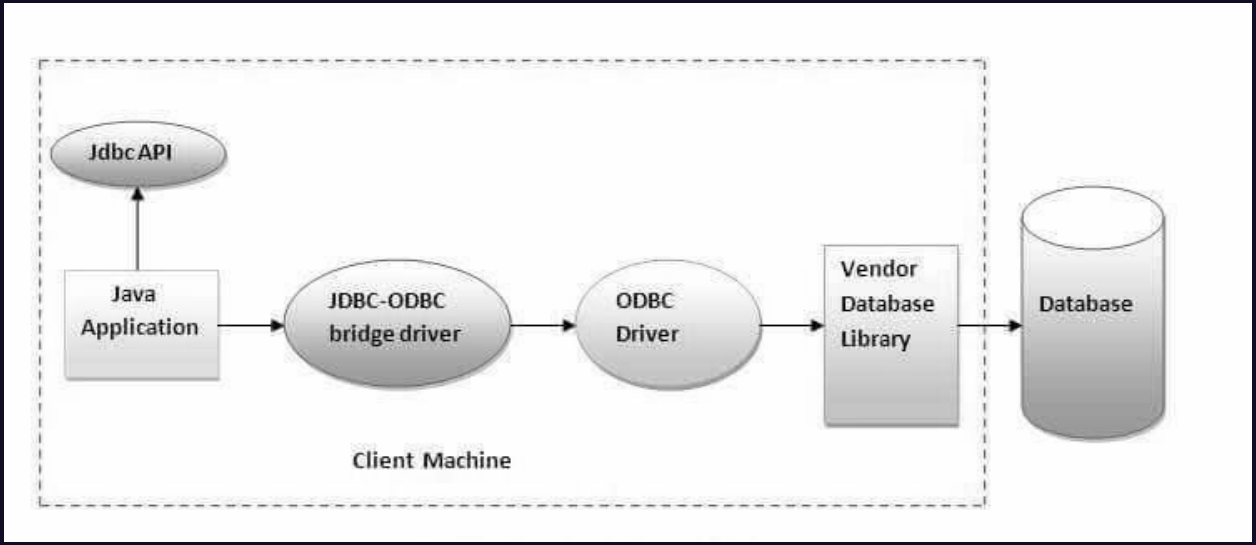
A JDBC driver is a software tool that helps Java applications communicate with databases. It works like a translator, converting Java instructions into commands that the database can understand. Using the JDBC driver, applications can send queries to the database, retrieve results, and update data in various types of databases. It handles the communication with the database server.

Type 1 : JDBC-ODBC bridge driver[Bridge Driver]

- The JDBC-ODBC Bridge Driver uses ODBC (Open Database Connectivity) to connect Java applications to a database. It translates JDBC calls into ODBC calls, which are then handled by the ODBC driver manager.
- The JDBC-ODBC bridge driver uses ODBC driver to connect to the database.
- The ODBC driver is platform-dependent, which makes this driver less efficient and deprecated in later versions of JDBC.
- Easy to use and can be easily connect to any database.
- As a drawback, ODBC is platform-specific, so the application needs to be on the same platform as the database.
- Performance bottleneck due to the translation layer between JDBC and ODBC.
- The ODBC driver needs to be installed on the client machine.

▶ JDBC Drivers

Type 1 : JDBC-ODBC bridge driver[Bridge Driver]



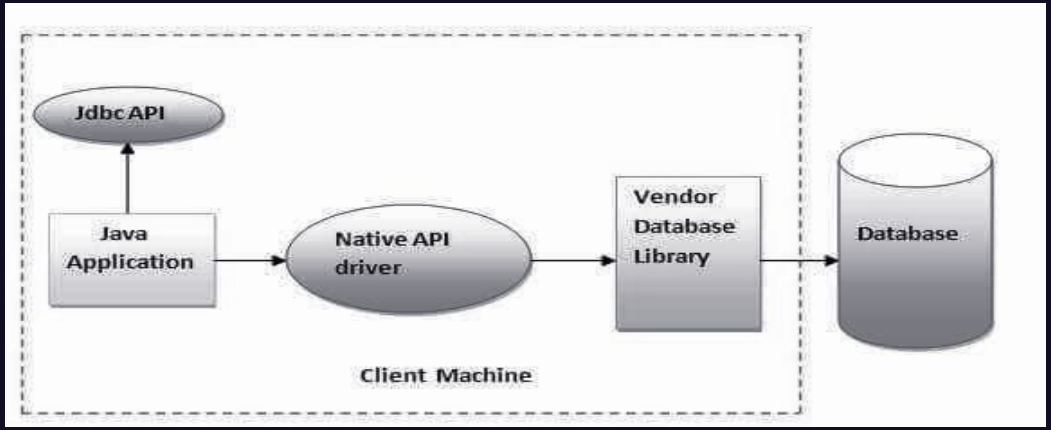
JDBC Drivers

Type 2 : Native API Driver[Partially Java Driver]

- The Native API driver relies on the database's client-side libraries for communication.
- It translates JDBC method calls into the native API calls specific to the database and to work with different databases, this driver requires the database's local API.
- This approach offers better security for data transfer compared to the Type-1 driver.
- The Native-API driver converts JDBC calls into database-specific native calls (using libraries written in languages like C, C++, or other low-level languages). These native calls are then used to communicate with the database.
- This driver communicates with the database using a native database client (for example, a native MySQL client or Oracle client) installed on the local machine.
- Performance upgraded than JDBC-ODBC bridge driver
- As a drawback, driver needs to be installed separately in individual client machines
- The Vendor client library needs to be installed on client machine.
- It is a database dependent driver.

▶ JDBC Drivers

Type 2 : Native API Driver[Partially Java Driver]



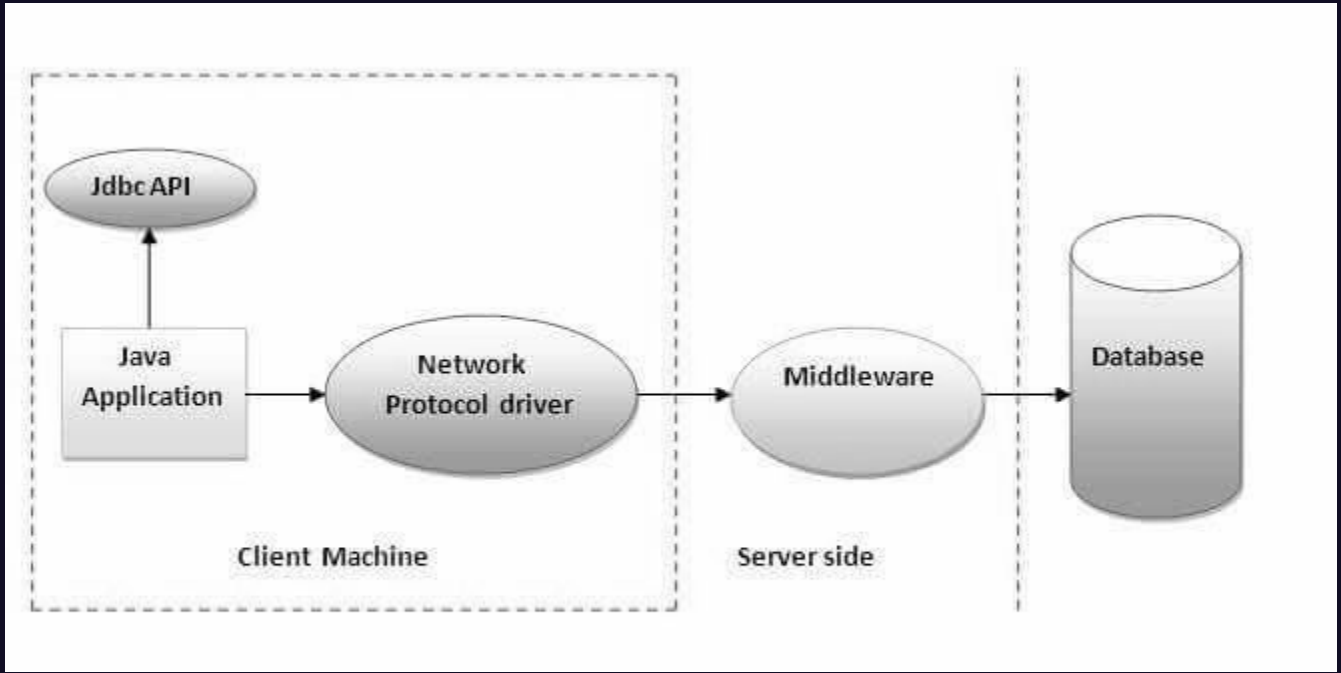
JDBC Drivers

Type 3 : Network Protocol Driver[Middleware Driver]

- The Network Protocol Driver communicates with the database server using a middle layer also known as middleware server which translates the JDBC calls into a database-specific protocol understood by the database.
- The driver sends the database communication over a network. When a JDBC call is made, it is passed to the middleware server, which then converts the call into the database-specific protocol and communicates with the database.
- It is platform independent as The driver can work across different platforms because the communication is done through a middle layer.
- As a drawback, it slower due to the added layer (middleware) and network latency.
- Requires additional infrastructure for the middleware and network support is required in client's machine.
- Maintenance of Network Protocol driver becomes costly because it requires database-specific coding to be done in the middle tier.

▶ JDBC Drivers

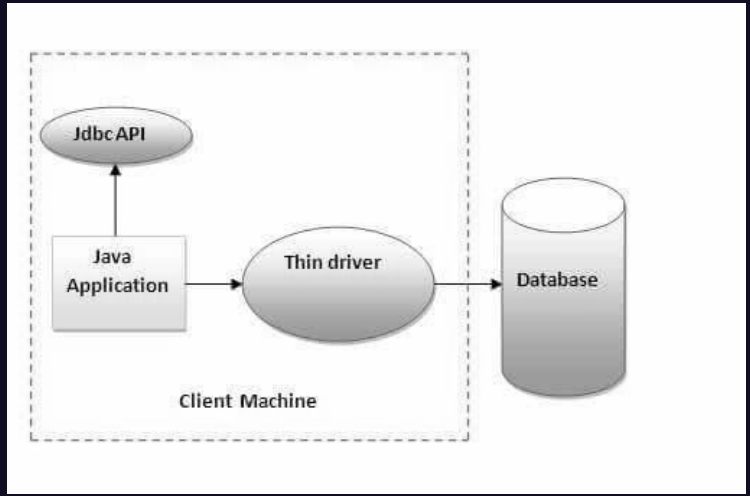
Type 3 : Network Protocol Driver[Middleware Driver]



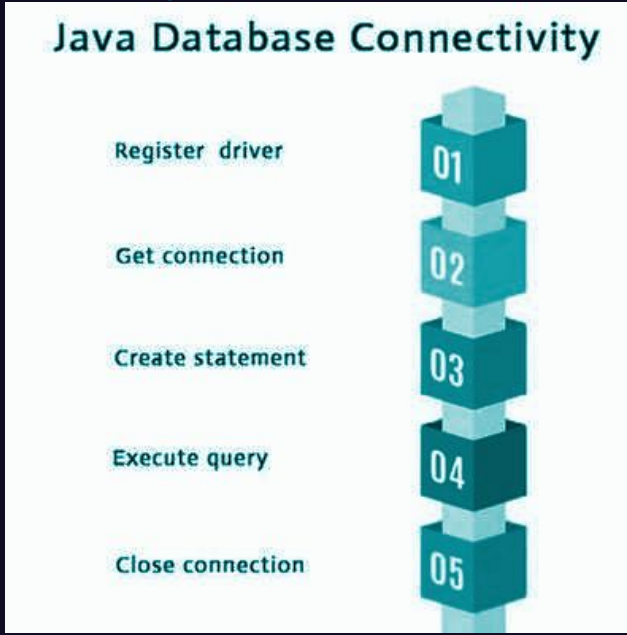
JDBC Drivers

Type 4 : Thin Driver[Pure Java Driver]

- Thin driver is a pure Java driver that communicates directly with the database using its native protocol, without requiring any native library or middleware.
- It's the most commonly used JDBC driver in modern Java applications because of its portability and efficiency.
- No software is required on either server or client side.
- Direct connection to the database without intermediate translation layers so it is better performance than all the drivers.



▶ JDBC Connectivity



▶ JDBC Connectivity

❖ Register Drivers

- ➢ The Class.forName() method is used to register the driver class. This method is used to dynamically load the driver class.
- ➢ Modern JDBC drivers like MySQL may no longer require this step as drivers are auto-registered since JDBC 4.0.

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

▶ JDBC Connectivity

❖ Create the connections

➢ The getConnection() method of DriverManager class is used to establish connection with the database.

```
Connection connection = DriverManager.getConnection(URL, username, password);
```

- *URL*
  - *The databaseURL typically in the format*
    - *jdbc:mysql://hostname:port/databaseName*
- *username*
  - *Database username*
- *password*
  - *Database password*

Eg:

```
String url = "jdbc:mysql://localhost:3306/javadatabase";
String username = "root";
String password = "password";
Connection connection = DriverManager.getConnection(url, username, password);
```

▶ JDBC Connectivity

❖ Create a Statement

- ➢ The Statement object is responsible for executing queries with the database.
- ➢ This object lets to execute SQL queries like SELECT, INSERT, UPDATE and DELETE.

```
Connection connection = DriverManager.getConnection(URL, username, password);
Statement statement = connection.createStatement();
```

▶ JDBC Connectivity

❖ Execute Query

- The executeQuery() method of Statement interface is used to execute queries to the database. This method returns the object of ResultSet that can be used to get all the records of a table.
  - executeQuery: Used for retrieving data (SELECT statements). Returns a ResultSet.
  - executeUpdate: Used for updating data (INSERT, UPDATE, DELETE). Returns an integer representing the number of rows affected.

```
ResultSet resultSet = statement.executeQuery("SELECT * FROM student");
```

▶ JDBC Connectivity

❖ Close the Connection

- ➢ This ensures the application doesn't leak memory or connections.

```
resultset.close()
statement.close()
connection.close()
```

```java
import java.sql.*;

public class JDBCExample {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/javadatabase";
        String user = "ktmncit";
        String password = "ncIT#123";
        try {
            // Load the driver (optional for JDBC 4.0+)
            Class.forName("com.mysql.cj.jdbc.Driver");
            // Establish the connection
            Connection connection = DriverManager.getConnection(url, user, password);
            System.out.println("Connected to the database!");
            // Create a statement
            Statement statement = connection.createStatement();
            // Execute a query; before to execute we need to have student table in our javadatabase
            String query = "SELECT * FROM student";
            ResultSet resultSet = statement.executeQuery(query);
            // Process the results
            while (resultSet.next()) {
                int id = resultSet.getInt("id");
                String name = resultSet.getString("name");
                System.out.println("ID: " + id + ", Name: " + name);
            }
            // Close the resources
            resultSet.close();
            statement.close();
            connection.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

▶ Result Set

- In JDBC , a ResultSet is an object that holds the data retrieved from a database after executing a SQL query. It acts as a pointer to the rows of the query result. The ResultSet object allows iteration through the retrieved rows and access to column values.
- The ResultSet object contains a cursor that points to the current row of data. Initially, the cursor is positioned before the first row, and it must be moved to access the rows.
- It is essentially a table of data representing the result of a database query.
- Based on cursor movement and operations, by default Result set is forward only and it read only respectively.
- For retrieving Data: The data can be retrieved from a ResultSet using methods like getString(), getInt(), getDate() etc.
- To Navigate: we can move through the rows of the result set, typically starting from the first row, using methods like next(), previous(), first() and last().

▶ Scrollable Result Set

- A scrollable ResultSet provides more flexibility by allowing both forward and backward movement through the data, which can be very useful in various scenarios like searching, updating or viewing large datasets.
- It is controlled when creating the statement.

```
Statement stmt = connection.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE,
ResultSet.CONCUR_READ_ONLY);
ResultSet rs = stmt.executeQuery("SELECT * FROM students");
```

▶ Updatable Result Set

- An updatable ResultSet in JDBC enables efficient data modification and management directly in the ResultSet, offering a convenient way to handle database changes in Java applications.
- It allows to retrieve data from a database and modify it directly within the ResultSet, which can then be reflected back in the database.

```
Statement stmt = conn.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE,
ResultSet.CONCUR_UPDATABLE);
ResultSet rs = stmt.executeQuery("SELECT * FROM students");
```

Scrollable and Readonly

```
import java.sql.*;

public class ScrollableResultSetExample {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/javadatabase";
        String user = "ktmncit";
        String password = "ncIT#123";
        try (Connection conn = DriverManager.getConnection(url, user, password);
            Statement stmt = conn.createStatement(
                    ResultSet.TYPE_SCROLL_INSENSITIVE,
                    ResultSet.CONCUR_READ_ONLY
            );
            ResultSet rs = stmt.executeQuery("SELECT id, name, age FROM students")) {

            // Move to the last row and display data
            if (rs.last()) {
                System.out.println("Last Row - ID: " + rs.getInt("id") +
                                   ", Name: " + rs.getString("name") +
                                   ", Age: " + rs.getInt("age"));
            }
            // Move to the first row and display data
            if (rs.first()) {
                System.out.println("First Row - ID: " + rs.getInt("id") +
                                   ", Name: " + rs.getString("name") +
                                   ", Age: " + rs.getInt("age"));
            }
            // Move to the second row and display data
            if (rs.absolute(2)) {
                System.out.println("Second Row - ID: " + rs.getInt("id") +
                                   ", Name: " + rs.getString("name") +
                                   ", Age: " + rs.getInt("age"));
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Scrollable and Updatable

```
import java.sql.*;
public class UpdatableResultSetExample {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/javadatabase";
         String user = "ktmncit";
        String password = "ncIT#123";
        try (Connection conn = DriverManager.getConnection(url, user, password);
             Statement stmt = conn.createStatement(
                    ResultSet.TYPE_SCROLL_SENSITIVE,
                    ResultSet.CONCUR_UPDATABLE
             );
             ResultSet rs = stmt.executeQuery("SELECT id, name, age FROM students")) {
            // Update the first row
            if (rs.first()) {
                rs.updateString("name", "Updated Name");
                rs.updateInt("age", 35);
                rs.updateRow();
                System.out.println("Row updated.");
            }
            // Insert a new row
            rs.moveToInsertRow();
            rs.updateInt("id", 104);
            rs.updateString("name", "New User");
            rs.updateInt("age", 28);
            rs.insertRow();
            System.out.println("New row inserted.");
            // Delete the last row
            if (rs.last()) {
                rs.deleteRow();
                System.out.println("Last row deleted.");
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

▶ Exception Handling in JDBC

Exception Handling is a mechanism in programming used to manage errors or unexpected events during program execution. It ensures that the program continues to run smoothly or terminates gracefully without crashing, while addressing the issue that caused the error.

- Importance of Exceptional handling in JDBC
  - Ensures application stability when database errors occur.
  - Helps prevent data corruption (e.g., incomplete transactions).
  - Provides meaningful error messages for developers to debug issues.
  - Automatically cleans up database resources like connections, avoiding memory leaks.
  - Enhances the user experience by providing user-friendly error messages.

▶ Common Exception in JDBC

- SQL Exception
  - Generic exception for SQL-related issues (e.g., syntax errors, connection failure).
- SQLIntegrityConstraintViolationException:
  - Occurs when constraints like primary key, foreign key, or unique constraints are violated. Like trying to insert duplicate primary key
- SQLSyntaxErrorException
  - Triggered by invalid SQL syntax. Like spelling mistake in writing SELECT
- SQLTransientException
  - Indicates temporary issues, such as network glitches or timeouts.
- SQLNonTransientException:
  - Indicates permanent issues, such as using a non-existent table.

▶ Basic Exception Handling in JDBC

```
try {
    // Database operations
} catch (SQLException e) {
    System.err.println("Error: " + e.getMessage());
    System.err.println("SQL State: " + e.getSQLState());
    System.err.println("Error Code: " + e.getErrorCode());
}
```

Specific Exception Handling in JDBC

```
try {
    // Database operations
} catch (SQLIntegrityConstraintViolationException e) {
    System.err.println("Constraint violation: " + e.getMessage());
} catch (SQLSyntaxErrorException e) {
    System.err.println("Syntax error: " + e.getMessage());
} catch (SQLException e) {
    System.err.println("General database error: " + e.getMessage());
}
```

Transaction Management with Exception Handling

```
try (Connection conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/mydb", "user", "password")) {
    conn.setAutoCommit(false); // Start transaction
    Statement stmt = conn.createStatement();
    stmt.executeUpdate("INSERT INTO users (name) VALUES ('John')");
    stmt.executeUpdate("INSERT INTO orders (user_id, amount) VALUES (LAST_INSERT_ID(), 100)");
    conn.commit(); // Commit transaction
} catch (SQLException e) {
    if (conn != null) {
        try {
            conn.rollback(); // Rollback on error
        } catch (SQLException rollbackEx) {
            System.err.println("Rollback failed: " + rollbackEx.getMessage());
        }
    }
    System.err.println("Transaction failed: " + e.getMessage());
}
```

▶ DDL

DDL commands are used to define, modify, or delete the structure of database objects like tables, schemas, indexes, or constraints. These commands change the database schema.

- CREATE
  - Creates a new database, table, index or other database objects.

```
//Creating a database
CREATE DATABASE my_database;


// Creating a table
CREATE TABLE employees (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100),
    department VARCHAR(50),
    salary DECIMAL(10, 2)
);
```

DDL

- ALTER
  - Modifies the structure of an existing table or database object.

```
// Adding a new column to an existing table

ALTER TABLE employees ADD COLUMN hire_date DATE;
```

DDL

- DROP
  - Deletes an existing database, table or database object.

```
//Deleting a table
DROP TABLE employees;

//Deleting a database
DROP DATABASE my_database;
```

DDL

- TRUNCATE
  - Removes all rows from a table without logging individual row deletions.

```
//Removing all data from a table:

TRUNCATE TABLE employees;
```

DDL Example

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;

public class DDLExample {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/mydatabase";
        String user = "root";
        String password = "password";
        try (Connection connection = DriverManager.getConnection(url, user, password);
             Statement statement = connection.createStatement()) {
            // Create a table
            String createTableSQL = "CREATE TABLE users ("
                                    + "id INT PRIMARY KEY AUTO_INCREMENT, "
                                    + "name VARCHAR(50), "
                                    + "email VARCHAR(50))";
            statement.execute(createTableSQL);
            System.out.println("Table created successfully!");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

▶ DML

DML commands are used to manipulate the data within tables. They are concerned with adding, updating, deleting, or retrieving data in a database

- **INSERT**
  - Adds new rows (records) into a table.

```
// Adding a new row into a employees table
INSERT INTO employees (name, department, salary)
VALUES (Ram, 'HR', 40000);

//Adding multiple rows
INSERT INTO employees (name, department, salary)
VALUES
(‘Shyam’, 'IT', 90000),
(‘Sita’, 'Finance', 55000);
```

DML

UPDATE

- Modifies existing data in a table.

```
// Updating the salary of a specific employee

UPDATE employees
SET salary = 30000
WHERE name = 'Ram';
```

DML

DELETE

- Removes specific rows from a table.

```
//Removing a specific row

DELETE FROM employees
WHERE name = 'Shyam';

//Removing all rows from a table

DELETE FROM employees;
```

▶ DML

❖ SELECT

➢ Retrieves data from one or more tables.

```
//Retrieving all columns
 SELECT * FROM employees;

//Retrieving specific columns
SELECT name, salary FROM employees;

//Filtering rows with conditions
SELECT * FROM employees
WHERE salary > 50000;

//Sorting Results
SELECT * FROM employees
ORDER BY salary DESC;
```

SELECT Example

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;

public class SelectQuery {
   public static void main(String[] args) {
       String url = "jdbc:mysql://localhost:3306/java_database";
       String user = "root";
       String password = "Ncit123";
       try {
           Class.forName("com.mysql.cj.jdbc.Driver");
           Connection connection = DriverManagergetConnection(url, user, password);
           System.out.println("Connected to the database!");
           // Create a statement
           Statement statement = connection.createStatement();
           String query = "SELECT * FROM student where roll <14";
           ResultSet resultSet = statement.executeQuery(query);
           while (resultSet.next()) {
               int id = resultSet.getInt("contactID");
               String name = resultSet.getString("name");
               int roll = resultSet.getInt("roll");
               long phone = resultSet.getLong("phone");
               String address = resultSet.getString("address");
               System.out.println("ID: " + id + ", Name: " + name + " , Phone: " + phone + " ,Roll: " + roll);
           }
```

SELECT Example Continue....

```
            resultSet.close();
            statement.close();
            connection.close();
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }
}
```

INSERT Example

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
public class InsertQuery {
   public static void main(String[] args) {
       String url = "jdbc:mysql://localhost:3306/java_database" ;
       String user = "root";
       String password = "Ncit123";
       Connection connection = null;
       PreparedStatement preparedStatement = null;
       try {
           Class.forName("com.mysql.cj.jdbc.Driver" );
           connection = DriverManager.getConnection(url, user, password);
           System.out.println("Connected to the database!" );
           String insertQuery = "INSERT INTO student (name, roll, phone, address) VALUES (?, ?,
?,?)";
           // Step 4: Set parameters using PreparedStatement
           preparedStatement = connection.prepareStatement(insertQuery);
           preparedStatement.setString( 1, "Messi"); // name
           preparedStatement.setInt( 2, 24); // roll
           preparedStatement.setString( 4, "Balkumari"); // address
           preparedStatement.setString( 3, "45545555"); // phone
           int rowsInserted = preparedStatement.executeUpdate();
```

INSERT Example Continue...

```
            if (rowsInserted > 0) {
                System.out.println("A new record was inserted successfully!" );
            }
        } catch (Exception e) {
            System.out.println(e.getMessage());         }
        finally {
            try {
                if (preparedStatement != null) preparedStatement.close();
                if (connection != null) connection.close();
            } catch (Exception e) {
                System.out.println(e.getMessage());
            }
        }
    }
}
```

UPDATE Example

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
public class UpdateQuery {
   public static void main(String[] args) {
       String url = "jdbc:mysql://localhost:3306/java_database" ;
       String user = "root";
       String password = "Ncit123";
       Connection connection = null;
       PreparedStatement preparedStatement = null;
       try {
          connection = DriverManager. getConnection(url, user, password);
          String query = "UPDATE student SET phone = ? WHERE contactiD = ?" ;
          preparedStatement = connection.prepareStatement(query);
           String newPhoneNumber = "980222222"; // New phone number
           int contactID = 5; // ID of the record to update
           preparedStatement.setString( 1, newPhoneNumber); // Set phone number
           preparedStatement.setInt( 2, contactID);         // Set ID
           int rowsUpdated = preparedStatement.executeUpdate();
```

UPDATE Example Continue.....

```
            if (rowsUpdated > 0) {
                System.out.println("Record updated successfully!");
            } else {
                System.out.println("No record found with the specified ID.");
            }

        } catch (Exception e) {
            System.out.println(e.getMessage());
        } finally {
            // Close resources
            try {
                if (preparedStatement != null) preparedStatement.close();
                if (connection != null) connection.close();
            } catch (Exception e) {
                System.out.println(e.getMessage());
            }
        }
    }
}
```

▶ SQL Injection

- SQL Injection is a type of security vulnerability that occurs when an attacker is able to manipulate an application's SQL query by injecting malicious SQL code through user inputs.
- This process allows the attacker to bypass authentication, retrieve or modify sensitive data, delete records or even execute administrative operations on the database.
- The vulnerability arises when user-supplied data is directly inserted into SQL queries without proper validation or escaping, which makes it possible for the attacker to alter the structure of the query and execute unintended SQL commands.

Suppose there is a login form that accepts a username and password. If the user inputs are directly used in a SQL query like this:

```
SELECT * FROM users WHERE username = 'admin' AND password = ' ' OR '1' = '1';

//An attacker might manipulate the password as shown above. Since '1' = '1' is always true, the query would return all records, potentially bypassing authentication or exposing sensitive data.
```

Prepared Statements

- Prepared Statements are a robust and secure way to execute SQL queries in a database.
- They prevent SQL injection by treating user inputs as data rather than executable SQL code.
- Prepared statements separate SQL logic from the data being passed into the query, ensuring that any user-provided input is escaped and treated safely.
- They help to prevent the SQL Injection as user inputs are treated as data, not part of the query structure and also help to escape the special characters.
- Query logic and user input are clearly separated, making code easier to understand and maintain.
- Performance is optimized as the SQL query is parsed and compiled once and can be reused multiple times with different parameters, reducing query parsing overhead.
- Prepared statements support dynamic binding of values, making them flexible for a variety of queries.

Prepared Statements

```
import java.sql.*;
public class PreparedStatementExample {
    public static void main(String[] args) {
        try {
            // Establish the database connection
            Connection  conn = DriverManager.getConnection("dbc:mysql://localhost:3306/mydb", "root", "password");
            // SQL query with placeholders
            String sql = "SELECT * FROM users WHERE username = ? AND password = ?";
            // Create the PreparedStatement
            PreparedStatement  pstmt = conn.prepareStatement(sql);
            // Set the values for the placeholders
            pstmt.setString(1, "admin");  // Set first parameter
            pstmt.setString(2, "securepassword"); // Set second parameter
            // Execute the query
            ResultSet  rs = pstmt.executeQuery();
            // Process the results
            if (rs.next()) {
                System.out.println("Login successful! Welcome, " + rs.getString("username"));
            } else {
                System.out.println("Invalid username or password.");
            }
        } catch (SQLException e) {
            e.printStackTrace();
        } finally {
            // Close resources in reverse order of opening
            try {
                if (rs != null) rs.close();
                if (pstmt != null) pstmt.close();
                if (conn != null) conn.close();
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
    }
}
```

Servlets and JSP

06

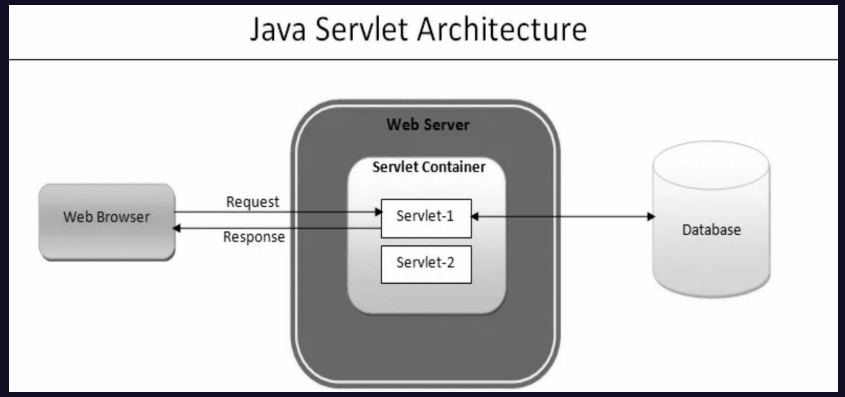
Web Application?

- A web application is a software program that operates on a web server and is accessed by users through a web browser over the internet.
- Unlike traditional desktop applications, which are installed and run locally on a user's computer, web applications rely on the client-server model, where the user's device (the client) interacts with the server to request and receive data.
- The client, typically a web browser, sends requests to the server via the internet, and the server processes these requests, performs the necessary tasks (such as retrieving or updating data) and then sends responses back to the client.
- The responses can be in various forms, such as HTML, JSON, or XML, which the browser renders for the user to see.
- It has become an essential part of how we interact with digital services. It combines the benefits of accessibility, scalability, and maintainability while offering powerful features to meet the needs of users.

Core Concepts of Java based Web Application

Servlet

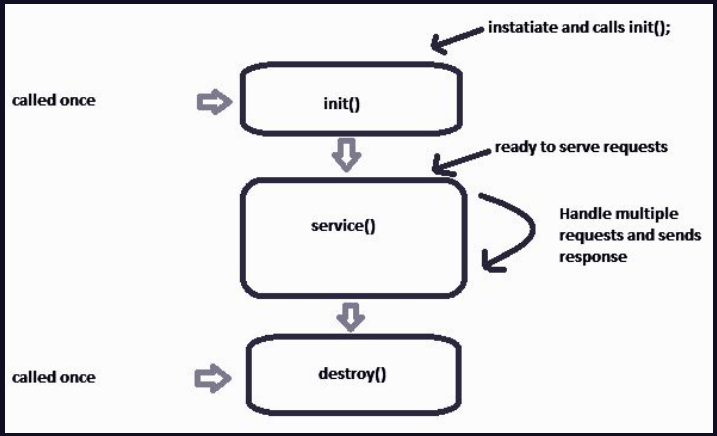
- A Servlet is a server-side technology that operates on a Java-enabled server and is utilized for developing web-based applications.
- It is responsible for processing requests sent by clients (usually browsers) and returning dynamic content.
- It can generate HTML, JSON, XML or any other type of content that can be served to a client.
- When a client (e.g., a browser) sends a request to a web server, the server determines which servlet should handle the request. The servlet container (part of the web server) instantiates the appropriate servlet and invokes its methods to process the request.

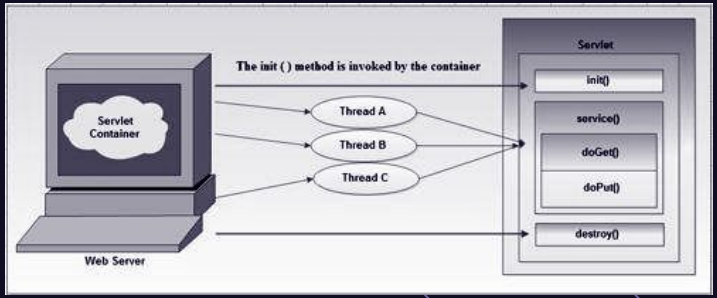


Core Concepts of Java based Web Application

Life Cycle of Servlets

- The Servlet lifecycle is a fundamental concept in Java web development, providing a structured way to manage the creation, execution and destruction of servlets
- Java Servlet life cycle consists of a series of events that begins when the Servlet container loads Servlet and ends when the container is closed down Servlet.
- A servlet container is the part of a web server or an application server that controls a Servlet by managing its life cycle.
- Basically there are three phases of the life cycle, first, the Servlet container loading Servlet in the memory, creating a servlet object and initializes it.
- Second, Servlet object services requests mapped it by the Servlet container.
- Thirdly Servlet container shuts down Servlet and Java Virtual Machine (JVM), which is the environment that runs Java applications, freeing the computer resources.





Core Concepts of Java based Web Application

Init() Method

- The init method in Java servlets plays a critical role in the servlet lifecycle. It is called once when the servlet is first loaded into memory and is used to perform any necessary initialization before the servlet begins handling client requests
- The init method is used to set up resources that the servlet will need throughout its lifecycle which could include initializing database connections, loading configuration parameters, setting up logging frameworks or other startup tasks.
- It ensures that the servlet is fully prepared to handle incoming requests efficiently.

```
public void init(ServletConfig config) throws ServletException {
    super.init(config);
    // Custom initialization logic
    String dbUrl = config.getInitParameter("dbUrl");

}
```

Core Concepts of Java based Web Application

Service() Method

- The service method in the Servlet API plays a critical role in handling client requests and sending responses
- Its primary role is to receive the request and identify the type of HTTP request (GET, POST, etc.), call the corresponding method (doGet, doPost, etc.) to handle the request type and finally respond with objects.
- When a client sends an HTTP request, the servlet container creates HttpServletRequest and HttpServletResponse objects and passes them to the service method.
- The service method checks the HTTP request method (e.g. GET, POST). and then dispatch the appropriate method and after processing , a response is sent back to the client.

```
public void service(HttpServletRequest req, HttpServletResponse res)
throws ServletException, IOException {
    // Logic to handle the request
}
```

Service() Method Request type

doGet() Method

- It handles HTTP GET requests. GET requests are typically used to request data from the server.
- It fetches web pages, querying data, retrieving resources like images or documents.
- In this method data is sent as URL parameters (query strings) so it is limited to a maximum URL length (varies by browser).

```
protected void doGet(HttpServletRequest request, HttpServletResponse
response) throws ServletException, IOException {
    // Code to handle GET request
}
```

Service() Method Request type

doPost() Method

- It handles HTTP POST requests. POST requests are typically used to send data to the server, often resulting in changes on the server side.
- It is used for submitting forms, uploading files, sending data that modifies server state.
- Here, data is sent in the request body, not visible in the URL.
- No size limitation (depends on server configuration).

```
protected void doPost(HttpServletRequest request, HttpServletResponse
response) throws ServletException, IOException {
    // Code to handle POST request
}
```

Service() Method Request type

doPut() Method

- It handles HTTP PUT requests. PUT requests are used to update or create resources on the server.
- Used for updating existing data, uploading or replacing files.
- The request body contains the data to be updated or created.

```
protected void doPut(HttpServletRequest request, HttpServletResponse
response) throws ServletException, IOException {
    // Code to handle PUT request
}
```

Service() Method Request type

doDelete() Method

- It handles HTTP DELETE requests. DELETE requests are used to remove resources from the server.
- Used for deleting a user, removing a file, or cancelling an order
- Request might not have a body; the resource to be deleted is typically identified via the URL or query parameters.

```
protected void doDelete(HttpServletRequest request, HttpServletResponse
response) throws ServletException, IOException {
    // Code to handle DELETE request
}
```

Core Concepts of Java based Web Application

Destroy() Method

- The destroy method is a critical component of the servlet lifecycle in Java, defined by the Servlet API.
- It is invoked by the servlet container (such as Tomcat, Jetty) when a servlet is being removed from service. This typically happens when the server is shutting down or when the servlet is being reloaded or undeployed.
- The primary role of the destroy method is to allow the servlet to release any resources it holds before being permanently removed. This includes closing database connections, stopping background threads, releasing file handles, or any other cleanup tasks.
- The destroy method is called only once during the lifecycle of a servlet, immediately before the servlet instance is garbage collected.

```
public void destroy() {
    // Cleanup code
}
```

Core Concepts of Java based Web Application

Java Server Pages(JSP)

- JavaServer Pages (JSP) is a technology used for developing web pages that support dynamic content.
- It is a part of the Java EE (Enterprise Edition) platform and allows to create web-based applications by embedding Java code within HTML pages.
- JSP pages are compiled into servlets by the web server and executed on the server side before sending the output to the client's browser.
- JSP is designed to simplify the creation of dynamic web pages by separating the business logic from the presentation layer. It allows embedding Java code directly into HTML or XML pages.
- JSP is a higher-level abstraction of servlets, making it easier to write and maintain.

Basic JSP Syntax

- **Directives:** Provides global setting to entire JSP pages.
  - <%@ page contentType="text/html; charset=ISO-8859-1" language="java" %>
- Scriptlets: Allow embedding Java code in the JSP page.

```
<%
int x = 5;
int y = 10;
out.println(x + y);
%>
```

- Accessing data from the request: JSP pages can access request parameters using request.getParameter().

```
<%= request.getParameter("name") %>
```

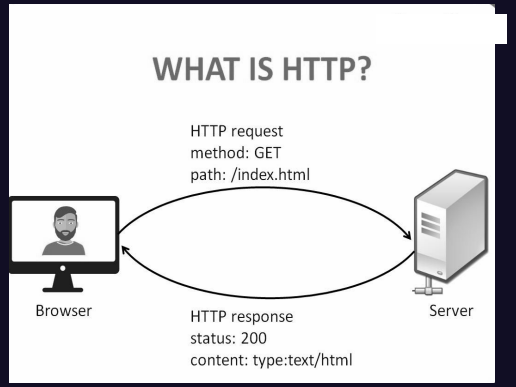
Core Concepts of Java based Web Application

Java Server Faces(JSF)

- JavaServer Faces (JSF) is a framework designed for constructing user interfaces for web applications in the Java platform.
- It utilizes a component-based architecture that allows to create reusable UI components, promoting a modular approach to building web interfaces.
- The framework includes a rich set of predefined UI components, such as input fields, buttons, tables, and forms. These components can be easily integrated into web pages, enabling rapid development and consistent user interface design.
- The component-based approach encourages reuse of UI elements, leading to faster development and maintenance.
- It ensures a consistent look and feel across the application, enhancing user experience with the use of standard components.
- It is well-suited for large-scale enterprise applications that require a consistent and rich user interface.

Http/https

- HTTP (Hypertext Transfer Protocol) is the foundation of data communication on the World Wide Web. It is an application-layer protocol used for transferring hypertext (or other resources) between a client (typically a web browser) and a server over the internet.
- It is a protocol, which defines the rules and conventions for how clients (like web browsers) and servers communicate with each other.
- It is stateless, meaning each request from a client to a server is treated as an independent transaction. The server does not retain information about previous requests.
- It works by sending requests from clients (like browsers) to web servers and receiving responses. These requests can be made to retrieve web pages, submit forms, download files, or perform other operations.
- Request and response are the key components of the http.
- HTTPS is the secure version of HTTP.



Http Methods

- An HTTP method is a type of action that tells the server what to do with the resource (data) that are being requested or sent.
- It is part of the HTTP request sent by the client (usually a web browser) to the server.

Most common methods of http

- **GET**
  - It helps to retrieve data from the server.
  - It is used for loading a webpage, fetching an image, or requesting information from a server.
  - Example: Viewing a website or searching online.
- **POST**
  - Send data to the server to create or update a resource.
  - Used for submitting a form, uploading a file, or creating a new post.
  - Example: Filling out a contact form and submitting it.

Http Methods

- **PUT**
  - Update or replace a resource on the server.
  - It is used for updating an existing resource, like changing a user profile or replacing a document.
  - Example: Editing your user profile information.

- **DELETE**
  - Remove a resource from the server.
  - It is used for deleting a post, removing a file, or deleting an account.
  - Example: Deleting an old blog post.

Http Response

- An HTTP response is the message sent by the server to the client (usually a web browser) after processing an HTTP request.
- It consists of several parts that communicate the status of the request and provide the requested data, if applicable.
- An HTTP response is how the server communicates the result of a client's HTTP request. It contains:
  - A status line (indicating success or error),
    - *HTTP/1.1 200 OK*
  - Headers (providing metadata about the response),
    - *Content-Type: text/html*
    - *Content-Length: 138*
    - *Cache-Control: no-cache*
  - A body (containing the requested data or an empty response).

Http Response Code

- 1xx Informational:
  - 100 Continue: The client should continue with the request.
  - 101 Switching Protocols: The server is switching protocols as requested by the client.
- 2xx Success:
  - 200 OK: The request was successful, and the server has sent the requested data.
  - 201 Created: The request was successful, and a new resource was created.
  - 204 No Content: The request was successful, but there's no content to return.
- 3xx Redirection:
  - 301 Moved Permanently: The resource has been permanently moved to a new URL.
  - 302 Found: The resource is temporarily available at a different URL.
  - 304 Not Modified: The resource has not been modified since the last request, so the client can use its cached version.
- 4xx Client Error:
  - 400 Bad Request: The server could not understand the request due to invalid syntax.
  - 401 Unauthorized: Authentication is required to access the resource.
  - 403 Forbidden: The server understands the request, but it refuses to authorize it.
  - 404 Not Found: The requested resource could not be found on the server.
  - 405 Method Not Allowed: The HTTP method used is not allowed for the requested resource.
- 5xx Server Error:
  - 500 Internal Server Error: The server encountered an unexpected condition that prevented it from fulfilling the request.
  - 502 Bad Gateway: The server, while acting as a gateway or proxy, received an invalid response from the upstream server.
  - 503 Service Unavailable: The server is temporarily unable to handle the request, often due to being overloaded or down for maintenance.

Java Servlet API

- The Java Servlet API is a standard for building web applications in Java.
- It defines classes and interfaces to manage HTTP requests and responses, handle sessions and interact with web clients.
- It is a powerful tool for creating web applications that respond to client requests dynamically.
- It forms the foundation of many Java-based web frameworks and is essential for Java web development.

Java Servlet API

```java
package serverlet;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.io.IOException;
@WebServlet("/hello-servlet")
public class HelloServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)throws ServletException, IOException {
        System.out.println("Get Request called");
        resp.setContentType("text/html");
        resp.getWriter().println("<html>");
        resp.getWriter().println("<head>");
        resp.getWriter().println("<title>Hello Servlet</title>");
        resp.getWriter().println("</head>");
        resp.getWriter().println("<body>");
        resp.getWriter().println("<h1>This is ServletDemo</h1>");
        resp.getWriter().println("</body>");
        resp.getWriter().println("</html>");
    }
    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)throws ServletException, IOException {
        resp.setContentType("text/plain");
        resp.getWriter().println("Hello This is the first string from GET");
    }
}
```

▶ Java Servlet API

- **Key Interfaces:**

- *Servlet: The core interface implemented by all servlets.*
- *HttpServlet: A subclass of GenericServlet that provides HTTP-specific methods like doGet, doPost, doPut, doDelete.*
- *ServletRequest: Encapsulates client request information.*
- *HttpServletRequest: Extends ServletRequest to provide HTTP-specific request information.*
- *ServletResponse: Encapsulates response information.*
- *HttpServletResponse: Extends ServletResponse to provide HTTP-specific response functionality.*
- *ServletConfig: Provides servlet configuration information from the web container.*
- *ServletContext: Allows servlets to interact with their web container and application-level information.*

- **Key Classes:**

- *GenericServlet: An abstract class that implements Servlet and provides basic functionality.*
- *HttpServlet: An abstract class that extends GenericServlet and provides methods to handle HTTP-specific requests.*
-

- **Common Methods:**

- *doGet(HttpServletRequest req, HttpServletResponse resp): Handles HTTP GET requests.*
- *doPost(HttpServletRequest req, HttpServletResponse resp): Handles HTTP POST requests.*
- *init(ServletConfig config): Initializes the servlet; called once when the servlet is first loaded.*
- *service(ServletRequest req, ServletResponse res): Handles requests; typically overridden by doGet or doPost.*
- *destroy(): Called when the servlet is being destroyed; used to release resources.*

Reading and Processing Form

- Reading and processing form data in a servlet involves receiving data sent by a client (typically through an HTML form) and handling it on the server side.
- HTML Form Creation

```
<!DOCTYPE html>
<html lang="en">
<head>
   <title>Servlet Form</title>
</head>
<body>
<form action="formDemo" method="get">
   <label for="username">Username:</label><input type="text" id="username"
name="username"/><br><br>
   <label for="password">Password:</label><input type="password" id="password"
name="password"/><br><br>
   <input type="submit" value="Submit"/>
</form>
</body>
</html>
```

Reading and Processing Form

Servlet to handle form data

```
package serverlet;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.io.IOException;

@WebServlet("/formDemo") // action of html form is same as we have made the webservlet here:formDemo
public class FormDemo extends HttpServlet {
   @Override
   protected void doGet(HttpServletRequest req, HttpServletResponse resp)throws ServletException, IOException {
       String username = req.getParameter("username");
       String password = req.getParameter("password");
       resp.setContentType("text/html");
       resp.getWriter().println("UserName:  " + username);
       resp.getWriter().println("Password:  " + password);
   }
   @Override
   protected void doPost(HttpServletRequest req, HttpServletResponse resp)throws ServletException, IOException
{
       String username = req.getParameter("username");
       String password = req.getParameter("password");
       resp.setContentType("text/html");
       resp.getWriter().println("UserName:  " + username);
       resp.getWriter().println("Password:  " + password);
   }
}
```

▶ Database connectivity through servlet

- Servlets are Java programs that run on a web server and are designed to handle HTTP requests and responses.
- They can interact with a database to retrieve, insert, update or delete data.
- Database connectivity through servlets typically involves using Java Database Connectivity (JDBC), which provides a standard API for connecting to relational databases.

To connect to an SQL database from a servlet, we typically follow following steps

- Import JDBC Libraries:
  - Ensure that we have the JDBC driver for your database installed and added to our project. we'll typically find JDBC drivers provided by the database vendor.
- Load the JDBC Driver:
  - In servlet, load the JDBC driver using Class.forName("driverClassName"). This step is usually required only once during application startup.

▶ Database connectivity through servlet

- Establish Database Connection:
  - Use the DriverManager.getConnection() method to establish a connection to thedatabase. Provide the database URL, username and password as parameters.
- Execute SQL Queries:
  - Once the connection is established, create a Statement or PreparedStatement object to execute SQL queries against the database.
- Process Results:
  - Retrieve and process the results returned by the SQL queries as needed.
- Close Database Resources:
  - After done with the database operation, make sure to close the database resources (Connection, Statement, ResultSet) to release database connections and prevent resource leaks.

Database connectivity through servlet [Select Query]

```java
package servletdb;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.io.PrintWriter;
import java.sql.*;
@WebServlet("/servletjdbc")
public class SelectServlet extends HttpServlet {
   @Override
   protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
       resp.setContentType("text/html");
       PrintWriter out = resp.getWriter();
       String url = "jdbc:mysql://localhost:3306/java_database";
       String user = "root";
       String password = "Ncit123";
       try {
           Class.forName("com.mysql.cj.jdbc.Driver");
           Connection connection = DriverManager.getConnection(url, user, password);
           System.out.println("Connected to the database!");
           // Create a statement
           Statement statement = connection.createStatement();
           String query = "SELECT * FROM student";
           ResultSet resultSet = statement.executeQuery(query);
```

Database connectivity through servlet [Select

```
while (resultSet.next()) {
                int id = resultSet.getInt("contactID");
                String name = resultSet.getString("name");
                int roll = resultSet.getInt("roll");
                long phone = resultSet.getLong("phone");
                String address = resultSet.getString("address");
      // For info, in chapter 5, we use to print locally so System.out.println was use whereas here we use
out.println to print in the browser.
            out.println("ID: " + id + ", Name: " + name + " , Phone: " + phone + " ,Roll: " + roll +"<br><br>");
            }
           resultSet.close();
            statement.close();
            connection.close();

        } catch (ClassNotFoundException e) {
            out.println("Class not found error" +e.getMessage());
        }
        catch (SQLException e) {
            out.println("Database Error"+e.getMessage());
        }
    }

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)throws ServletException, IOException {
        super.doPost(req, resp);
    }
}
```

Database connectivity through servlet [Insert with the Form

```
package servletdb;
import javax.servlet.RequestDispatcher;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;

@WebServlet("/postservlets")
public class PostServlet extends HttpServlet {
   String url = "jdbc:mysql://localhost:3306/java_database";
   String user = "root";
   String password = "Ncit123";
   Connection connection = null;
   PreparedStatement preparedStatement = null;
   @Override
   protected void doGet(HttpServletRequest req, HttpServletResponse resp)throws ServletException, IOException {
       RequestDispatcher dispatcher = req.getRequestDispatcher("/PostServlet.html");
       dispatcher.forward(req, resp);
   }
```

Database connectivity through servlet [Insert with the Form Data]

```
@Override
protected void doPost(HttpServletRequest req, HttpServletResponse resp)throws ServletException, IOException {
//name, roll, phone and address are received from the HTML form
String name = req.getParameter("name");
   String roll = req.getParameter("roll");
   String phone = req.getParameter("phone");
   String address = req.getParameter("address");
   resp.setContentType("text/html");
      // For info, in chapter 5, we use to print locally so System.out.println was use whereas here we use
resp.getWriter().println to print in the browser.
   resp.getWriter().println("Name:  " + name);
   resp.getWriter().println("Roll:  " + roll);
   resp.getWriter().println("Phone:  " + phone);
   resp.getWriter().println("Address:  " + address);
   try {
       Class.forName("com.mysql.cj.jdbc.Driver");
       connection = DriverManager.getConnection(url, user, password);
       System.out.println("Database Connected!");
       String insertQuery ="INSERT INTO student (name, roll, phone, address) VALUES (?, ?, ?,?);"
       preparedStatement = connection.prepareStatement(insertQuery);
       preparedStatement.setString(1, name); // name
       preparedStatement.setInt(2, Integer.parseInt(roll)); // roll
       preparedStatement.setString(4, address); // address
       preparedStatement.setString(3, phone); // phone
       int rowsInserted = preparedStatement.executeUpdate();
       if (rowsInserted > 0) {
           resp.getWriter().println("Data is recorded");
       }
   }
```

Database connectivity through servlet [Insert with the Form Data] Continue

```
catch (Exception e) {
            resp.getWriter().println(e.getMessage());          }
        finally {
            try {
                if (preparedStatement != null) preparedStatement.close();
                if (connection != null) connection.close();
            } catch (Exception e) {
                resp.getWriter().println(e.getMessage());}
        }
    }
}
```

Database connectivity through JSP [Select Query]

```
<%@ page import="java.sql.*" %>
<%@ page import="java.io.*" %>

<html>
<head>
   <title>JDBC Connectivity with JSP</title>
</head>
<body>
   <h2>Database Connection with JSP</h2>

   <%          //JAVA code is integrated here
               String url = "jdbc:mysql://localhost:3306/java_database" ;
               String user = "root";
               String password = "Ncit123";
               try {
                   Class.forName( "com.mysql.cj.jdbc.Driver" );
                   Connection connection = DriverManager.getConnection(url, user, password);
                   System.out.println( "Connected to the database!" );
                   // Create a statement
                   Statement statement = connection.createStatement();
                   String query = "SELECT * FROM student" ;
                   ResultSet resultSet = statement.executeQuery(query);
```

Database connectivity through JSP [Select Query] continue....

```
    while (resultSet.next()) {
         int id = resultSet.getInt("contactID");
         String name = resultSet.getString("name");
         int roll = resultSet.getInt("roll");
         long phone = resultSet.getLong("phone");
         String address = resultSet.getString("address");
         //Here output is printed in browser by out.println rather than System.out.println
         out.println("ID: " + id + ", Name: " + name + " , Phone: " + phone + " ,Roll: " + roll +"<br><br>");
          }
    // Close connection
       resultSet.close();
       statement.close();
       connection.close();

       } catch (ClassNotFoundException e) {
            out.println("Class not found error" +e.getMessage());
       }
       catch (SQLException e) {
             out.println("Database Error"+e.getMessage());
       }

   %>

</body>
</html>
```

Database connectivity through JSP [Insert Query with Form

index.jsp

```
 //this index.jsp contains the form view only.

<!DOCTYPE html>
<html lang="en">
<head>
 <title>Servlet Form</title>
</head>
<body>
<form action="insertquery.jsp" method="post">
 <label for="name">Name:</label><input type="text" id="name" name="name"/><br><br>
 <label for="roll">Roll:</label><input type="text" id="roll" name="roll"/><br><br>
 <label for="phone">Phone:</label><input type="text" id="phone" name="phone"/><br><br>
 <label for="address">Address:</label><input type="text" id="address" name="address"/><br><br>
 <input type="submit" value="Submit"/>
</form>
</body>
</html>
```

Database connectivity through JSP [Insert Query] continue....

insertquery.jsp

```
//this contains the html encoded with java code
<%@ page import="java.sql.*" %>
<%@ page import="java.io.*" %>

<html>
<head>
   <title>JDBC Connectivity with JSP for Form Data</title>
</head>
<body>
   <h2>Database Connection with JSP for Form Data</h2>

   <%     // JAVA code start from here
           String name = request.getParameter("name");
           String roll = request.getParameter("roll");
           String phone = request.getParameter("phone");
           String address = request.getParameter("address");
           String url ="jdbc:mysql://localhost:3306/java_database"
           String user ="root";
           String password ="Nepal#8848";
           Connection connection = null;
           PreparedStatement preparedStatement = null;
               try {
                    Class.forName("com.mysql.cj.jdbc.Driver");
                    connection = DriverManager.getConnection(url, user, password);
                    System.out.println("Connected to the database!");
                    String insertQuery ="INSERT INTO student (name, roll, phone, address) VALUES (?, ?, ?,?);"
```

Database connectivity through JSP [Insert Query] continue....

Insertquery.jsp

```
        //this contains the html encoded with java code
                preparedStatement = connection.prepareStatement(insertQuery);
                preparedStatement.setString1(, name); // name
                preparedStatement.setInt2(, Integer.parseInt(roll)); // roll
                preparedStatement.setString4(, address); // address
                preparedStatement.setString3(, phone); // phone
                int rowsInserted = preparedStatement.executeUpdate();
                if (rowsInserted >0) {
                     out.println("<h3>Registration successful!</h3>");
                  }
                  else{
                        out.println("<h3>Failed to register. Please try again.</h3>");
                  }
              } catch (Exception e) {
                      out.println(e.getMessage());
                       }
              finally {
                  try {
                      if (preparedStatement != null) preparedStatement.close();
                      if (connection != null) connection.close();
                  } catch (Exception e) {
                      out.println(e.getMessage()); }
              }
   %>
</body>
</html>
```

Cookie

- A cookie is a small piece of data stored on the client-side (in the user's browser) by the server.
- Cookie in servlets are a simple but powerful way to store and retrieve small pieces of data
- It can store information such as user preferences, session identifiers or authentication details and the cookie data is sent back to the server with every subsequent HTTP request.
- In servlets, cookie is used to store data in the form of key-value pairs.
- It is particularly useful for remembering user settings, tracking user behavior, or managing sessions without needing a database.

Cookie

Creating and configuring Cookie

- **Create a Cookie Object**: A Cookie object is created by passing a name and a value.
  - **Cookie Name:** The name of the cookie, which acts as the key to access its value.
  - **Cookie Value:** The value associated with the cookie name.
- **Set the Max Age:** The setMaxAge(int seconds) method defines how long the cookie is valid. The max age is specified in seconds:
  - **Positive Value:** The cookie will persist for the specified number of seconds.
  - **Zero (0):** The cookie is immediately removed when the browser is closed.
  - **Negative Value:** The cookie is considered a session cookie, meaning it lasts only for the duration of the browser session and is removed when the browser is closed.
- **Set the Path**: The setPath(String path) method defines the URL path for which the cookie is valid.
  - **"/":** This makes the cookie available for the entire site.
  - If a specific path is set, the cookie will only be sent for requests to that path or its sub-paths.

Cookie

Creating and configuring Cookie

- **Optionally Set Domain:**
  - The setDomain(String domain) method specifies the domain to which the cookie belongs. By default, it is set to the domain of the servlet that set the cookie.
- **Optionally Set Secure Flag:**
  - The setSecure(boolean flag) method ensures the cookie is only sent over secure (HTTPS) connections if set to true.
- **Optionally Set HttpOnly Flag:**
  - The setHttpOnly(boolean flag) method ensures the cookie is inaccessible to JavaScript running in the browser, enhancing security.
- **Add the Cookie to the Response:**
  - The response.addCookie(Cookie cookie) method adds the cookie to the HTTP response, which will be sent back to the client.

Cookie

```
// Create a new cookie with the name "username" and value "ncit"
Cookie userCookie = new Cookie("username", "ncit");

// Set the maximum age of the cookie (in seconds)
// Cookie will expire in 1 hour (3600 seconds)
userCookie.setMaxAge(60 * 60); // 1 hour

// Optionally, make the cookie secure (only sent over HTTPS)
userCookie.setSecure(true);

// Optionally, make the cookie HttpOnly (inaccessible to JavaScript)
userCookie.setHttpOnly(true);

// Add the cookie to the response to send it to the client
response.addCookie(userCookie);
```

Cookie creation and retrieval with Servlet

```
package cookies;
import javax.servlet.ServletException;
import javax.servlet.annotation. WebServlet;
import javax.servlet.http.Cookie;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.io.PrintWriter;

@WebServlet("/cookie")
public class CookieJava extends HttpServlet {
   @Override
   protected void doGet(HttpServletRequest req, HttpServletResponse resp)  throws
ServletException, IOException {
               resp.setContentType( "text/html");
                PrintWriter out = resp.getWriter();
                Cookie userCookie =  new Cookie("username", "ncit");
                userCookie.setMaxAge( 60 * 60); // 1 hour
                 resp.addCookie(userCookie);
```

Cookie creation and retrieval with Servlet [Continue..]

```
                    out.println("<html><body>");
                    out.println("<h1>Cookie Set!</h1>");
                    out.println("<p>Cookie 'username' has been set to 'ncit'.</p>" );
                    out.println("</body></html>");

                     //handling cookie retrieval
                     Cookie[] cookies = req.getCookies();
                     if (cookies != null) {
                      for (Cookie c : cookies) {
                       if ("username".equals(c.getName())) {
                      String username = c.getValue();

                          out.println(username);
            }
        }
    }

  }

}
```

▶ Session

- A session is a mechanism to track user data like login detail across multiple HTTP requests.
- The server assigns each user a unique Session ID (stored as a cookie) to recognize them.
- A session is used to maintain stateful information about a user across multiple requests.
- As HTTP is a stateless protocol, sessions allow web applications to remember user interactions.
- Session is stored on the server and identified via a unique SESSIONID.

Storing and retrieving data using Session in JSP

```
Set Session
<%
   session.setAttribute("username", "ncit");
%>

Retrieve
<%
   String user = (String) session.getAttribute("username");
   out.println("User: " + user);
%>
```

Session Vs Cookie

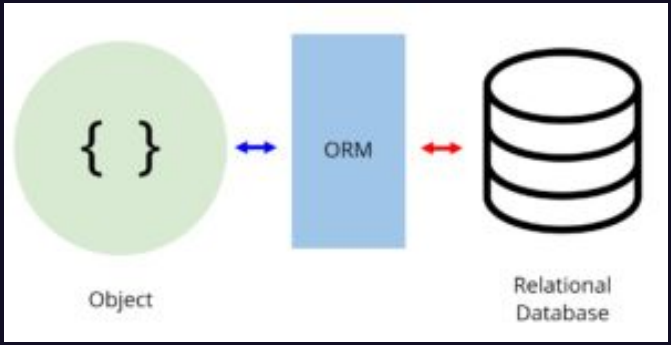
Feature	Session	Cookie
Definition	A server-side storage mechanism that keeps user data across multiple requests.	A client-side storage mechanism that saves small data on the user's browser.
Storage Location	Stored on the server.	Stored on the client's browser.
Security	More secure since data is stored on the server.	Less secure, as users can modify or steal cookies.
Lifetime	Exists until timeout or manual invalidation.	Exists until expiry time set in the cookie.
Access	Can be accessed only by the server.	Can be accessed by both client (JavaScript) and server.
Usage	Used for storing user authentication, shopping cart data, etc.	Used for storing user preferences, session ID, tracking data, etc.
Data Handling	Stores objects and complex data types.	Stores only plain text.
Expiration	Default timeout is 30 minutes (configurable).	Can have an expiry date or last until the browser is closed.
Dependency	Requires a session ID (JSESSIONID) to track users.	Works independently and does not require session tracking.
Best Used For	Sensitive information (e.g., user authentication, cart details).	Non-sensitive information (e.g., user preferences, tracking analytics).

Advanced topics in Java

07

ORM

- Object-Relational Mapping (ORM) is a programming technique that facilitates the interaction between object-oriented programming languages (like Java) and relational databases.
- It acts as a bridge, allowing manipulation of database data using object-oriented paradigms rather than traditional SQL queries.
- ORM frameworks, like Hibernate and JPA (Java Persistence API), map Java objects (entities) to database tables, enabling seamless data manipulation without extensive SQL code.
- ORM in Java simplifies database interactions, enhances productivity, and aligns with object-oriented programming principles.



Advantages of ORM

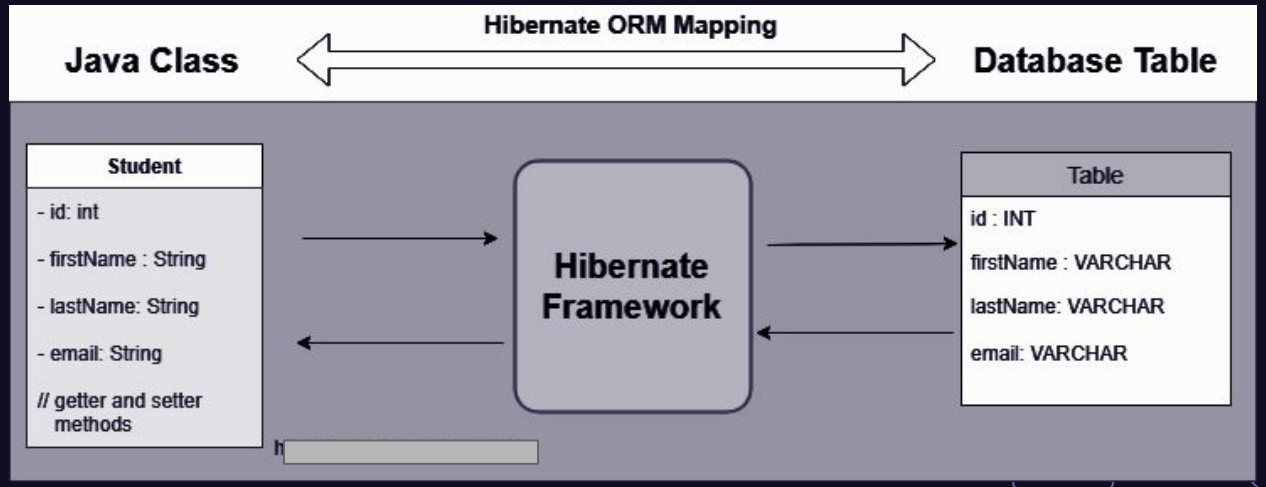
- Productivity:
  - Reduces boilerplate code by automating CRUD operations.
  - Simplifies complex database interactions, allowing more focus on business logic.
- Maintainability:
  - Centralizes database access logic, making the codebase easier to manage and update.
  - Reduces the risk of errors in SQL queries, ensuring more reliable code.
- Database Independence:
  - Allows switching between different databases with minimal code changes, enhancing portability.
  - ORM frameworks provide dialects for various databases, making applications more flexible.
- Transaction Management:
  - Simplifies transaction handling, ensuring data consistency and integrity across operations.

Disadvantages of ORM

- Performance Overhead:
  - The abstraction layer introduced by ORM can lead to performance issues, especially in applications with complex or high-volume database interactions.
- Learning Curve:
  - Understanding and effectively using ORM frameworks requires learning the framework-specific concepts and configurations.
- Complex Queries:
  - For highly complex queries or advanced database features, ORM can become cumbersome or less efficient compared to raw SQL.
- Less Control Over SQL:
  - Automatic SQL generation might not be optimized for every use case, potentially leading to suboptimal query performance.
- Debugging Challenges:
  - Debugging ORM-related issues can be more complex than debugging traditional SQL queries due to the abstraction layer.

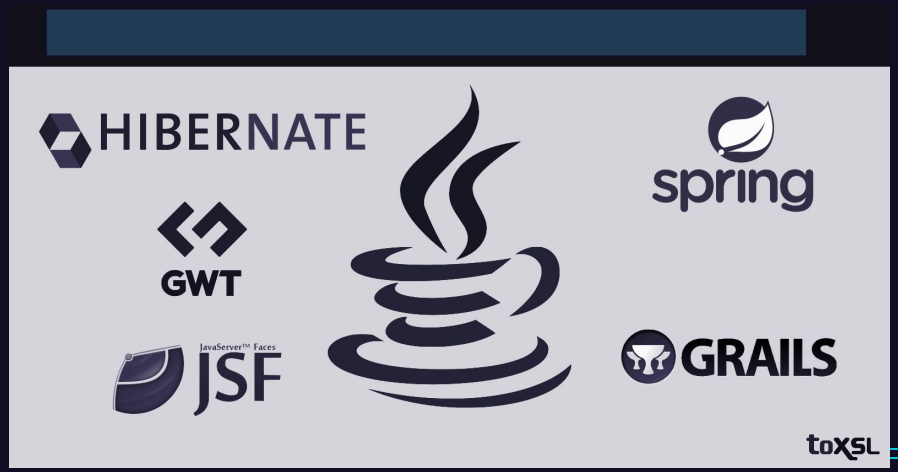
Hibernate

- Hibernate is an open-source, object-relational mapping (ORM) framework for Java.
- It facilitates the mapping of Java objects (POJOs – Plain Old Java Objects) to database tables, enabling seamless interaction between an object-oriented model and a relational database.
- It abstracts away the complexities of interacting with a database and provides a high-level API for performing CRUD (Create, Read, Update, Delete) operations on data.



Web Framework

- A web framework is a software framework designed to aid in the development of web applications, including both client-side and server-side operations.
- It provides a structured foundation with pre-written code, which simplifies and accelerates the process of building web applications.
- It is designed to address the needs of modern web development by offering tools, libraries, and components that help manage repetitive tasks and provide a consistent approach to development.



Advantages of Web Framework

- Ease of Development:
  - Frameworks provide tools and components that reduce the need to write boilerplate code, which speeds up development.
- Security:
  - Many Java web frameworks come with built-in security features, including authentication and authorization,
- Modularization:
  - Frameworks support the separation of concerns (e.g., MVC pattern), which helps keep the codebase organized and easier to maintain.
- Scalability:
  - Java web frameworks like Spring Boot and Play provide features like asynchronous processing and load balancing, enabling scalability for high-traffic applications.
- Community Support:
  - Most Java frameworks, particularly Spring and JSF, have strong community support, extensive documentation, and libraries to simplify development.

Disadvantages of Web Framework

- Complexity:
  - Some frameworks like Spring require a steep learning curve due to their feature-rich nature and extensive configuration.
- Overhead:
  - Some frameworks can introduce performance overhead, particularly when used for small-scale or simple applications.
- Limited Flexibility:
  - While frameworks offer structure and guidelines, they may impose conventions that limit developers' flexibility.
- Complicated Configuration:
  - Some frameworks require extensive configuration, which can increase the setup time and make debugging difficult.

Spring Boot

- The Spring Framework is a comprehensive and modular framework for building Java-based enterprise applications which is one of the most widely used frameworks in the Java ecosystem, offering robust support for developing everything from simple web applications to large, complex enterprise systems.
- Spring Boot is one the subset of Spring Boot Framework which simplifies the development of Spring applications by providing auto-configuration, embedded servers, and convention over configuration principles.
- It makes easier and faster to build Spring-based applications, especially microservices.
- Spring Boot eliminates the need for explicit configuration by automatically configuring the application based on the libraries and dependencies
- It allows to run the application as a stand-alone application without needing an external servlet container like Tomcat or Jetty. It embeds a web server directly into the application, which simplifies deployment.
- It eliminates boilerplate code and XML configuration as It relies on convention over configuration principles

Concurrency in java

- Concurrency refers to the ability of a system to handle multiple tasks at the same time or in overlapping time periods, without necessarily executing them simultaneously.
- It is a fundamental concept in modern programming, especially for handling multiple tasks in real-time.
- It is about dealing with lots of things at once, but not necessarily doing them all at the same time.
- I has the advantage of improved performance with efficient resource utilisation and enhance the user experience.
- It provides significant benefits like improved performance and scalability, but it comes with challenges, particularly in terms of managing thread safety and avoiding deadlocks.

Multithreading in java

- Multithreading in Java is a powerful feature that helps execute multiple tasks concurrently, improving the performance and responsiveness of applications
- Multithreading refers to the ability of a CPU to execute multiple threads concurrently within a single process where Threads are smaller units of execution that share the same memory space and can perform different tasks simultaneously, either truly or in a time-sharing manner.
- It improves the performance and responsiveness of an application by allowing multiple tasks to be carried out concurrently, making better use of available resources, especially on multi-core processors.

Design Pattern

- Design patterns provide standardized and reusable solutions to common design problems in software systems, through which, systems can be built with a clear structure, making them more maintainable, scalable, and flexible.
- Applying design patterns lead to improved software architecture, which is crucial for building robust and high-quality systems.
- Design patterns address common challenges faced during software design, providing pre-defined solutions that have been refined and tested in real-world scenarios.
- Rather than offering ready-made code, design patterns present templates for resolving specific types of problems.

Singleton Pattern

- The Singleton Pattern is one of the most commonly used design patterns which ensures that a class has only one instance and provides a global point of access to that instance.
- This pattern is particularly useful when there is need to control access to shared resources like a database connection, logging service or configuration settings.

```
public class Singleton {
    // Create a private static variable of the same class
    private static Singleton instance;

   // Private constructor to prevent instantiation from other classes
    private Singleton() { }

    // Public static method to get the instance
    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();  // Create the instance when needed
        }
        return instance;  // Return the same instance each time
    }
}
```

Factory Pattern

- The Factory Method Pattern is a creational pattern that defines an interface for creating an object, but allows subclasses to alter the type of objects that will be created.
- This pattern allows a class to delegate the responsibility of creating objects to its subclasses, making the system more extensible and flexible.
- This pattern is needed when a superclass defines the object creation interface, but the decision regarding the type of objects to create is delegated to subclasses and when different subclasses need to be instantiated based on specific configurations or conditions.
- Other side of this pattern is that It adds complexity because of the additional abstract classes and concrete classes.
- The client may not always know which concrete class to choose, requiring additional logic to instantiate the correct subclass.

Factory Pattern

```
// Define the product interface
interface Animal {
    void sound();
}
// Concrete products
class Dog implements Animal {
    public void sound() {
        System.out.println("Bark");
    }
}
class Cat implements Animal {
    public void sound() {
        System.out.println("Meow");
    }
}
// Abstract creator class
abstract class AnimalFactory {
    public abstract Animal createAnimal();
}
// Concrete creator classes
class DogFactory extends AnimalFactory {
    public Animal createAnimal() {
        return new Dog();
    }
}
```

Factory Pattern

```
class CatFactory extends AnimalFactory {
    public Animal createAnimal() {
        return new Cat();
    }
}

// Client code
public class Client {
    public static void main(String[] args) {
        AnimalFactory factory = new DogFactory();
        Animal animal = factory.createAnimal();
        animal.sound();  // Output: Bark
    }
}
```

Abstract Factory Pattern

- The Abstract Factory Pattern is creational pattern that provides an interface for creating families of related or dependent objects without specifying their concrete classes.
- It allows a system to be independent of how its objects are created, composed, and represented.
- It is used when there is need for creating products that belong to different families, but the exact type of product is not known until runtime.
- Also when there is need to ensure that products in a family are used together, making it easier to switch between product families without changing the code.

Abstract Factory Pattern

```
// Define abstract product interfaces
interface Button {
void render();
}
interface Checkbox {
void render();
}
// Concrete products
class WindowsButton implements Button {
public void render() {
System.out.println("Rendering Windows Button");
}
}
class MacButton implements Button {
public void render() {
System.out.println("Rendering Mac Button");
}
}

class WindowsCheckbox implements Checkbox {
public void render() {
System.out.println("Rendering Windows Checkbox");
}
}
```

```
//Define abstract factory interface
interface GUIFactory {
Button createButton();
Checkbox createCheckbox();
}
// Concrete factories
class WindowsFactory implements GUIFactory {
public Button createButton() {
return new WindowsButton();
}
public Checkbox createCheckbox() {
return new WindowsCheckbox();
}
}
class MacFactory implements GUIFactory {
public Button createButton() {
return new MacButton();
}
public Checkbox createCheckbox() {
return new MacCheckbox();
}}
// Client code
public class Client {
public static void main(String[] args) {
GUIFactory factory = new WindowsFactory();
Button button = factory.createButton();
Checkbox checkbox = factory.createCheckbox();
button.render(); // Output: Rendering Windows Button
checkbox.render(); // Output: Rendering Windows Checkbox
}}
```

Advantages of ORM

- Productivity:
  - Reduces boilerplate code by automating CRUD operations.
  - Simplifies complex database interactions, allowing more focus on business logic.
- Maintainability:
  - Centralizes database access logic, making the codebase easier to manage and update.
  - Reduces the risk of errors in SQL queries, ensuring more reliable code.
- Database Independence:
  - Allows switching between different databases with minimal code changes, enhancing portability.
  - ORM frameworks provide dialects for various databases, making applications more flexible.
- Transaction Management:
  - Simplifies transaction handling, ensuring data consistency and integrity across operations.